



INDIVIDUAL ASSIGNMENT

CT118-3-3-ODL

OPTIMIZATION AND DEEP LEARNING

CSSE__CT118-3-3-ODL-L-3_2025-05-01

STUDENT NAME : YEONG MAN WEI
TP NUMBER : TP065870
HAND OUT DATE : 16 MAY 2025
HAND IN DATE : 08 AUGUST 2025
WEIGHTAGE : 100%
LECTURER NAME : MR. RAHEEM MAFAS

INSTRUCTIONS TO CANDIDATES:

- 1. Submit your assignment via Moodle.**
- 2. Students are advised to underpin their answers with the use of references (cited using the 7th Edition of APA Referencing Style).**
- 3. Late submission will be awarded zero (0) unless Extenuating Circumstances (EC) are upheld.**
- 4. Cases of plagiarism will be penalized.**
- 5. You must obtain 50% overall to pass this module.**

Table of Contents

1.0 Introduction.....	11
2.0 Dataset and Algorithm Selection.....	12
2.1 Problem Statement.....	12
2.2 Justification of Dataset Selection.....	14
2.2.1 Data Dictionary	15
2.3 Literature Review	18
2.3.1 Application of Deep Learning in Diabetes Prediction	18
2.3.1.1 Growing Interest in Deep Learning for Diabetes Prediction	18
2.3.1.2 Advantages Over Traditional Methods	18
2.3.1.3 Real-World Applications and Future Directions	19
2.3.2 Performance of Different Deep Learning Models.....	20
2.3.2.1 Multilayer Perceptrons (MLPs).....	20
2.3.2.2 Convolutional Neural Networks (CNNs)	21
2.3.2.3 Transformers.....	22
2.3.2.4 Deep Neural Network (DNNs).....	23
2.3.3 Challenges in Using Deep Learning for Diabetes Prediction	25
2.3.3.1 Data Quality and Imbalance	25
2.3.3.2 Computational Costs	25
2.3.3.3 Model Interpretability.....	25
2.4 Predictive Neural Network Models Selection and Justification.....	26
2.4.1 Deep Neural Networks (DNNs)	26
2.4.2 Convolutional Neural Networks (CNNs)	26
2.4.3 Justification for Model Selection	27
2.5 Summary	28
3.0 Exploratory Data Analysis (EDA).....	30
3.1 Data Understanding.....	30
3.1.1 Dataset Loading and Initial Exploration	30
3.1.2 Graphical View of Variables before Data Preprocessing	34
3.1.2.1 Categorical Variables.....	34
3.1.2.2 Numerical Variables	53
3.1.2.3 Target Variables – Diabetes_binary.....	57

3.1.2.4 Correlation Heatmap	58
3.2 Data Preprocessing	60
3.2.1 Missing Values	60
3.2.2 Outliers	60
3.2.3 Data Standardization	67
3.2.4 Data Transformation.....	69
3.2.5 Check Data Types after Preprocessing.....	70
3.3 Data Splitting and Resampling.....	71
3.3.1 Check Target Variable Count	71
3.3.2 Data Splitting (Train-Test Split)	71
3.3.3 SMOTE Techniques on Training Data.....	72
3.4 Summary of Data Preparation	74
4.0 Model Building.....	76
4.1 Deep Neural Networks Base Model (DNNs)	77
4.1.1 Model Initialization.....	77
4.1.2 Training Process	80
4.1.3 Evaluation Results	82
4.1.3.1 Confusion Matrix and Classification Report.....	83
4.1.3.2 Learning Curve (Accuracy)	85
4.1.3.3 Learning Curve (Loss).....	86
4.1.3.4 ROC and AUC Score.....	87
4.2 Convolutional Neural Network (CNN).....	88
4.2.1 Model Initialization.....	88
4.2.2 Training Process	91
4.2.3 Evaluation Results	93
4.2.3.1 Confusion Matrix and Classification Report.....	95
4.2.3.2 Learning Curve (Accuracy)	97
4.2.3.3 Learning Curve (Loss).....	98
4.2.3.4 ROC and AUC Score.....	99
4.3 Summary of Model Building	100
5.0 Model Tuning	101
5.1 Deep Neural Networks Base Model (DNNs)	101

5.1.1 Hyperparameter Tuning	101
5.1.2 Fine Tuning Model with Best Parameters	106
5.1.3 Evaluation Fine Tuning Results.....	107
5.1.3.1 Confusion Matrix and Classification Report.....	108
5.1.3.2 Learning Curve (Accuracy)	110
5.1.3.3 Learning Curve (Loss).....	111
5.1.3.4 ROC and AUC Score.....	112
5.2 Convolutional Neural Network (CNN).....	113
5.2.1 Hyperparameter Tuning	113
5.1.2 Fine Tuning Model with Best Parameters	119
5.2.3 Evaluation Fine Tuning Results.....	120
5.1.3.1 Confusion Matrix and Classification Report.....	121
5.2.3.2 Learning Curve (Accuracy)	123
5.2.3.3 Learning Curve (Loss).....	124
5.2.3.4 ROC and AUC Score.....	125
5.3 Summary of Model Tuning.....	126
6.0 Model Evaluation and Discussion	127
6.1 Comparison Results of Base and Tuned Model for (DNNs).....	128
6.2 Comparison Results of Base and Tuned Model for (CNN).....	129
6.3 Comparison Results of Base and Tuned Models	130
6.4 Comparison Learning Curve (Accuracy) for Base and Tuned Models	132
6.5 Comparison Learning Curve (Loss) for Base and Tuned Models	133
6.6 Summary of Model Evaluation and Discussion.....	135
7.0 Conclusion	136
7.1 Comparison with Literature	136
7.2 Limitations	137
7.3 Recommendations.....	138
8.0 References	139

List of Figures

Figure 1: Regional Distribution of Diabetes Worldwide.....	12
Figure 2: Global Impact of Diabetes: Key Statistics	12
Figure 3: Flowchart of MLP Model for Diabetes Risk Prediction	20
Figure 4: Convolutional Neural Network (CNN) architecture	21
Figure 5: Transformer Model Architecture for Sequence Prediction.....	22
Figure 6: Deep Neural Network (DNN) for diabetes retinopathy (DR) prediction	23
Figure 7: Source Code of Import Dataset.....	30
Figure 8: Dataset Row Display	30
Figure 9: Dataset Row Display	30
Figure 10: Source Code of Data Exploration.....	31
Figure 11: Output of Dataset Structure	31
Figure 12: Display Columns Output	31
Figure 13: Source Code of Dataset Describe	32
Figure 14: Summary Statistics of the Dataset	32
Figure 15: Summary Statistics of the Dataset	32
Figure 16: Source Code of Descriptive Statistics	33
Figure 17: Descriptive Statistics Table	33
Figure 18: Source Code of Categorical Variables by Using Bar Chart	34
Figure 19: Bar Chart for HighBP	35
Figure 20: Bar Chart for HighChol.....	36
Figure 21: Bar Chart for CholCheck.....	37
Figure 22: Bar Chart for Smoker.....	38
Figure 23: Bar Chart for Smoker.....	39
Figure 24: Bar Chart for HeartDiseaseorAttack	40
Figure 25: Bar Chart for PhysActivity	41
Figure 26: Bar Chart for Fruits	42
Figure 27: Bar Chart for Veggies.....	43
Figure 28: Bar Chart for HvyAlcoholConsump	44
Figure 29: Bar Chart for AnyHealthcare.....	45
Figure 30: Bar Chart for NoDocCost	46

Figure 31: Bar Chart for GenHlth.....	47
Figure 32: Bar Chart for DiffWalk.....	48
Figure 33: Bar Chart for Sex	49
Figure 34: Bar Chart for Age	50
Figure 35: Bar Chart for Education	51
Figure 36: Bar Chart for Income	52
Figure 37: Source Code of Numerical Variables by using Bar Chart	53
Figure 38: Distribution of BMI	54
Figure 39: Boxplot of BMI	54
Figure 40: Distribution of MenHlth	55
Figure 41: Boxplot of MenHlth.....	55
Figure 42: Distribution of PhysHlth.....	56
Figure 43: Boxplot of PhysHlth	56
Figure 44: Source Code of Target Variable	57
Figure 45: Pie Chart of Target Variable	57
Figure 46: Source Code of Correlation Heatmap	58
Figure 47: Correlation Heatmap.....	59
Figure 48: Source Code of Missing Values	60
Figure 49: Output of Missing Values	60
Figure 50: Bar Chart of Education before Preprocessing.....	60
Figure 51: Source Code of Calculating Diabetes Rate	61
Figure 52: Diabetes Rate for Each Education Level	61
Figure 53: Source Code of Education Distribution.....	61
Figure 54: Education Counts Before and After Merge	62
Figure 55: Education Bar Chart Before and After Merge.....	62
Figure 56: Source Code of Display Numerical Variables.....	63
Figure 57: Display Numerical Variables	63
Figure 58: Source Code of Validating Outliers	64
Figure 59: Output for Each Numerical Variables	65
Figure 60: Boxplot of Numerical Variables after Handling Outliers	66
Figure 61: Source Code of Data Standardization	67
Figure 62: Output for Data Standardization	67

Figure 63: Source Code for Numerical Variables after Data Standardization	67
Figure 64: Distribution of Numerical Variables after Data Standardization	68
Figure 65: Source Code of Data Transformation for Age	69
Figure 66: Distribution of Age Groups after Data Transformation	69
Figure 67: Source Code of Data Types After Preprocessing.....	70
Figure 68: Data Types After Preprocessing	70
Figure 69: Source Code of Checking Total Numbers of Diabetes for Each Class	71
Figure 70: Total Numbers of Diabetes for Each Class.....	71
Figure 71: Source Code of Data Splitting	71
Figure 72: Output after Data Splitting.....	71
Figure 73: Source Code of Smote Techniques	72
Figure 74: Source Code of Smote Techniques	72
Figure 75: Output After Smote Techniques	73
Figure 76: Bar Chart for Diabetes Count Before and After Class Imbalance	73
Figure 77: Source Code of Base Model (DNNs).....	77
Figure 78: Source Code of Model Architecture for Base Model (DNNs)	78
Figure 79: Model Architecture for Base Model (DNNs)	78
Figure 80: Source Code of Training Base Model (DNNs)	80
Figure 81: Model Training for Base Model (DNNs).....	80
Figure 82: Model Training for Base Model (DNNs).....	81
Figure 83: Source Code of Model Evaluation for Base Model (DNNs)	82
Figure 84: Source Code of Model Evaluation for Base Model (DNNs)	82
Figure 85: Confusion Matrix for Base Model (DNNs).....	83
Figure 86: Classification Report for Base Model (DNNs)	84
Figure 87: Learning Curve (Accuracy) for Base Model (DNNs)	85
Figure 88: Learning Curve (Loss) for Base Model (DNNs)	86
Figure 89 : ROC and AUC Score for Base Model (DNNs).....	87
Figure 90: Source Code for Convolutional Neural Network Base Model (CNN)	88
Figure 91: Source Code of Model Architecture for Base Model (CNN).....	89
Figure 92: Model Architecture for Base Model (CNN)	90
Figure 93: Source Code for Training Base Model (CNN).....	91
Figure 94: Model Training for Base Model (CNN)	92

Figure 95: Model Training for Base Model (CNN)	92
Figure 96: Source Code of Model Evaluation for Base Model (CNN).....	93
Figure 97: Source Code of Model Evaluation for Base Model (CNN).....	94
Figure 98: Confusion Matrix for Base Model (CNN)	95
Figure 99: Classification Report for Base Model (CNN).....	96
Figure 100: Learning Curve (Accuracy) for Base Model (CNN)	97
Figure 101: Learning Curve (Loss) for Base Model (CNN).....	98
Figure 102: ROC and AUC Score for Base Model (CNN).....	99
Figure 103: Source Code of Model Architecture for Tuned Model (DNNs).....	101
Figure 104: Source Code of Model Architecture for Tuned Model (DNNs).....	102
Figure 105: Source Code of Best Hyperparameters for Tuned Model (DNNs).....	102
Figure 106: Best Hyperparameters for DNNs.....	104
Figure 107: Model Architecture for Tuned Model (DNNs).....	105
Figure 108: Source Code for Fine Tuning Model with Best Parameters (DNNs).....	106
Figure 109: Model Training for Tuned Model (DNNs)	106
Figure 110: Source Code of Model Evaluation for Tuned Model (DNNs).....	107
Figure 111: Source Code of Model Evaluation for Tuned Model (DNNs).....	107
Figure 112: Confusion Matrix for Tuned Model (DNNs)	108
Figure 113: Classification Report for Tuned Model (DNNs).....	109
Figure 114: Learning Curve (Accuracy) for Tuned Model (DNNs).....	110
Figure 115: Learning Curve (Loss) for Tuned Model (DNNs)	111
Figure 116: ROC and AUC Score for Tuned Model (DNNs)	112
Figure 117: Source Code of Tuned Model (CNN).....	113
Figure 118: Source Code of Model Architecture for Tuned Model (CNN)	114
Figure 119: Source Code of Best Hyperparameters for Tuned Model (CNN)	114
Figure 120: Source Code of Model Evaluation for Tuned Model (CNN)	119
Figure 121: Model Training for Tuned Model (CNN).....	119
Figure 122: Model Training for Tuned Model (CNN).....	119
Figure 123: Source Code of Model Evaluation for Tuned Model (CNN)	120
Figure 124: Figure 113: Source Code of Model Evaluation for Tuned Model (CNN)	120
Figure 125: Confusion Matrix for Tuned Model (CNN).....	121
Figure 126: Classification Report for Tuned Model (CNN)	122

Figure 127: Learning Curve (Accuracy) for Tuned Model (CNNs).....	123
Figure 128: Learning Curve (Loss) for Tuned Model (CNN).....	124
Figure 129: ROC and AUC Score for CNN	125
Figure 130: Comparison Learning Curve (Accuracy) for Base and Tuned Models	132
Figure 131: Comparison Learning Curve (Loss) for Base and Tuned Models.....	133

List of Tables

Table 1: Hyperparameter Used to Tune DNNs	103
Table 2: Hyperparameter Used to Tune CNN	115
Table 3: Best Hyperparameters for CNN	116
Table 4: Model Architecture for CNN	117
Table 5: Comparison Results of Base and Tuned Model for (DNNs)	128
Table 6: Comparison Results of Base and Tuned Model for (CNN)	129
Table 7: Comparison Results of Base and Tuned Models	130

1.0 Introduction

This assignment focuses on identifying an optimal deep learning model for predicting high-risk factors for diabetes using historical data. The problem involves applying deep learning techniques to predict and analyse factors that contribute to the risk of diabetes. The assignment is divided into two main sections which are data preparation and pre-processing then followed by the application of deep learning methods with optimization techniques. The first section includes the selection of an appropriate dataset, analysis of its suitability and the justification for using two predictive neural network models. Additionally, an exploratory data analysis (EDA) will be conducted to prepare the data. In the second section, deep learning models will be built, tuned and evaluated with a focus on optimizing hyperparameters. The assignment concludes with a critical evaluation of the final model's performance and a comparison with existing literature in the field.

2.0 Dataset and Algorithm Selection

2.1 Problem Statement

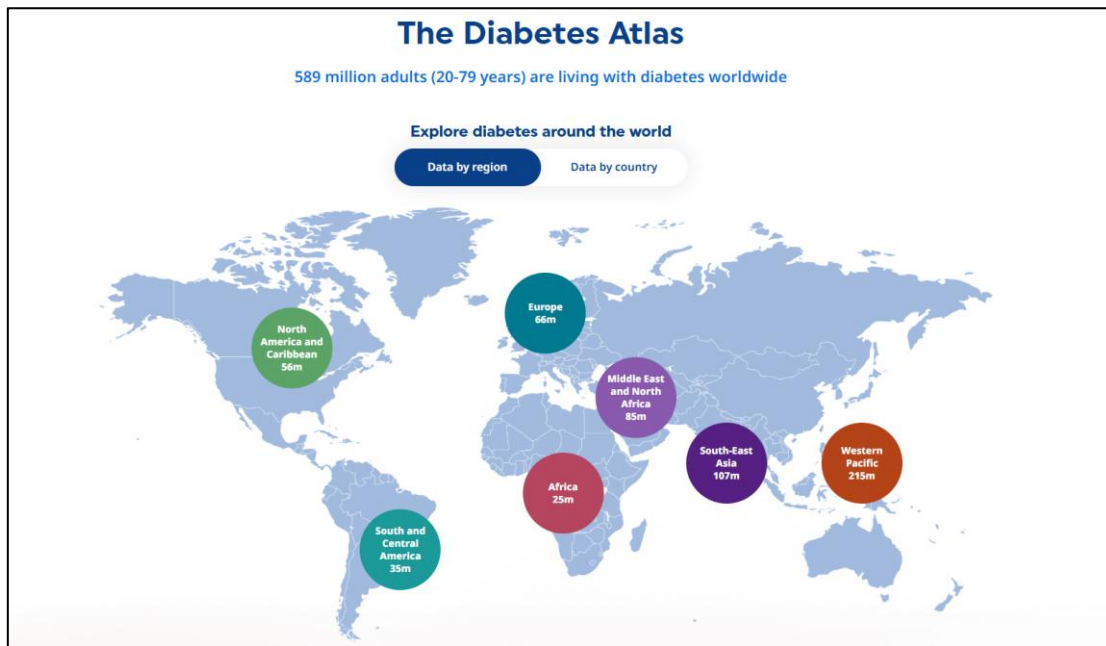


Figure 1: Regional Distribution of Diabetes Worldwide

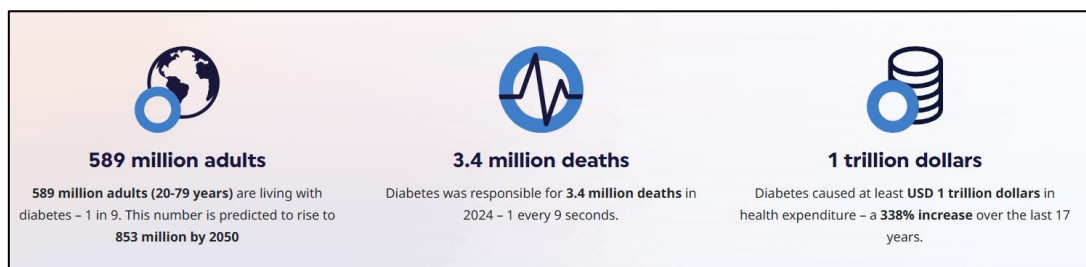


Figure 2: Global Impact of Diabetes: Key Statistics

Diabetes mellitus, particularly Type 2 Diabetes (T2D) has become a global health crisis which has affecting over 589 million adults (20–79 years) worldwide with projections estimating the number will rise to 853 million by 2050 (Diabetes Atlas, 2025). The rapid increase in prevalence is largely driven by factors such as urbanization, chronic health conditions and poor dietary habits. Diabetes is responsible for 3.4 million deaths annually with equating to approximately one death every 6 seconds (Diabetes Atlas, 2025). The global economic burden of diabetes is staggering with global health expenditures linked to diabetes reaching \$1.015 trillion in 2024 and has marking a 338% increase over the last 17 years (Diabetes Atlas, 2025). Research from the International Diabetes Federation (2024) reveals that 43% of adults living with diabetes, approximately 252 million people are undiagnosed and with nearly 90% of them

residing in low- and middle-income countries. The situation is further exacerbated by the fact that 81% of people with diabetes live in low- and middle-income countries.

In Malaysia, the number of adults with diabetes has escalated significantly from 2 million in 2011 and is projected to reach 4.8 million by 2024, eventually increasing to 7.5 million by 2050 (Diabetes Atlas, 2025). Additionally, the economic impact of diabetes in the United States alone reached an estimated \$412.9 billion in 2022 with had including \$306.6 billion in direct medical costs (Hammarlund, 2025). These staggering figures highlight the urgent need for proactive measures to combat this escalating epidemic.

A key challenge in addressing this growing health crisis is the late diagnosis of diabetes, which often results in irreversible damage and severe complications such as cardiovascular disease, kidney failure and neuropathy. Traditional diagnostic methods typically identify diabetes only after significant harm has occurred and was leaving a critical gap in early for proactive detection. Furthermore, conventional risk assessment tools are often limited in their ability to capture the complex interactions between genetic, metabolic and lifestyle factors (Ullah et al., 2022).

Given the escalating prevalence and economic impact of diabetes, early detection and prevention are critical to mitigating these challenges. Predictive modeling using deep learning techniques offers a promising solution. These technologies can analyze health-related data, identify individuals at high risk for diabetes and enable timely medical interventions. By accurately predicting diabetes risk, these models can help reduce the incidence of diabetes-related complications, improve patient outcomes and alleviate the burden on healthcare systems globally.

2.2 Justification of Dataset Selection

The Diabetes Health Indicators Dataset from the Behavioral Risk Factor Surveillance System (BRFSS) has been selected to investigate the problem of diabetes prediction. This dataset is publicly available on Kaggle and contains 236,378 survey responses collected by the Centers for Disease Control and Prevention (CDC) in the United States. The BRFSS is an ongoing state-based telephone survey that has been collecting data since 1984 (National Center for Health Statistics, 2023). It gathers valuable insights into health-related risk behaviors, chronic health conditions and the use of preventive services among U.S. adults. Over the years, it has become one of the largest and most comprehensive sources of data on the health status and behaviors of U.S. adults.

This dataset is well-suited for building predictive models aimed at identifying individuals at high risk for diabetes. It includes a wide range of demographic, behavioral and health-related indicators such as age, gender, body mass index (BMI), smoking habits, blood pressure, physical activity and various chronic conditions, including stroke and heart disease. These indicators are important in understanding the multifaceted risk factors for diabetes and make the dataset a rich resource for developing models that can predict the likelihood of diabetes onset. The diversity of features within the dataset ensures that deep learning models can be developed to account for a broad range of genetic, metabolic and lifestyle factors influencing the development of diabetes.

In addition, the Diabetes Health Indicators Dataset is large to train strong deep learning models. With 236,378 survey responses, it provides a substantial sample size that enables the creation of models capable of generalizing well to real-world data. This large dataset allows for more accurate predictions and better model performance that is important in applications like diabetes risk prediction, where high accuracy is necessary for early diagnosis and intervention. The dataset's sheer size and richness enhance its suitability for predictive analysis to ensure that models developed from it can make reliable and nuanced predictions.

In conclusion, it offers a comprehensive range of relevant features and its large sample size makes it an excellent foundation for developing models that can accurately identify high-risk individuals and improve early diagnosis and intervention strategies for diabetes.

2.2.1 Data Dictionary

The dataset contains the following columns:

Column Name	Description	Data Type	Type
Diabetes_binary	0 = no diabetes, 1 = diabetes	Integer	Categorical (Ordinal)
HighBP	0 = normal blood pressure, 1 = high blood pressure	Integer	Categorical (Binary)
HighChol	0 = no high cholesterol, 1 = high cholesterol	Integer	Categorical (Binary)
CholCheck (Cholesterol Check)	0 = no cholesterol checks in 5 years, 1 = yes cholesterol check in 5 years	Integer	Categorical (Binary)
BMI	Body Mass Index	Float	Numerical (Continuous)
Smoker	Have you smoked at least 100 cigarettes in your entire life? (0 = no, 1 = yes)	Integer	Categorical (Binary)
Stroke	Ever told you had a stroke? (0 = no, 1 = yes)	Integer	Categorical (Binary)
HeartDiseaseorAtta ck	Coronary Heart Disease (CHD) or Myocardial Infarction (MI) (0 = no, 1 = yes)	Integer	Categorical (Binary)
PhysActivity	Physical activity in past 30 days, not including job (0 = no, 1 = yes)	Integer	Categorical (Binary)
Fruits	Consume fruit 1 or more times per day (0 = no, 1 = yes)	Integer	Categorical (Binary)
Veggies	Consume vegetables 1 or more times per day (0 = no, 1 = yes)	Integer	Categorical (Binary)
HvyAlcoholConsu mp	Heavy drinkers (adult men having more than 14 drinks per week, adult women more than 7) (0 = no, 1 = yes)	Integer	Categorical (Binary)

AnyHealthcare	Have any kind of healthcare coverage includes health insurance, prepaid plans such as HMO. (0 = no, 1 = yes)	Integer	Categorical (Binary)
NoDocCost	Was there a time in the past 12 months when you needed to see a doctor but could not because of cost? (0 = no, 1 = yes)	Integer	Categorical (Binary)
GenHlth	General health rating: 1 = excellent, 2 = very good, 3 = good, 4 = fair, 5 = poor	Integer	Categorical (Ordinal)
MentHlth	Number of days during the past 30 days when mental health was not good (0-30 days)	Integer	Numerical (Discrete)
PhysHlth	Number of days during the past 30 days when physical health was not good (0-30 days)	Integer	Numerical (Discrete)
DiffWalk	Do you have serious difficulty walking or climbing stairs? (0 = no, 1 = yes)	Integer	Categorical (Binary)
Sex	Gender: 0 = female, 1 = male	Integer	Categorical (Binary)
Age	13-level age category 1 = '18 to 24', 2 = '25 to 29', 3 = '30 to 34', 4 = '35 to 39', 5 = '40 to 44', 6 = '45 to 49', 7 = '50 to 54', 8 = '55 to 59', 9 = '60 to 64', 10 = '65 to 69', 11 = '70 to 74', 12 = '75 to 79', 13 = '80 or older'	Integer	Categorical (Ordinal)
Education	Education level: 1 = 'Never Attended School', 2 = 'Elementary',	Integer	Categorical (Ordinal)

	3 = 'Junior High School', 4 = 'Senior High School', 5 = 'Undergraduate Degree', 6 = 'Master'		
Income	Income scale: 1-4 = 'Less Than \$10,000', 5 = 'Less Than \$35,000', 6 = 'Less Than \$50,000', 7 = 'Less Than \$75,000', 8 = 'Less Than \$100,000', 9 = 'Less Than \$150,000', 10 = 'Less Than \$200,000', 11 = '\$200,000 or More'	Integer	Categorical (Ordinal)

Table 1: Data Dictionary

2.3 Literature Review

2.3.1 Application of Deep Learning in Diabetes Prediction

The global prevalence of Type 2 Diabetes (T2D) has surged with over 589 million adults affected and this number is expected to rise to 853 million by 2050 (Diabetes Atlas, 2025). Traditional diagnostic methods often detect diabetes too late and is highlighting the need for earlier detection. By applying deep learning models, complex health data can be processed. This is offering a promising solution for the earlier identification of high-risk individuals and enabling proactive intervention and improved patient outcomes.

2.3.1.1 Growing Interest in Deep Learning for Diabetes Prediction

Deep learning is a subset of machine learning that has emerged as a powerful tool in healthcare especially for diabetes prediction. This is due to its ability to analyse complex datasets and identify intricate patterns within them. In diabetes prediction, deep learning models can process large volumes of health-related data such as electronic health records (EHRs), genetic information and lifestyle factors to predict the onset of diabetes with high accuracy (Khokhar et al., 2025). Traditional models often struggle to account for the nonlinear relationships between risk factors but deep learning models can handle these complexities effectively.

A systematic review by Zhu et al. (2024) highlights the growing effectiveness of deep learning models across various diabetes-related tasks such as diagnosis, glucose management and predicting complications. The study found that deep learning techniques often outperform traditional machine learning models in terms of accuracy and reliability. This growing recognition has led to a broader adoption of these models in clinical settings to enhance early diagnosis and intervention strategies for diabetes.

2.3.1.2 Advantages Over Traditional Methods

Traditional predictive models such as logistic regression and decision trees have been widely used in diabetes prediction. However, these models typically fail to capture the intricate, nonlinear relationships within the data such as how obesity exacerbates insulin resistance in patients with high cholesterol. In contrast, deep learning models excel in identifying these complex patterns due to their multi-layered architecture. For example, Convolutional Neural

Networks (CNNs) and Multilayer Perceptrons (MLPs) have shown promising results in predicting diabetes risk factors and outcomes (Bontha et al., 2025).

In a study by El-Bashbishy et al. (2024), deep learning was successfully applied in paediatric diabetes prediction and was achieving an accuracy rate of 99.8%. This high performance demonstrates the potential of deep learning models to assist clinicians in the early and accurate diagnosis of diabetes and thereby facilitating timely interventions.

2.3.1.3 Real-World Applications and Future Directions

The integration of deep learning models into clinical practice is progressing rapidly. These models can analyze diverse datasets such as EHRs, medical imaging and sensor data to provide comprehensive risk assessments for diabetes. For example, deep learning algorithms have been successfully used to predict blood glucose levels which helps in identifying diabetic retinopathy from retinal images and assess the risk of diabetes-related complications (Gong et al., 2025).

The future of deep learning in diabetes prediction holds great promise as models are validated across larger and more diverse populations. So, ongoing research aims to improve model interpretability and personalized treatment strategies by developing explainable AI models. This can enable clinicians to trust and integrate these technologies more effectively into their decision-making processes.

2.3.2 Performance of Different Deep Learning Models

Deep learning has shown significant promise in improving the accuracy of diabetes prediction. Different models with their own unique architectures offer varying strengths in capturing complex relationships within health data.

2.3.2.1 Multilayer Perceptrons (MLPs)

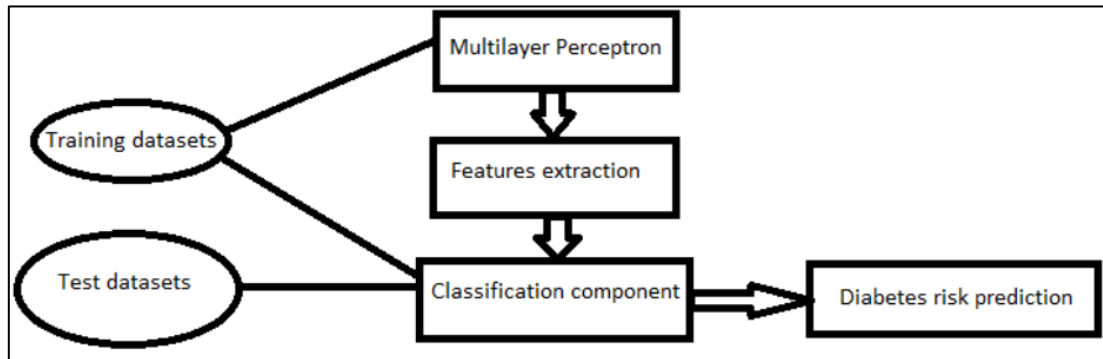


Figure 3: Flowchart of MLP Model for Diabetes Risk Prediction

Multilayer Perceptrons (MLPs) are one of the most widely used deep learning models in diabetes prediction. MLPs is a type of feedforward neural network that are particularly effective for tabular data and is making them well-suited for structured datasets. These models are known for their simplicity and versatility which can effectively handling both numerical and categorical data (Sonia et al., 2023).

As shown in the figure above, the MLP model consists of training and test datasets that are processed through the network. First, features are extracted from the training data and then pass through multiple hidden layers where the classification component makes predictions. Finally, the model outputs the diabetes risk prediction (Fallucchi et al., 2021).

Multi-layer Perceptron (MLP) models have shown strong potential in diabetes prediction especially with datasets containing key features such as age, BMI and blood pressure. Research by Prasetyo et al. (2024) found that the MLP model achieved its highest accuracy of approximately 84.91% with 5 neurons in Hidden Layer 1 and 10 neurons in Hidden Layer 2. This performance underscores the MLP's effectiveness in diabetes risk prediction. While MLPs can suffer from overfitting without proper regularization, the study highlights their reliability and accuracy when appropriately tuned and makes them a valuable tool for diabetes prediction compared to traditional models like Support Vector Machines (SVM).

2.3.2.2 Convolutional Neural Networks (CNNs)

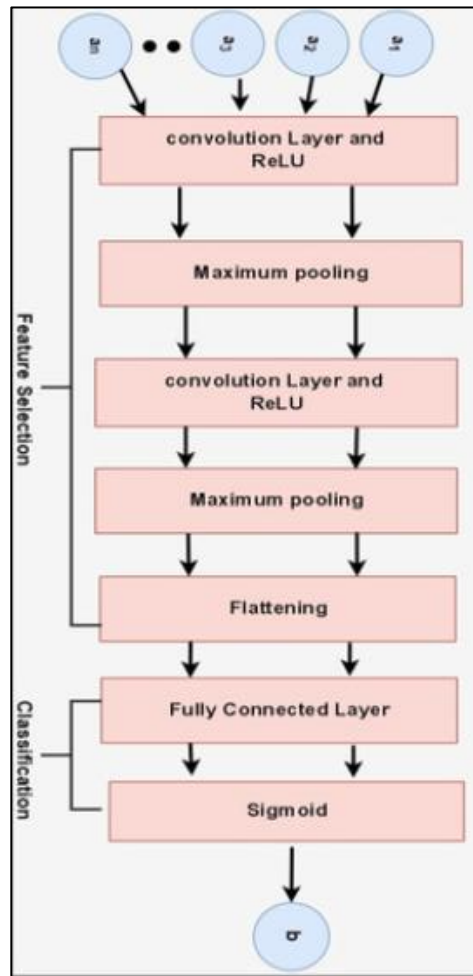


Figure 4: Convolutional Neural Network (CNN) architecture

Convolutional Neural Networks (CNNs), traditionally used for image classification tasks have also been applied to tabular health data in diabetes prediction. CNNs are effective at automatically extracting features from data and useful for capturing complex patterns and relationships between diabetes risk factors.

As shown in the figure above, the CNN architecture begins by processing the input data through multiple convolution layers, where important features are automatically learned (Madan et al., 2022). These layers are followed by pooling layers that help reduce dimensionality and preserve essential information. This makes it easier for the model to identify patterns associated with diabetes risk.

In a study by Ismail et al. (2020), a CNN-based model was proposed for diagnosing and referring common diseases like obesity, high blood pressure and lifestyle factors by extracting regular patterns from correlated health parameters. With a large dataset of 4,759,777 medical

records, the CNN model demonstrated remarkable performance by achieving an accuracy of 95.60% in diagnosing these diseases. This performance was significantly higher than other models and show that CNN's effectiveness in extracting meaningful features from complex medical data. However, it requires substantial computational resources and large datasets to perform optimally.

Besides, Qadeer et al. (2025) used a CNN model for diabetes prediction by achieving an F1-score of 0.80 with an accuracy of 75.32%. The model demonstrated good precision (0.76) and recall (0.80) which suggesting a balanced performance in identifying diabetic individuals. The ROC AUC of 0.81 further indicates its strong ability to distinguish between positive and negative cases. This study highlights the reliability of CNNs in predicting diabetes with its high recall and balanced performance across different data splits.

2.3.2.3 Transformers

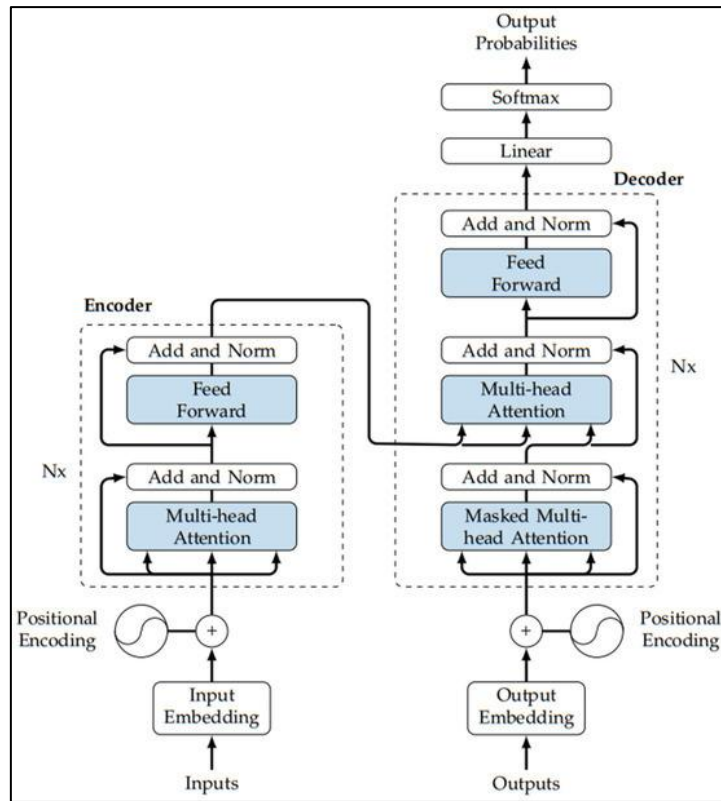


Figure 5: Transformer Model Architecture for Sequence Prediction

Transformer models which are applying self-attention mechanisms to process sequential data by weighing the importance of different input elements and have shown remarkable success in diabetes prediction tasks. Unlike recurrent neural networks, Transformers can capture long-range dependencies in medical data more effectively and is making them particularly suitable for analyzing complex electronic health records (EHRs) and time-series health data.

As shown in the figure above, the Transformer architecture consists of two main components which is the encoder and decoder that are working together to process the input data and generate predictions. The multi-head attention mechanism in the encoder helps the model focus on important relationships between different parts of the input to enhance its ability to handle complex medical data (García-Mosquera et al., 2025).

A study by Li et al. (2020) demonstrated this capability by applying a BERT-based Transformer model to predict poor glycemic control in type 2 diabetes patients using EHRs. Their model achieved an accuracy of 85.7% and an F1-score of 0.82. This is significantly outperforming conventional machine learning approaches like XGBoost. Additionally, further evidence of Transformers' effectiveness comes from Zhu et al. (2024), who developed a Transformer architecture for predicting 30-day hospital readmissions among diabetic patients. Their model attained an impressive AUC-ROC of 0.89 and is surpassing both traditional logistic regression and recurrent neural network approaches.

These studies collectively highlight how Transformer models' ability to process complex and high-dimensional medical data makes them valuable for diabetes-related prediction tasks that are from glycemic control monitoring to hospitalization risk assessment.

2.3.2.4 Deep Neural Network (DNNs)

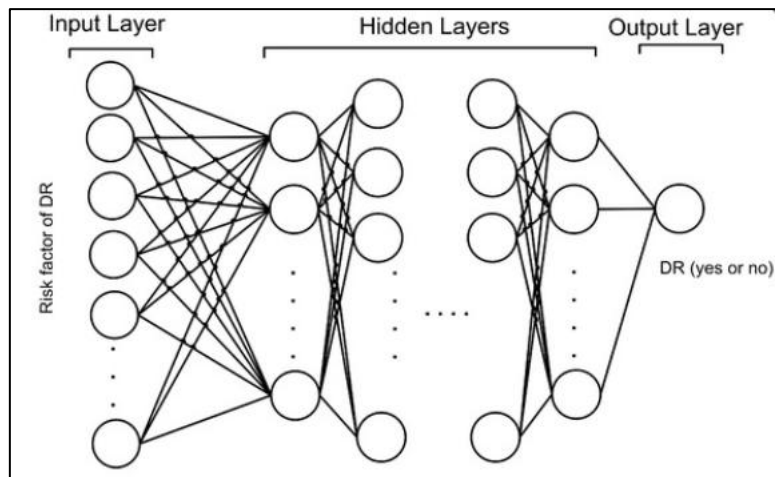


Figure 6: Deep Neural Network (DNN) for diabetes retinopathy (DR) prediction

Deep Neural Networks (DNNs) is a class of neural networks with multiple hidden layers between the input and output layers. DNNs are famous for their capability on capturing the model complex relationships and are effective in learning from large datasets with intricate patterns to make them ideal for tasks like diabetes risk prediction (Mall et al., 2023). DNNs

have shown remarkable success in predicting diabetes-related risks due to their ability to capture intricate relationships between various risk factors such as age, BMI, blood pressure and family history of diabetes.

As shown in figure above, the DNN architecture consists of an input layer, one or more hidden layers and an output layer. So, the hidden layers are apply for the non-linear transformations to the input data which allows the network to learn complex feature representations. The activation functions, such as ReLU in the hidden layers help the model to capture non-linear relationships for enhancing its prediction capability (Alfian et al., 2022).

A study by Alfian et al in 2022 applied a DNN model with Recursive Feature Elimination (RFE) to predict diabetes retinopathy (DR) using various risk factors. The DNN model achieved an accuracy of 82.033%, sensitivity of 76.000% and specificity of 80.389%. The proposed model outperformed traditional machine learning approaches such as k-nearest neighbor (KNN) and support vector machine (SVM) by achieving a higher recall. This has makes DNN useful in medical applications where identifying positive cases is important.

Besides, further evidence of DNN's effectiveness is seen in the study by Prasetyo et al., 2024, where a DNN model was used to predict diabetes-related complications. The model achieved an impressive F1-score of 0.82 and this is surpassing other popular machine learning models like decision trees and XGBoost. These studies have highlighted how DNNs are powerful tools for diabetes-related risk prediction which is providing more accurate and reliable predictions in clinical settings compared to traditional approaches.

2.3.3 Challenges in Using Deep Learning for Diabetes Prediction

2.3.3.1 Data Quality and Imbalance

One of the primary challenges in using deep learning for diabetes prediction is data quality and imbalance. Healthcare datasets often have missing values especially in crucial features like age, BMI and blood pressure. These gaps can hinder the model's ability to capture key patterns in the data and leads to suboptimal performance (Joel et al., 2022). Imbalanced data is another significant issue in diabetes prediction as non-diabetic cases tend to dominate datasets. This imbalance can lead to bias in the models and make it more likely to predict the majority class and less likely to identify at-risk diabetic patients (Abousaber et al., 2025).

2.3.3.2 Computational Costs

Another challenge highlighted in past research is the computational cost of training deep learning models for large datasets. Models like Transformers, CNNs and LSTMs require powerful computing resources such as GPUs or TPUs to train efficiently (Haidarh et al., 2025). These models are computationally expensive and may be impractical for smaller healthcare institutions or resource-limited settings. As deep learning models grow in complexity, the training time also increases and is making it less accessible for broader implementation.

2.3.3.3 Model Interpretability

Model interpretability is a significant barrier to the adoption of deep learning in healthcare, where transparency is important for clinical decision-making. Deep learning models like Transformers and CNNs are often considered "black-box" models because their internal workings are difficult to interpret. This lack of transparency prevents clinicians from understanding how a model arrived at a particular decision and limits its utility in clinical practice (Hassija et al., 202). However, despite progress in the development of interpretability methods, fully transparent deep learning models remain a significant challenge in the healthcare sector.

2.4 Predictive Neural Network Models Selection and Justification

Two deep learning models which is **Deep Neural Networks (DNNs)** and **Convolutional Neural Networks (CNNs)** are selected for predicting diabetes risk using the Diabetes Health Indicators Dataset. The suitability of these models stems from the dataset's large size, diverse features and the complex relationships between them.

2.4.1 Deep Neural Networks (DNNs)

Deep Neural Networks (DNNs) are well-suited for this dataset because they excel at capturing complex and nonlinear relationships between the various features. The Diabetes Health Indicators DNNs are designed to process mixed data types, and this makes them ideal for this dataset that includes both numerical and categorical variables important for predicting diabetes risk.

The large sample size with 236,378 responses in the dataset further supports the use of DNNs as they require a significant amount of data to train effectively. This large sample allows the model to generalize well and ensure a strong performance in predicting diabetes risk based on diverse health indicators.

For example, BMI and age interact in a nonlinear way to influence diabetes risk. DNNs are particularly adept at capturing these complex interactions through multiple hidden layers, where they can learn from relationships such as how high blood pressure (HighBP) combined with obesity (BMI) increases the risk of developing diabetes. The research in the literature review section has shown that DNNs can outperform traditional models like SVM and KNN in similar health prediction tasks due to their ability to process large, complex datasets and identify nuanced relationships between variables (e.g., Prasetyo et al., 2024).

2.4.2 Convolutional Neural Networks (CNNs)

While Convolutional Neural Networks (CNNs) are primarily associated with image processing, their ability to detect local patterns and extract relevant features makes them highly effective for tabular data as well. The Diabetes Health Indicators Dataset contains binary variables like (HighChol, Smoker, Stroke) and ordinal variables like (GenHlth, Age) that may exhibit subtle relationships that CNNs can uncover. By applying convolutional layers to the dataset, CNNs can automatically learn complex feature combinations such as how a combination of high

cholesterol (HighChol) and lack of physical activity (PhysActivity) may significantly increase the likelihood of diabetes.

The advantage of CNNs lies in their ability to process data without the need for detailed feature engineering. They can automatically identify which combinations of features are most important for predicting diabetes risk which demonstrates that CNNs are suitable for complex and high-dimensional datasets.

Although CNNs require substantial computational resources, they have shown remarkable performance in healthcare applications by automatically learning higher-order relationships from large datasets (Madan et al., 2022). For the Diabetes Health Indicators Dataset, CNNs can provide additional accuracy by capturing interactions that traditional models may miss.

2.4.3 Justification for Model Selection

The Diabetes Health Indicators Dataset is suitable for both DNNs and CNNs due to its large size and diverse feature set. The presence of both numerical and categorical data types makes DNNs an ideal choice for modeling the complex and nonlinear interactions between features. DNNs can handle such mixed data efficiently and have proven to work well with datasets containing a large number of variables. Besides, CNNs are beneficial because of their feature extraction capabilities. Given the presence of both binary and ordinal variables, CNNs can automatically detect higher-order interactions between features such as the relationship between smoking and physical health. This ability to automatically extract relevant features reduces the need for feature selection and makes CNNs ideal for this dataset where interactions between features are complex and subtle.

In conclusion, both DNNs and CNNs offer distinct advantages for predicting diabetes risk using the Diabetes Health Indicators Dataset. DNNs are excellent for modeling the complex relationships in mixed data types, while CNNs excel at feature extraction and identifying hidden patterns. Both models could deliver strong performance and improve prediction accuracy while effectively handling the dataset's complexity.

2.5 Summary

In summary, the dataset selected for this study is the Diabetes Health Indicators Dataset which is sourced from the Behavioral Risk Factor Surveillance System (BRFSS). The dataset contains 236,378 survey responses on various health indicators. This dataset includes diverse features such as demographic information, health conditions like high blood pressure and cholesterol, lifestyle factors like smoking habits and physical activity as well as chronic conditions such as stroke and heart disease. The dataset provides a rich resource for developing predictive models to identify individuals at high risk for diabetes.

The problem addressed in this study revolves around the early detection and prediction of diabetes. As the global prevalence of diabetes continues to rise, early intervention is important to prevent severe complications. The traditional diagnostic methods often fail to detect diabetes until significant damage has occurred, and this highlights the importance of predictive modeling. Deep learning techniques offer a promising solution to this issue by analyzing complex datasets and identifying individuals at high risk for diabetes to enable timely medical intervention and improved patient outcomes.

The dataset justification highlights its suitability for diabetes prediction, with its large sample size and diverse set of features. The dataset is well-equipped to model the complex relationships between various health factors and diabetes risk to provide a solid foundation for building accurate predictive models. Additionally, the large number of samples allows for the development of deep learning models that can generalize well to real-world data.

In the literature review, it is emphasized that deep learning has shown significant promise in the healthcare field in diabetes prediction. Models like Deep Neural Networks (DNNs), Convolutional Neural Networks (CNNs) and Transformers have demonstrated superior performance compared to traditional methods in handling complex and non-linear relationships within health data. These models are increasingly being adopted in clinical settings for early diagnosis and intervention, as they can effectively process and analyze large volumes of data to improve the accuracy of predictions.

Finally, DNNs and CNNs has been selected and justified based on the dataset's size and the complex interactions between features. DNNs are ideal for capturing non-linear relationships in mixed data types, while CNNs offer the advantage of feature extraction by automatically identifying important combinations of features for predicting diabetes risk. Both models are

well-suited for the dataset and are expected to provide accurate predictions by effectively handling the complexity of diabetes risk factors.

In conclusion, the Diabetes Health Indicators Dataset is a valuable resource for building predictive models for diabetes risk. By applying deep learning techniques, this study aims to improve the early detection of diabetes, reduce complications and alleviate the burden on healthcare systems globally. The use of DNNs and CNNs offers a comprehensive approach to tackling the challenges posed by diabetes prediction to provide valuable insights for future research and clinical applications.

3.0 Exploratory Data Analysis (EDA)

3.1 Data Understanding

3.1.1 Dataset Loading and Initial Exploration

```
import os
import pandas as pd
from tabulate import tabulate

file_path = "diabetes_binary_health_indicators.csv"

if os.path.isfile(file_path):
    df = pd.read_csv(file_path)
    print("Dataset loaded successfully!")

    print("\nFirst five rows of the dataset:")
    print(tabulate(df.head(), headers='keys', tablefmt='pretty', showindex=False))
else:
    print(f"Error: The file '{file_path}' was not found!")
```

Figure 7: Source Code of Import Dataset

The figure above shows the source code used to load the diabetes_binary_health_indicators.csv dataset. It checks if the file exists, imports the dataset and then displays the first five rows of the data to ensure successful loading.

```
Dataset loaded successfully!

First five rows of the dataset:
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Diabetes_binary | HighBP | HighChol | CholCheck | BMI | Smoker | Stroke | HeartDiseaseorAttack | PhysActivity | Fruits |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | 0 | 1 | 1 | 15 | 1 | 0 | 0 | 0 | 1 |
| 1 | 1 | 0 | 1 | 28 | 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | 33 | 0 | 0 | 0 | 1 | 1 |
| 1 | 0 | 1 | 1 | 29 | 0 | 1 | 1 | 1 | 1 |
| 0 | 0 | 0 | 1 | 24 | 1 | 0 | 0 | 0 | 0 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

Figure 8: Dataset Row Display

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Veggies | HvyAlcoholConsump | AnyHealthcare | NoDocCost | GenHlth | MentHlth | PhysHlth | DiffWalk | Sex | Age | Education | Income |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | 0 | 1 | 0 | 5 | 10 | 20 | 1 | 0 | 11 | 4 | 5 |
| 0 | 0 | 1 | 0 | 2 | 0 | 0 | 0 | 0 | 11 | 4 | 3 |
| 1 | 0 | 1 | 0 | 2 | 10 | 0 | 0 | 0 | 9 | 4 | 7 |
| 1 | 0 | 1 | 0 | 5 | 0 | 30 | 1 | 1 | 12 | 3 | 4 |
| 0 | 0 | 1 | 0 | 3 | 0 | 0 | 1 | 1 | 13 | 5 | 6 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

Figure 9: Dataset Row Display

The figure above shows the output of the first five rows of the dataset to verify that the variables have been imported successfully.

```

# (Rows, Columns)
print("(Rows, Columns)")
print(df.shape , "\n")

# View the data types and missing values
print(df.info(), "\n")

# check columns
print("[Columns]")
print(df.columns, "\n")

```

Figure 10: Source Code of Data Exploration

```

(Rows, Columns)
(236378, 22)

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 236378 entries, 0 to 236377
Data columns (total 22 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Diabetes_binary                       236378 non-null  int64
1   HighBP                               236378 non-null  int64
2   HighChol                             236378 non-null  int64
3   CholCheck                            236378 non-null  int64
4   BMI                                   236378 non-null  int64
5   Smoker                               236378 non-null  int64
6   Stroke                               236378 non-null  int64
7   HeartDiseaseorAttack                 236378 non-null  int64
8   PhysActivity                         236378 non-null  int64
9   Fruits                               236378 non-null  int64
10  Veggies                              236378 non-null  int64
11  HvyAlcoholConsump                   236378 non-null  int64
12  AnyHealthcare                       236378 non-null  int64
13  NoDocCost                           236378 non-null  int64
14  GenHlth                             236378 non-null  int64
15  MentHlth                             236378 non-null  int64
16  PhysHlth                             236378 non-null  int64
17  DiffWalk                             236378 non-null  int64
18  Sex                                  236378 non-null  int64
19  Age                                  236378 non-null  int64
20  Education                            236378 non-null  int64
21  Income                               236378 non-null  int64

```

Figure 11: Output of Dataset Structure

```

[Columns]
Index(['Diabetes_binary', 'HighBP', 'HighChol', 'CholCheck', 'BMI', 'Smoker',
      'Stroke', 'HeartDiseaseorAttack', 'PhysActivity', 'Fruits', 'Veggies',
      'HvyAlcoholConsump', 'AnyHealthcare', 'NoDocCost', 'GenHlth',
      'MentHlth', 'PhysHlth', 'DiffWalk', 'Sex', 'Age', 'Education',
      'Income'],
      dtype='object')

```

Figure 12: Display Columns Output

The figure above shows the output that provides key information about the dataset. It shows that the dataset contains **236,378 rows** and **22 columns**. The function `df.info()` reveals that all columns have **236,378 non-null** entries which indicates there are no missing values in any of the columns. Furthermore, all columns have the data type is `int64` which means that each

column contains integer values. The `df.columns` output lists all the column names such as “Diabetes_binary” and “High_Bp”.

```
df.describe()
```

Figure 13: Source Code of Dataset Describe

	Diabetes_binary	HighBP	HighChol	CholCheck	BMI	Smoker	Stroke	HeartDiseaseorAttack	PhysActivity	Fruits
count	236378.000000	236378.000000	236378.000000	236378.000000	236378.000000	236378.000000	236378.000000	236378.000000	236378.000000	236378.000000
mean	0.294414	0.418558	0.402059	0.963347	28.953579	0.411997	0.038900	0.086548	0.779231	0.621259
std	0.455780	0.493324	0.490315	0.187909	6.552055	0.492196	0.193356	0.281172	0.414766	0.485074
min	0.000000	0.000000	0.000000	0.000000	12.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	1.000000	24.000000	0.000000	0.000000	0.000000	1.000000	0.000000
50%	0.000000	0.000000	0.000000	1.000000	28.000000	0.000000	0.000000	0.000000	1.000000	1.000000
75%	1.000000	1.000000	1.000000	1.000000	32.000000	1.000000	0.000000	0.000000	1.000000	1.000000
max	1.000000	1.000000	1.000000	1.000000	99.000000	1.000000	1.000000	1.000000	1.000000	1.000000

Figure 14: Summary Statistics of the Dataset

AnyHealthcare	NoDocCost	GenHlth	MentHlth	PhysHlth	DiffWalk	Sex	Age	Education	Income
236378.000000	236378.000000	236378.000000	236378.000000	236378.000000	236378.000000	236378.000000	236378.000000	236378.000000	236378.000000
0.962573	0.063737	2.480717	3.937710	3.751297	0.244113	0.477824	7.863930	5.139099	6.927451
0.189807	0.244284	1.029134	7.886506	8.245907	0.429561	0.499509	3.236997	0.946185	2.375450
0.000000	0.000000	1.000000	0.000000	0.000000	0.000000	0.000000	1.000000	1.000000	1.000000
1.000000	0.000000	2.000000	0.000000	0.000000	0.000000	0.000000	5.000000	4.000000	5.000000
1.000000	0.000000	2.000000	0.000000	0.000000	0.000000	0.000000	8.000000	5.000000	7.000000
1.000000	0.000000	3.000000	4.000000	2.000000	0.000000	1.000000	10.000000	6.000000	9.000000
1.000000	1.000000	5.000000	30.000000	30.000000	1.000000	1.000000	13.000000	6.000000	11.000000

Figure 15: Summary Statistics of the Dataset

The figures above provide an overview of the dataset using `df.describe()`. These key statistics such as mean, standard deviation and percentiles offer insights into the central tendency and range of values within each column. This summary allows for a quick assessment of the dataset's structure to highlight the variability, patterns and potential outliers that may warrant further analysis.


```

import pandas as pd
import matplotlib.pyplot as plt

summary = df.describe().transpose()

# Convert count to integers
summary['count'] = summary['count'].astype(int)

# Create figure
fig, ax = plt.subplots(figsize=(14, 10))
ax.axis('off')

table = ax.table(cellText=summary.round(2).values,
                 colLabels=summary.columns,
                 rowLabels=summary.index,
                 cellLoc='center',
                 loc='center',
                 colWidths=[0.20] + [0.12]*7)

# Style adjustments
table.auto_set_font_size(False)
table.set_fontsize(11)

# Make header row bold
for (row, col), cell in table.get_celld().items():
    if row == 0:
        cell.set_text_props(weight='bold')
        cell.set_height(0.06)
        cell.set_edgecolor('lightgray')

# Adjust layout
plt.tight_layout(pad=2.0)
plt.show()

```

Figure 16: Source Code of Descriptive Statistics

	count	mean	std	min	25%	50%	75%	max
Diabetes_binary	236378.0	0.29	0.46	0.0	0.0	0.0	1.0	1.0
HighBP	236378.0	0.42	0.49	0.0	0.0	0.0	1.0	1.0
HighChol	236378.0	0.4	0.49	0.0	0.0	0.0	1.0	1.0
CholCheck	236378.0	0.96	0.19	0.0	1.0	1.0	1.0	1.0
BMI	236378.0	28.95	6.55	12.0	24.0	28.0	32.0	99.0
Smoker	236378.0	0.41	0.49	0.0	0.0	0.0	1.0	1.0
Stroke	236378.0	0.04	0.19	0.0	0.0	0.0	0.0	1.0
HeartDiseaseorAttack	236378.0	0.09	0.28	0.0	0.0	0.0	0.0	1.0
PhysActivity	236378.0	0.78	0.41	0.0	1.0	1.0	1.0	1.0
Fruits	236378.0	0.62	0.49	0.0	0.0	1.0	1.0	1.0
Veggies	236378.0	0.83	0.38	0.0	1.0	1.0	1.0	1.0
HvyAlcoholConsump	236378.0	0.06	0.24	0.0	0.0	0.0	0.0	1.0
AnyHealthcare	236378.0	0.96	0.19	0.0	1.0	1.0	1.0	1.0
NoDocCost	236378.0	0.06	0.24	0.0	0.0	0.0	0.0	1.0
GenHlth	236378.0	2.48	1.03	1.0	2.0	2.0	3.0	5.0
MentHlth	236378.0	3.94	7.89	0.0	0.0	0.0	4.0	30.0
PhysHlth	236378.0	3.75	8.25	0.0	0.0	0.0	2.0	30.0
DiffWalk	236378.0	0.24	0.43	0.0	0.0	0.0	0.0	1.0
Sex	236378.0	0.48	0.5	0.0	0.0	0.0	1.0	1.0
Age	236378.0	7.86	3.24	1.0	5.0	8.0	10.0	13.0
Education	236378.0	5.14	0.95	1.0	4.0	5.0	6.0	6.0
Income	236378.0	6.93	2.38	1.0	5.0	7.0	9.0	11.0

Figure 17: Descriptive Statistics Table

The figure above presents the descriptive statistics table using matplotlib. It provides a clearer visual representation of the data by displaying rounded values for improved readability.

3.1.2 Graphical View of Variables before Data Preprocessing

Exploratory Data Analysis (EDA) will be conducted on the raw dataset using graphical techniques before proceeding with data preprocessing. Both categorical and numerical variables will be examined to identify patterns within the data.

3.1.2.1 Categorical Variables

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Define the categorical columns
categorical_cols = [
    'HighBP', 'HighChol', 'CholCheck', 'Smoker', 'Stroke',
    'HeartDiseaseorAttack', 'PhysActivity', 'Fruits', 'Veggies', 'HvyAlcoholConsump',
    'AnyHealthcare', 'NoDocCost', 'GenHlth', 'DiffWalk', 'Sex', 'Age', 'Education', 'Income'
]

sns.set_style("whitegrid")

# Loop through each categorical column and create a countplot
for col in categorical_cols:
    plt.figure(figsize=(12, 6))
    ax = sns.countplot(data=df, x=col, edgecolor="black", alpha=0.7, color="#D291BC")

    for p in ax.patches:
        ax.annotate(f'{int(p.get_height())}',
                    (p.get_x() + p.get_width() / 2., p.get_height()),
                    ha='center', va='bottom', fontsize=10, color='black')

    plt.title(f"Bar Chart of {col}")
    plt.xlabel(col)
    plt.ylabel("Count")
    plt.grid(False)
    plt.xticks(rotation=0, ha='center', rotation_mode='anchor')
    ax.set_xlim(-0.5, len(df[col].unique()) - 0.5)
    plt.show()
```

Figure 18: Source Code of Categorical Variables by Using Bar Chart

This source code above is used to visualize the distribution of categorical variables in a dataset using bar charts. It loops through each categorical column and creates a countplot for each, which displays the frequency of each category. The code uses seaborn to generate visually appealing bar charts with the count of each category annotated on top of the bars for clarity. It customizes the plots by adding titles, axis labels and adjusting the x-axis labels for better readability. By visualizing the categorical data using bar chart, it helps to identify patterns, trends and distributions in the dataset and this can make it easier to analyze and understand the data before further processing.

HighBP

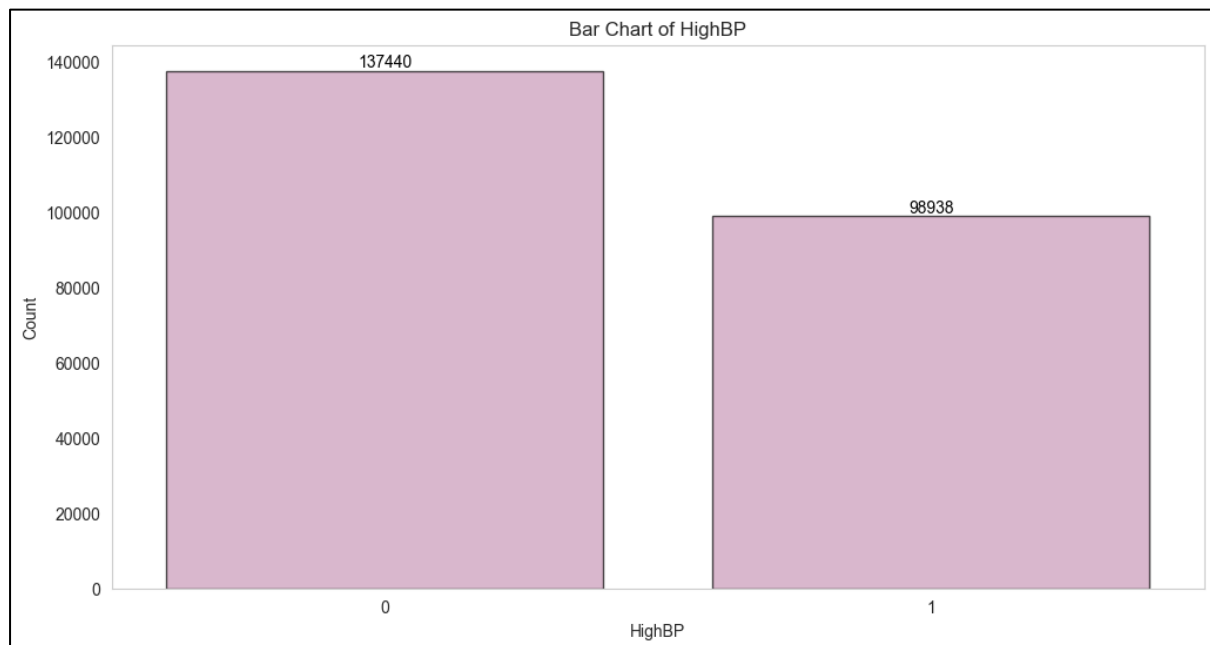


Figure 19: Bar Chart for HighBP

Figure above shows the distribution of individuals with and without high blood pressure (**HighBP**). The chart indicates that a larger portion of the population, 137,440 individuals do not have high blood pressure (0) compared to 98,938 individuals with high blood pressure (1). This could be reflective of general population trends, where high blood pressure is often considered a chronic condition that affects a significant number of individuals. Several factors could contribute to high blood pressure such as lifestyle choices, diet, physical activity and age. It's also possible that preventive healthcare measures such as regular exercise, healthy diets and medication have helped a large portion of the population maintain normal blood pressure levels. The disparity in the numbers may highlight a potential need for increased focus on health education and intervention for those at risk of developing high blood pressure in the future.

HighChol

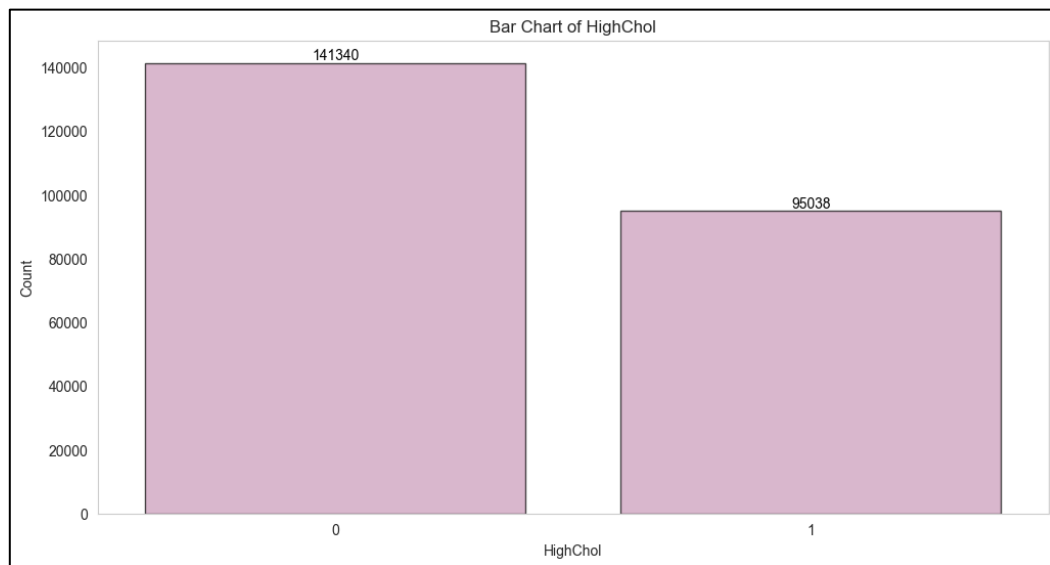


Figure 20: Bar Chart for HighChol

Figure above shows the distribution of individuals with high cholesterol (**HighChol**). It shows that a majority of the population, 141,340 individuals do not have high cholesterol (0), while 95,038 individuals do (1). This suggests that high cholesterol is prevalent in a significant portion of the population but still not the majority. Factors such as diet, exercise, and genetic predisposition can play a role in the variation of cholesterol levels among different individuals.

CholCheck

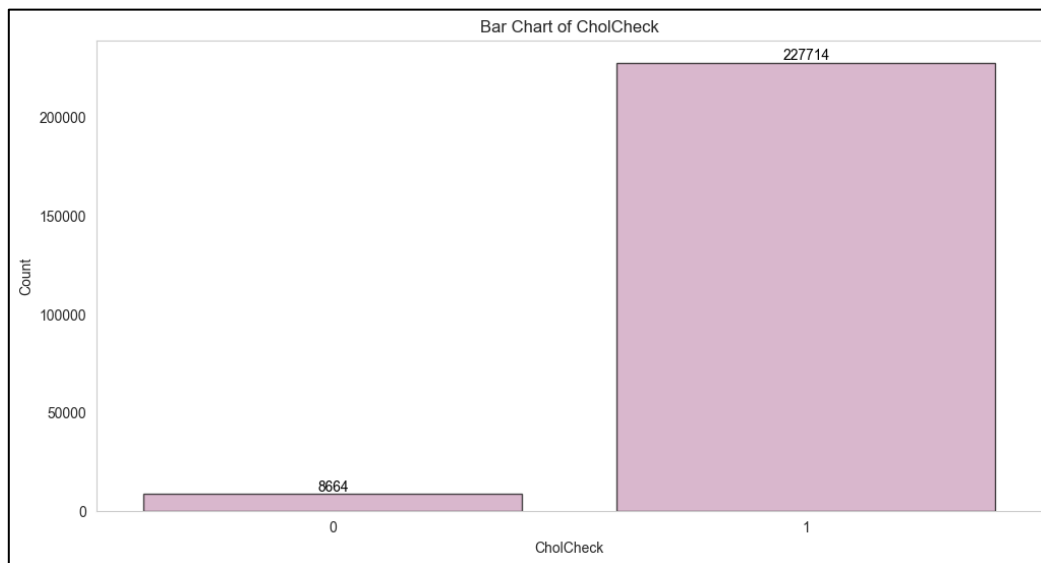


Figure 21: Bar Chart for CholCheck

The second bar chart shows the **CholCheck** variable, which indicates whether individuals have had a cholesterol check. A vast majority, 227,714 individuals, have had their cholesterol checked (1), while only 8,664 individuals have not (0). This suggests that most individuals are actively monitoring their cholesterol levels, likely due to the importance of regular health screenings in preventing cardiovascular diseases. It could also indicate the effectiveness of health programs or policies promoting routine checkups.

Smoker

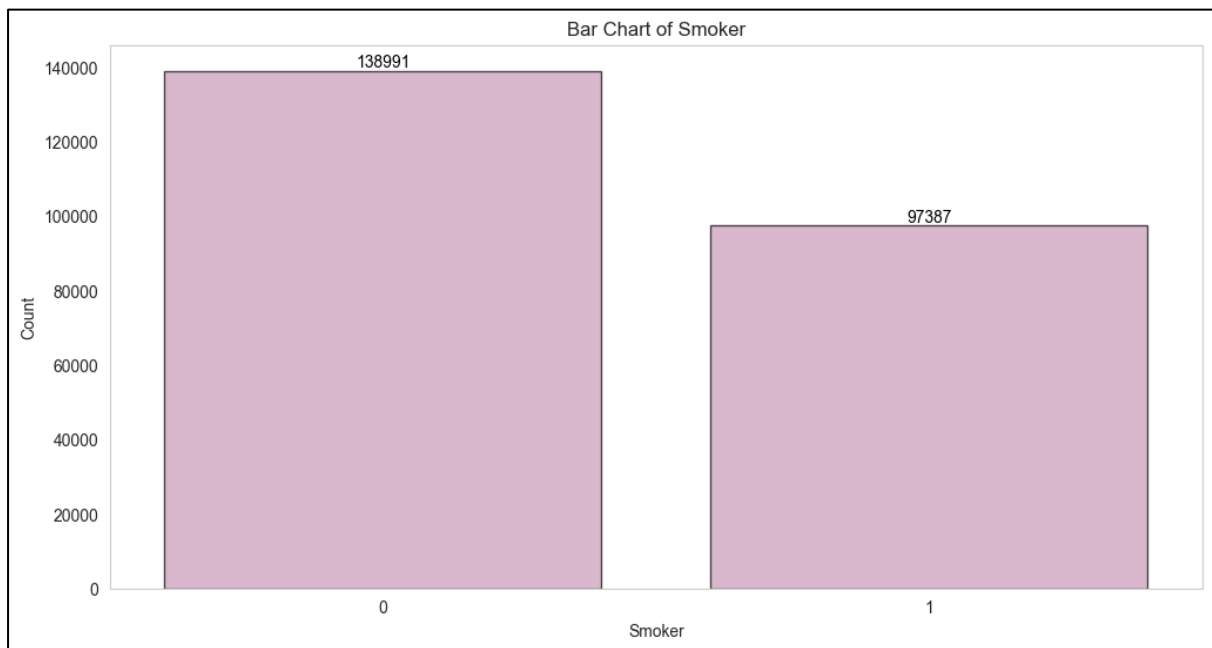


Figure 22: Bar Chart for Smoker

Figure above represents the distribution of smokers in the dataset (**Smoker**). It shows that most of the population, 138,991 individuals do not smoke (0), while 97,387 individuals are smokers (1). This indicates that smoking is relatively less common in this dataset with a significant portion of individuals being non-smokers. This could reflect a positive trend in public health, where smoking rates have been decreasing due to increased awareness and anti-smoking initiatives.

Stroke

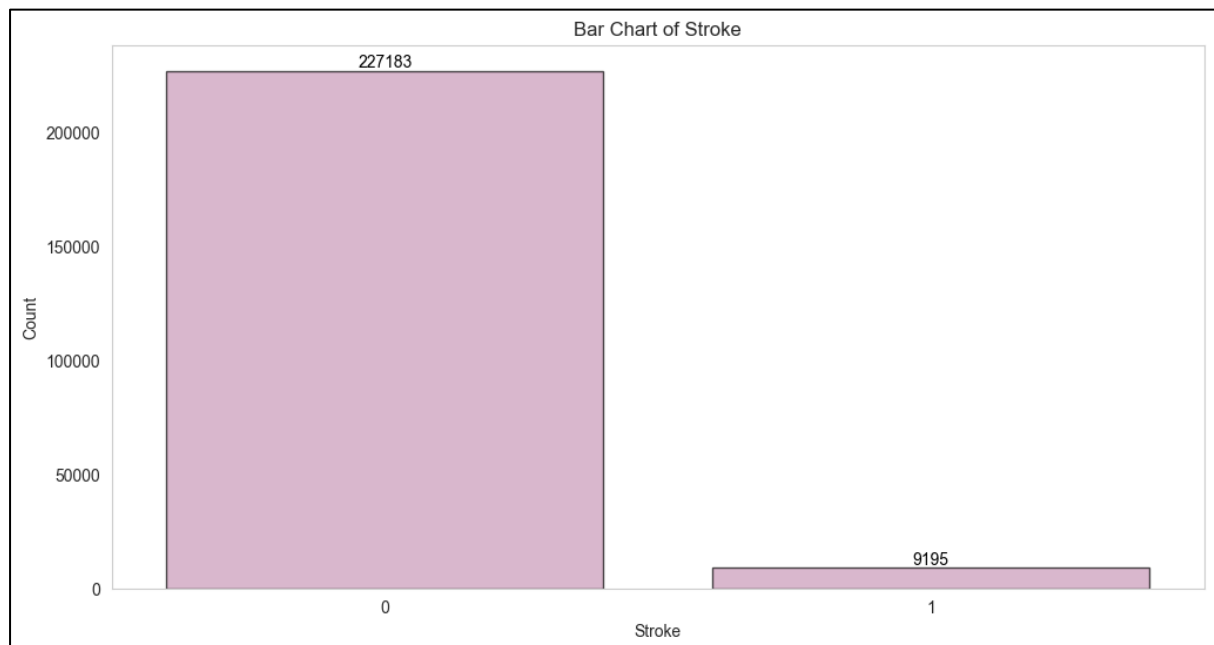


Figure 23: Bar Chart for Smoker

The figure above shows the distribution of the **Stroke** variable with most individuals (227,183) not having had a stroke (0) and fewer individuals (9,195) having experienced one (1). This creates a left-skewed distribution where most data points are clustered around "0," and the stroke cases form a smaller tail. This distribution could be due to the fact that strokes are less common compared to other health conditions and the majority of people do not experience one in their lifetime. Additionally, the dataset may be representative of a population with access to preventive healthcare and lifestyle factors that reduce the risk of stroke which has contributing to the relatively low number of stroke cases.

HeartDiseaseorAttack

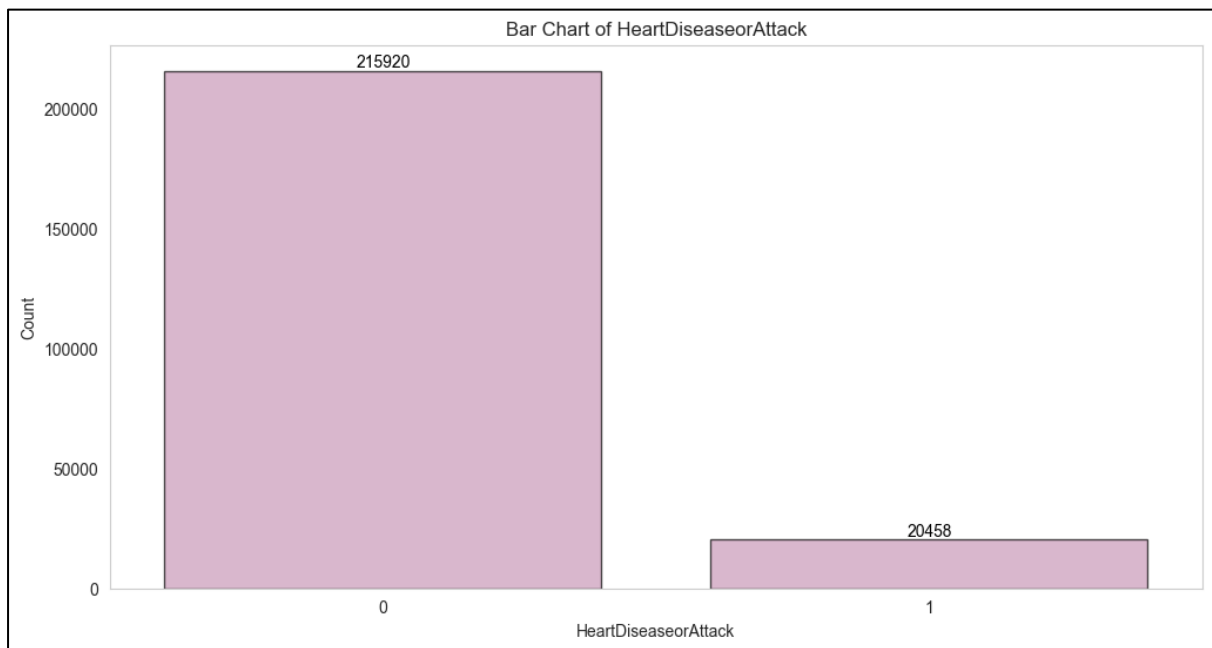


Figure 24: Bar Chart for HeartDiseaseorAttack

Figure above shows the distribution of the **HeartDiseaseorAttack** variable. The chart reveals that most individuals, 215,920 have not experienced heart disease or a heart attack (0) while 20,458 individuals have (1). This creates a right-skewed distribution where most of the data points are clustered around "0" with the stroke cases forming a small tail on the right. This suggests that heart disease and heart attacks are relatively less common and this may reflect the general population or effective health interventions to prevent cardiovascular diseases.

PhysActivity

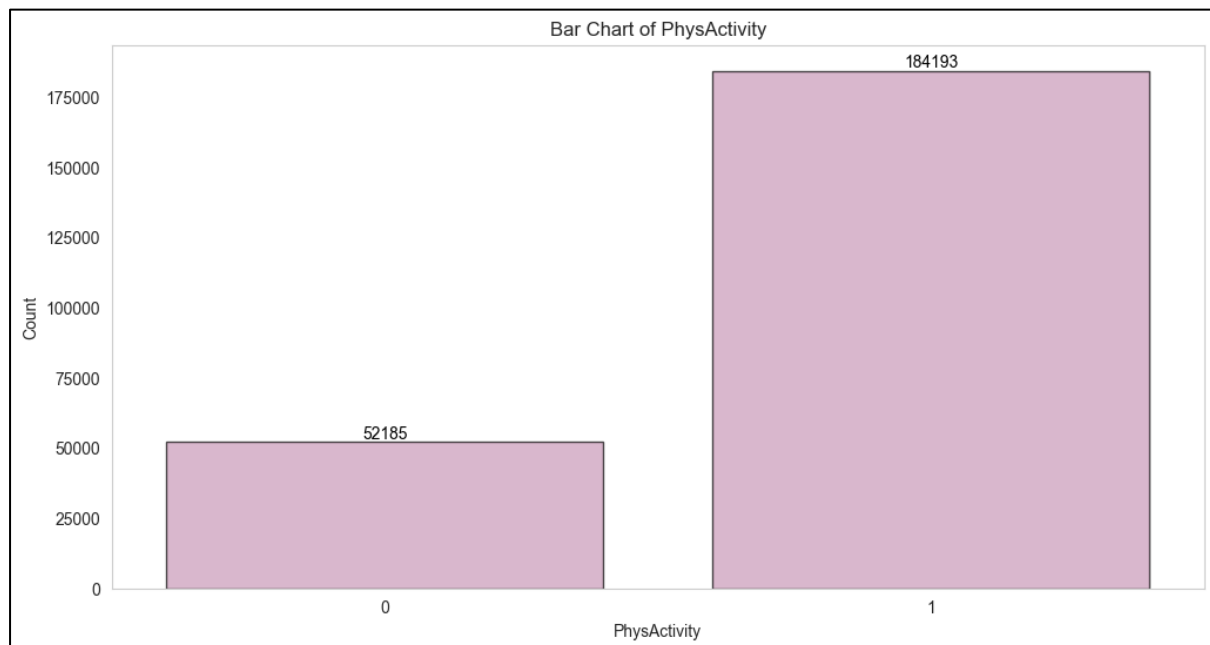


Figure 25: Bar Chart for PhysActivity

Figure above shows the distribution of the **PhysActivity** variable. The chart reveals that 184,193 individuals engage in physical activity (1) while 52,185 individuals do not (0). This suggests that a significant portion of the population in the dataset participates in regular physical activity. It could reflect positive health behaviour where individuals are motivated to maintain an active lifestyle which could potentially influenced by public health campaigns or access to exercise programs. The relatively lower count of individuals who do not engage in physical activity may indicate growing awareness of the benefits of staying active such as improved cardiovascular health and better overall well-being. Furthermore, this distribution may point to the importance of promoting physical activity as a preventive measure for various health issues in the population.

Fruits

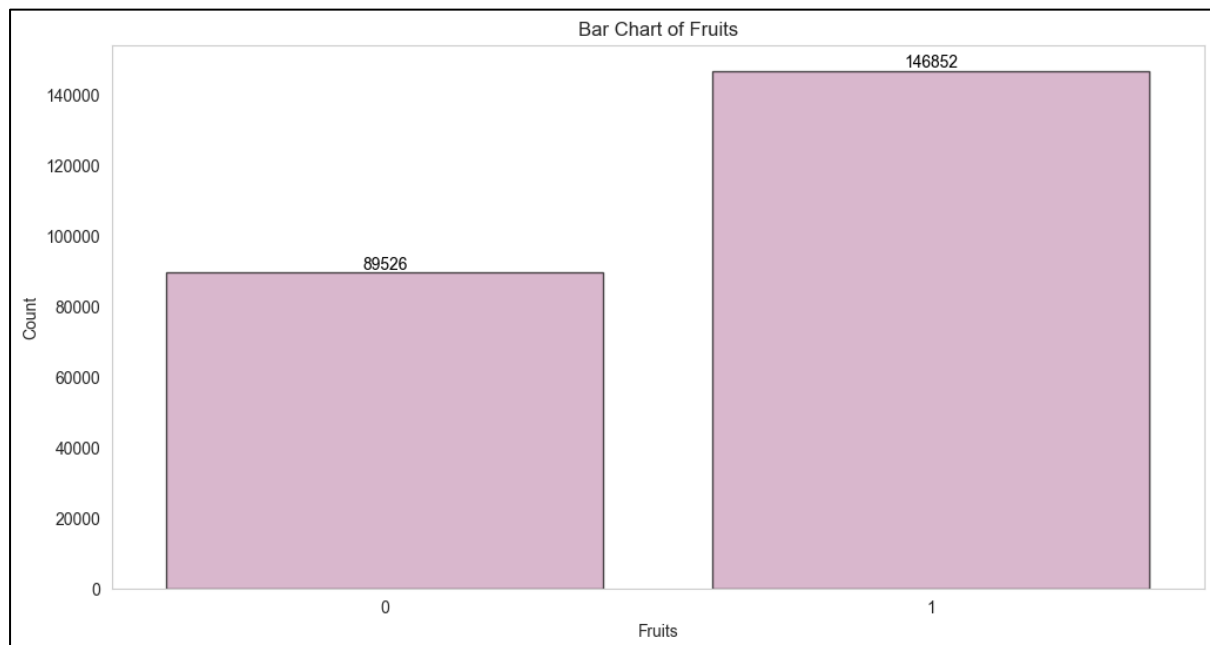


Figure 26: Bar Chart for Fruits

Figure above shows the distribution of the **Fruits** variable. The chart shows that 146,852 individuals consume fruits regularly (1) while 89,526 do not (0). This distribution indicates that fruit consumption is relatively common with more individuals making fruits a part of their daily diet. It suggests a positive trend towards healthier eating habits which could possibly influenced by public health campaigns or growing awareness of the benefits of fruits for maintaining a balanced diet. The relatively larger number of individuals consuming fruits could be reflective of a population that is more health-conscious or educated about the importance of nutrition.

Veggies

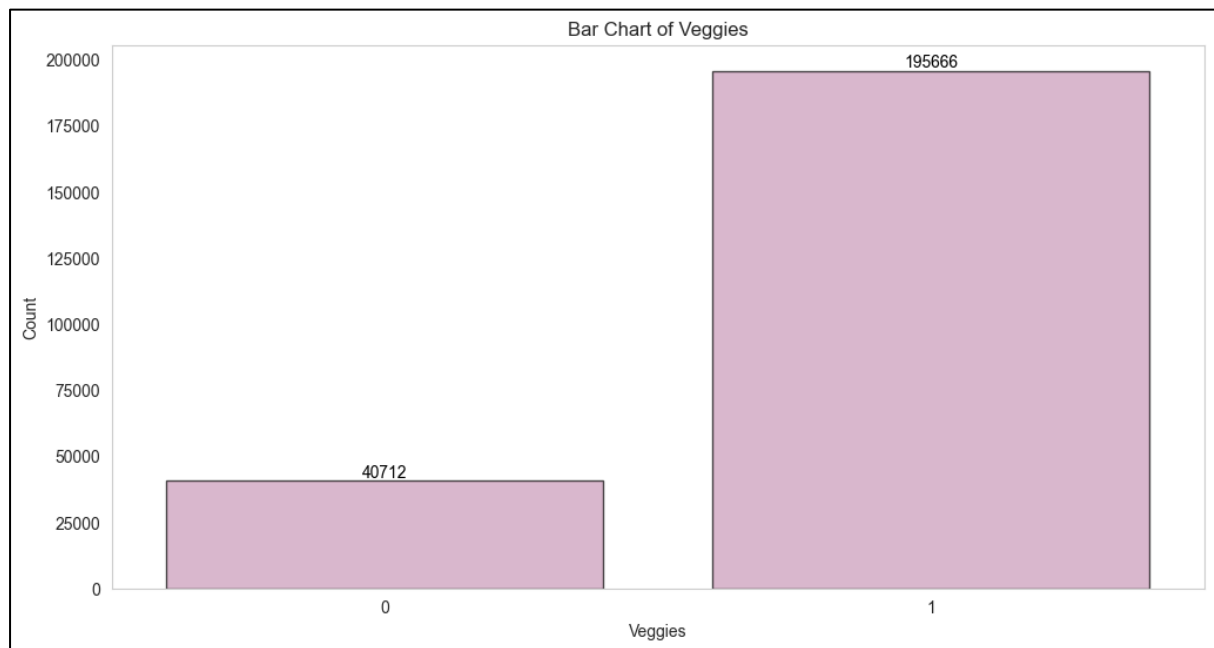


Figure 27: Bar Chart for Veggies

Figure above shows the distribution of the **Veggies** variable. The bar chart reveals that 195,666 individuals consume vegetables regularly (1) while 40,712 do not (0). This suggests that a large portion of the population is following healthy eating habits as regular vegetable consumption is linked to better nutrition, improved digestive health and reduced risk of chronic diseases. The significant number of individuals consuming vegetables reflects the growing awareness of the importance of a balanced diet for overall well-being.

HvyAlcoholConsump

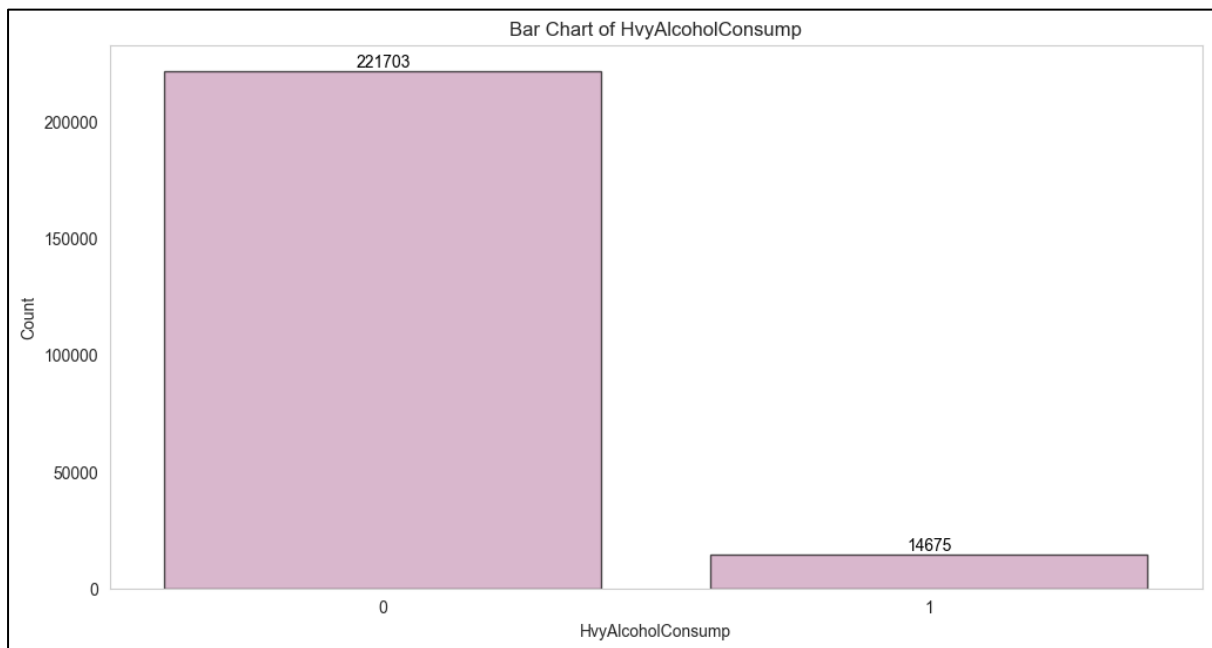


Figure 28: Bar Chart for HvyAlcoholConsump

Figure above shows the distribution of the **HvyAlcoholConsump** variable. The chart shows that 221,703 individuals do not consume alcohol heavily (0) while 14,675 individuals report heavy alcohol consumption (1). This indicates that most of the population avoids heavy drinking which has reflecting health-conscious behaviors. The relatively lower number of individuals with heavy alcohol consumption emphasizes the success of such efforts or the societal preference for moderation.

AnyHealthcare

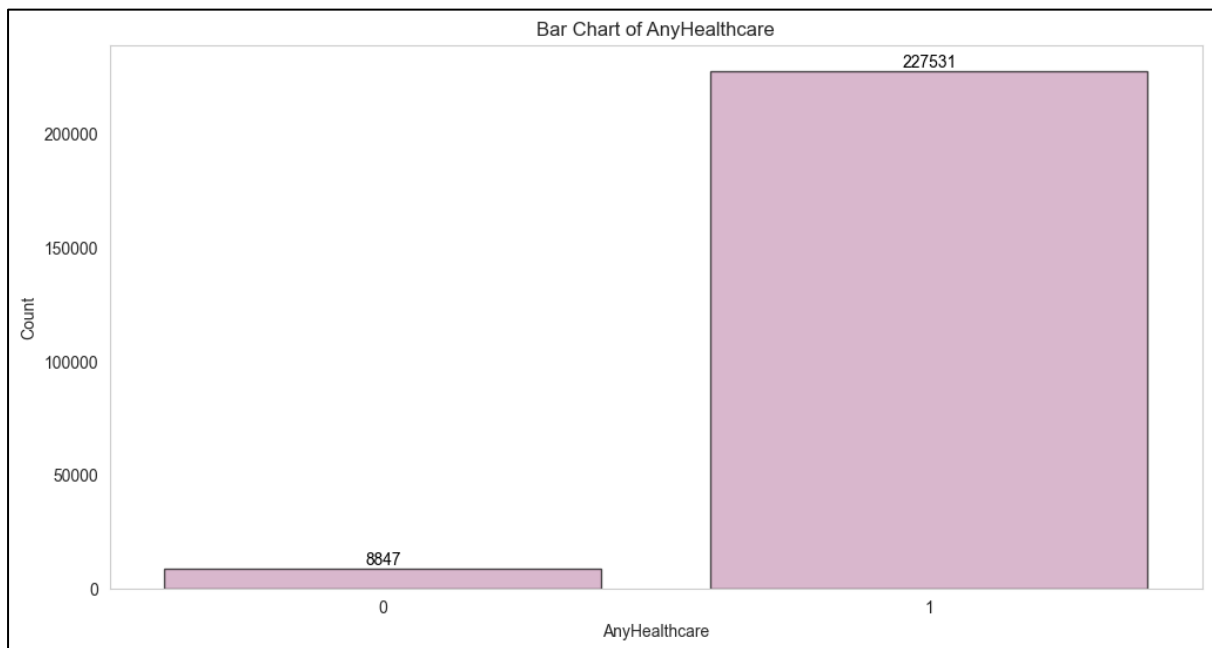


Figure 29: Bar Chart for AnyHealthcare

Figure above shows the distribution of the **AnyHealthcare** variable. The chart indicates that 227,531 individuals have access to healthcare services (1) while only 8,847 individuals do not (0). This suggests that the majority of the population has access to healthcare which could be due to effective healthcare policies, public health programs or insurance coverage and this ensures that most people can receive medical care when needed. The widespread access to healthcare is an encouraging indicator of the quality of the healthcare system in place.

NoDocCost

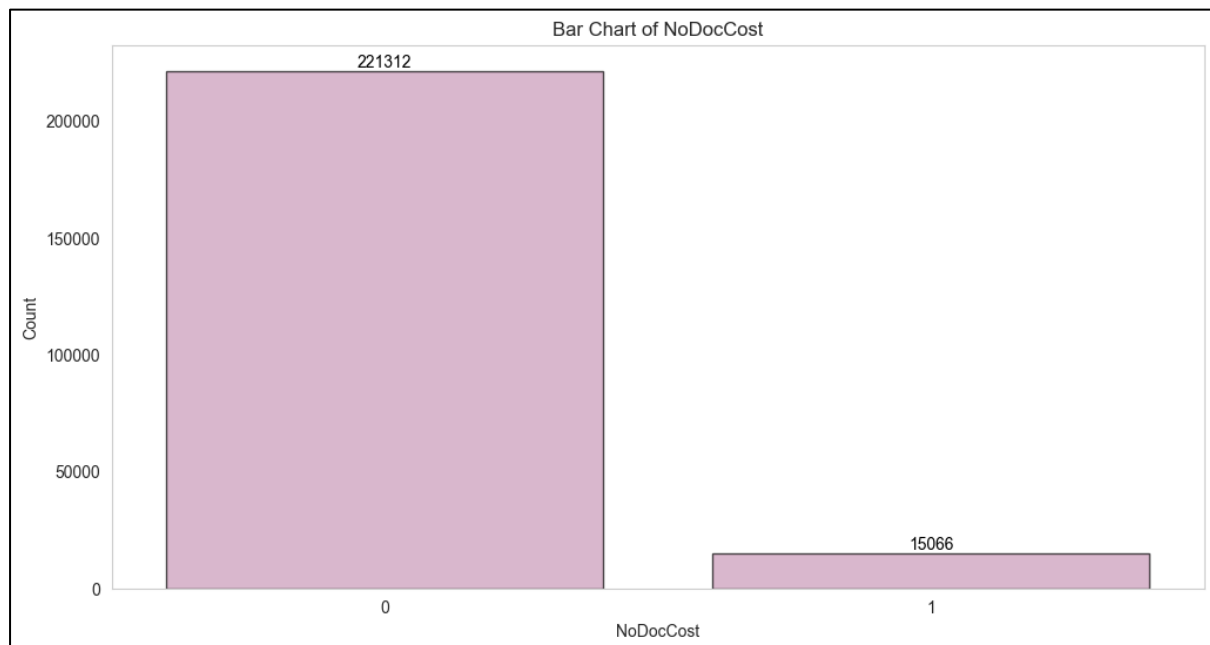


Figure 30: Bar Chart for NoDocCost

Figure above shows the distribution of the **NoDocCost** variable. The chart reveals that the majority of individuals, 221,312 did not report being unable to see a doctor due to cost (0) while 15,066 individuals reported not being able to see a doctor because of cost (1). This suggests that most individuals were able to access healthcare services when needed, indicating good access to medical care for most of the population. However, the smaller group who reported being unable to see a doctor due to cost highlights an important issue related to healthcare affordability. This is pointing to the need for more accessible healthcare policies or financial support for those facing economic barriers to medical care.

GenHlth

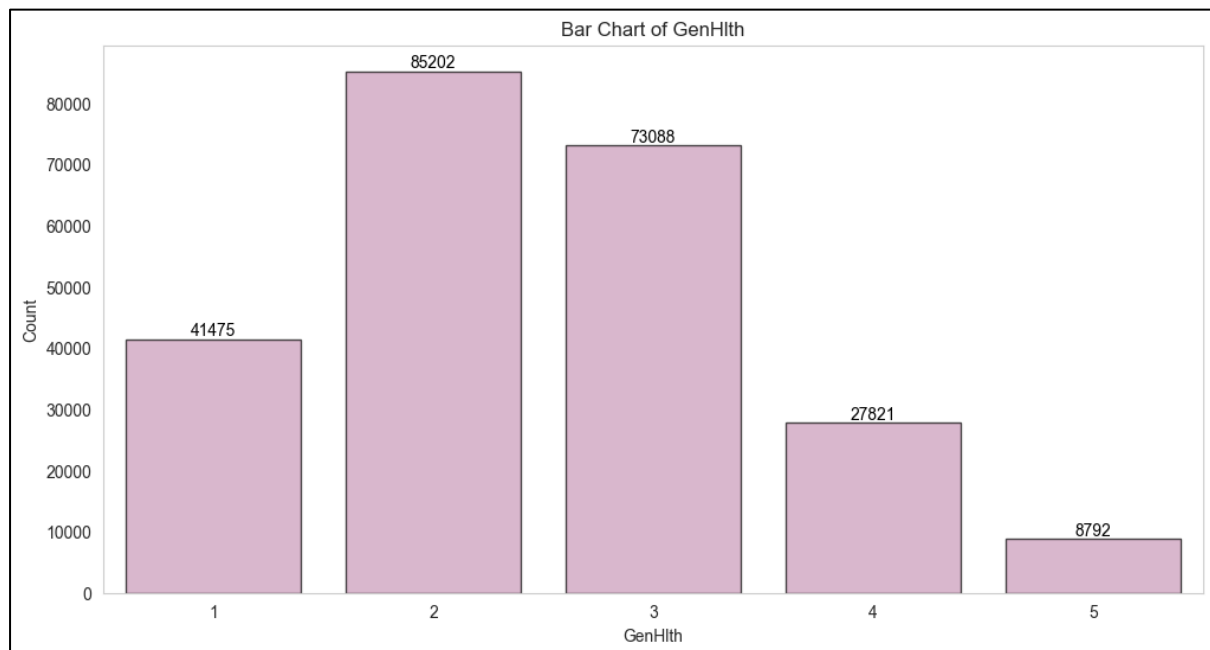


Figure 31: Bar Chart for GenHlth

Figure above shows the distribution of the **GenHlth** variable. The chart shows that the majority of individuals, 85,202 report a general health rating of "2" with good health while 41,475 individuals report "1" with excellent health. This suggests that a large portion of the population perceives their health to be at least good with a significant number reporting excellent health. The distribution appears right skewed with fewer individuals in the "4" and "5" categories which represent poor health. This indicates that the majority of the population are in good health. The slight decline towards "5" shows that very few people rate their health as poor, which may reflect general positive health conditions. This distribution highlights the importance of health interventions that maintain or improve general well-being while also identifying a smaller group that may have health problem.

DiffWalk

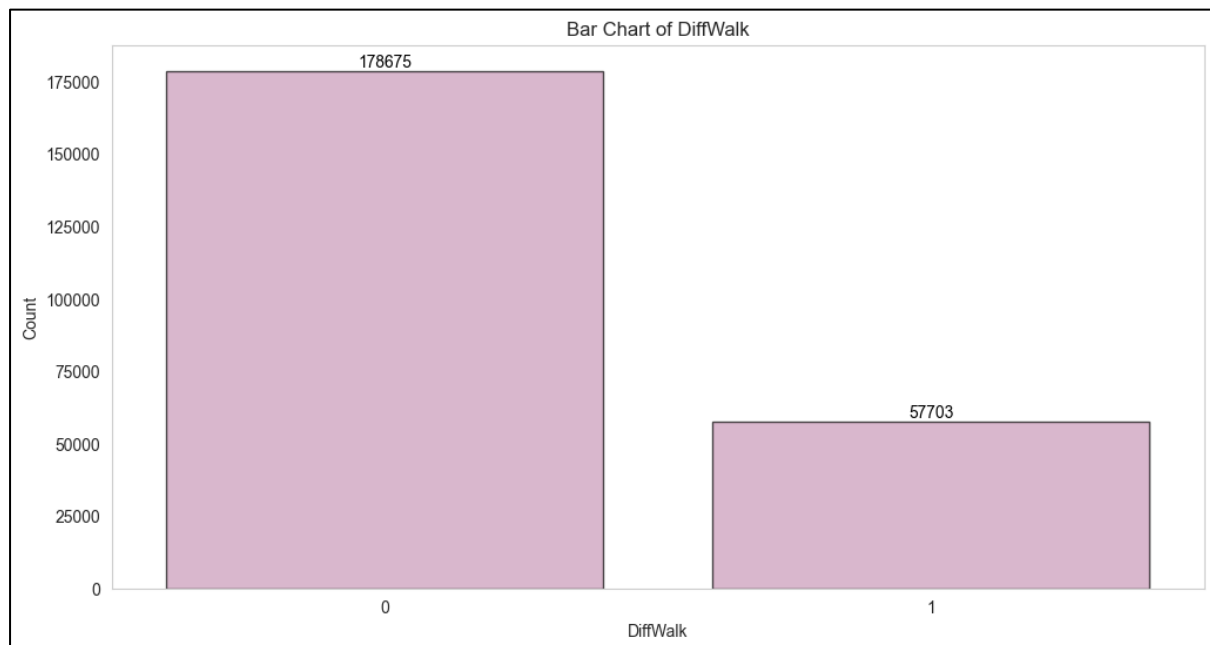


Figure 32: Bar Chart for DiffWalk

Figure above shows the distribution of the **DiffWalk** variable. The chart indicates that 178,675 individuals have no difficulty walking (0) while 57,703 individuals report having difficulty walking (1). This suggests that most individuals do not experience mobility issues which could reflect a generally healthy population with good physical function. However, the presence of a significant number of individuals reporting difficulty walking highlights a concerning issue. This could be due to a variety of factors such as aging, chronic health conditions or past injuries that affect mobility. The significant portion of the population experiencing mobility challenges emphasizes the need for targeted healthcare interventions and accessibility measures such as physical therapy and accommodations in public spaces to support those facing mobility difficulties.

Sex

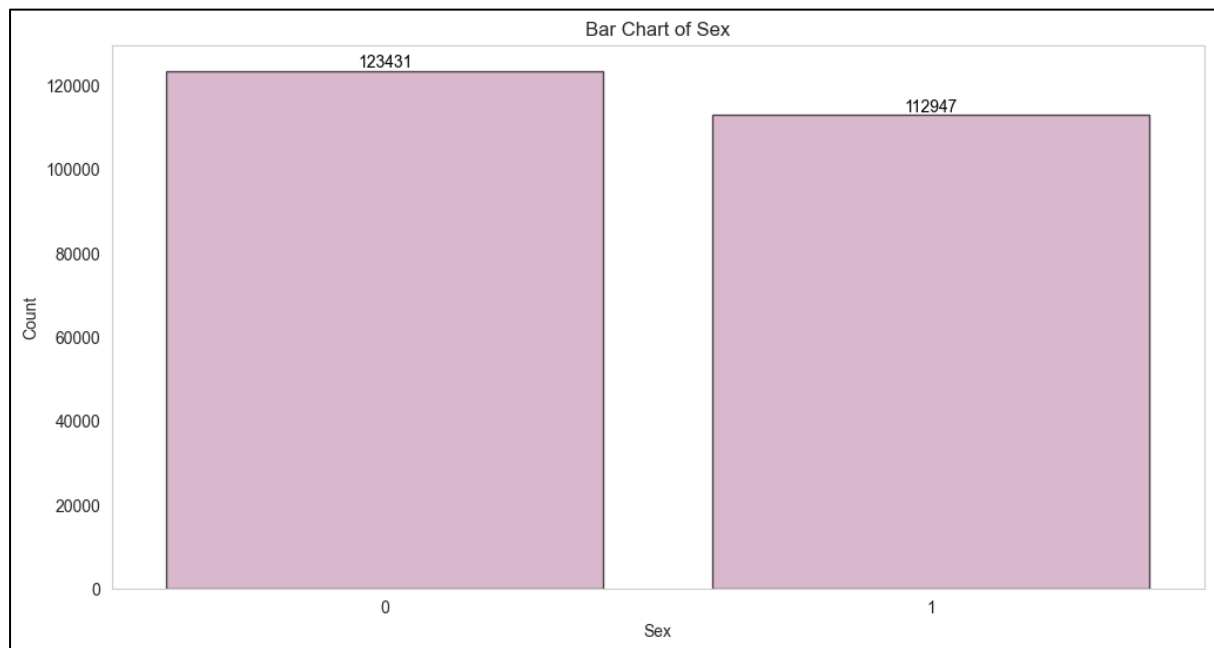


Figure 33: Bar Chart for Sex

Figure above shows the distribution of the **Sex** variable. The chart shows that there are 123,431 females (0) and 112,947 males (1), indicating a fairly balanced gender distribution in the dataset. This suggests that the dataset is representative of a diverse population in terms of gender with close to equal representation of both sexes. The balanced representation allows for better generalization of findings and analysis that can be applied across different gender groups in the population.

Age

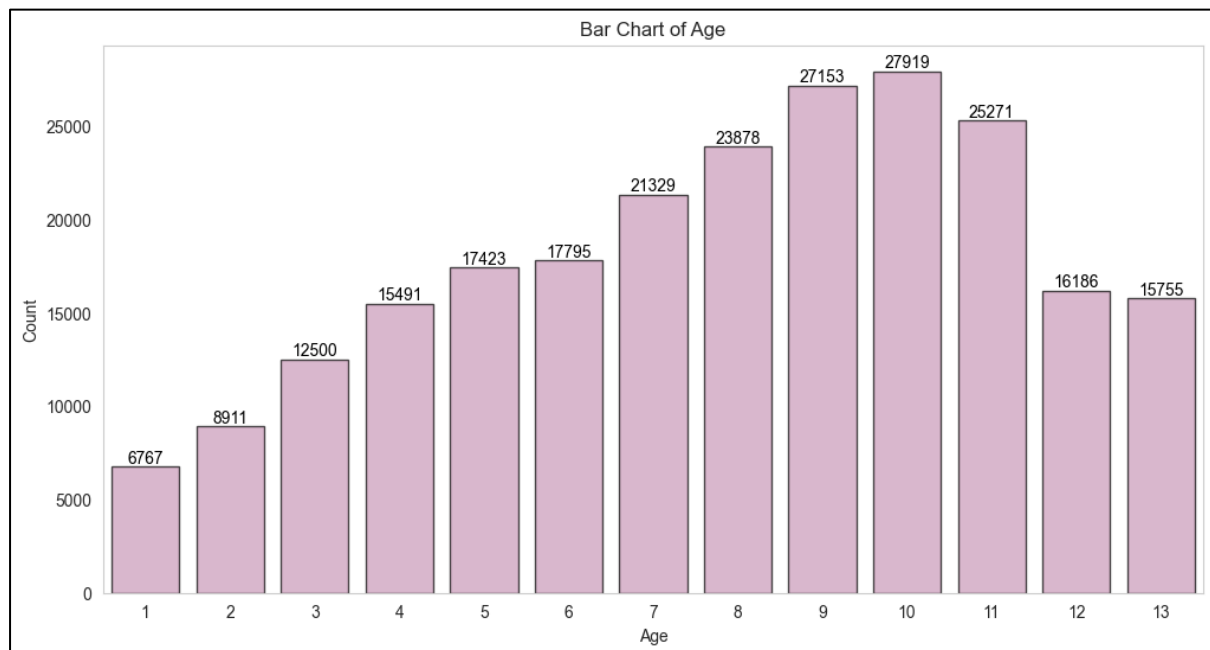


Figure 34: Bar Chart for Age

Figure above shows the distribution of the **Age** variable. The chart shows the frequency of individuals across different age groups with the highest counts observed in age groups "9" and "10" which total count of 27,153 and 27,919 individuals respectively. The distribution is roughly uniform with a peak around middle age and followed by a gradual decline in both younger and older groups. This distribution may reflect a broad demographic spread in the population with a larger portion in middle age, which could represent working adults or a stable age group. It suggests that the dataset captures a diverse population across various age ranges.

Education

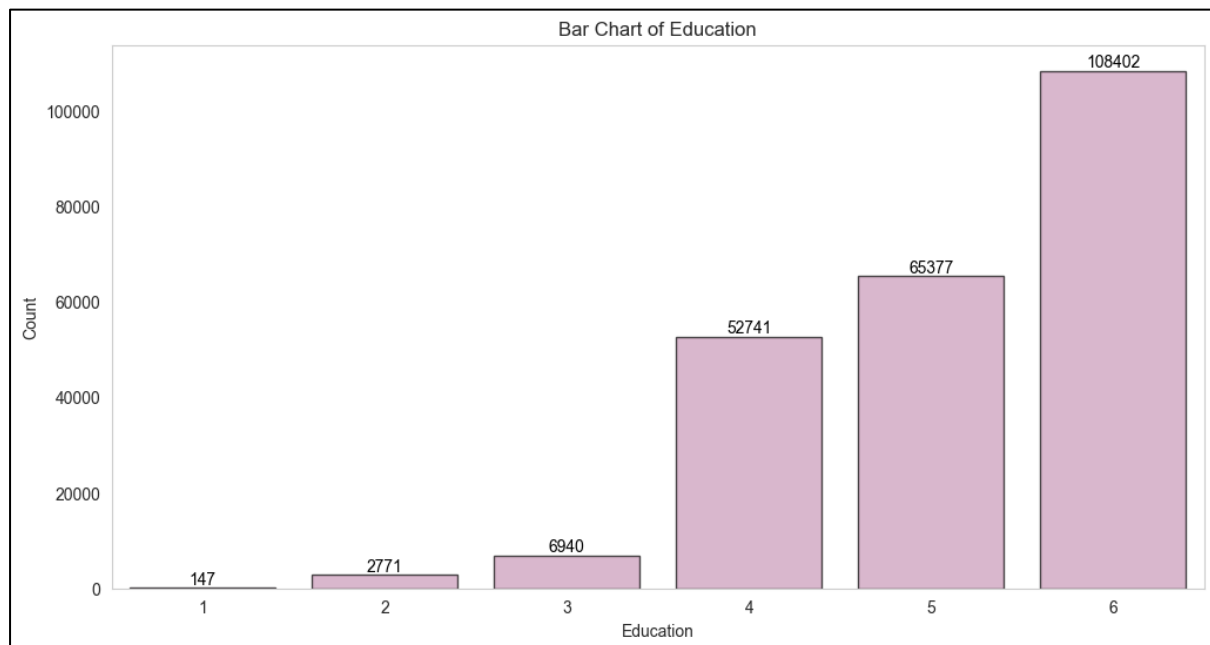


Figure 35: Bar Chart for Education

The figure above shows the distribution of the **Education** variable. The chart shows that the majority of individuals fall into the higher education levels, with 108,402 individuals in category "6" which is the highest education level. The distribution appears to be left-skewed as there are fewer individuals in the lower education levels such as categories 1 and 2, while the majority of the population has completed higher levels of education (categories 4, 5, and 6). This left-skewed distribution suggests that most individuals in the dataset have attained a relatively higher level of education.

Income

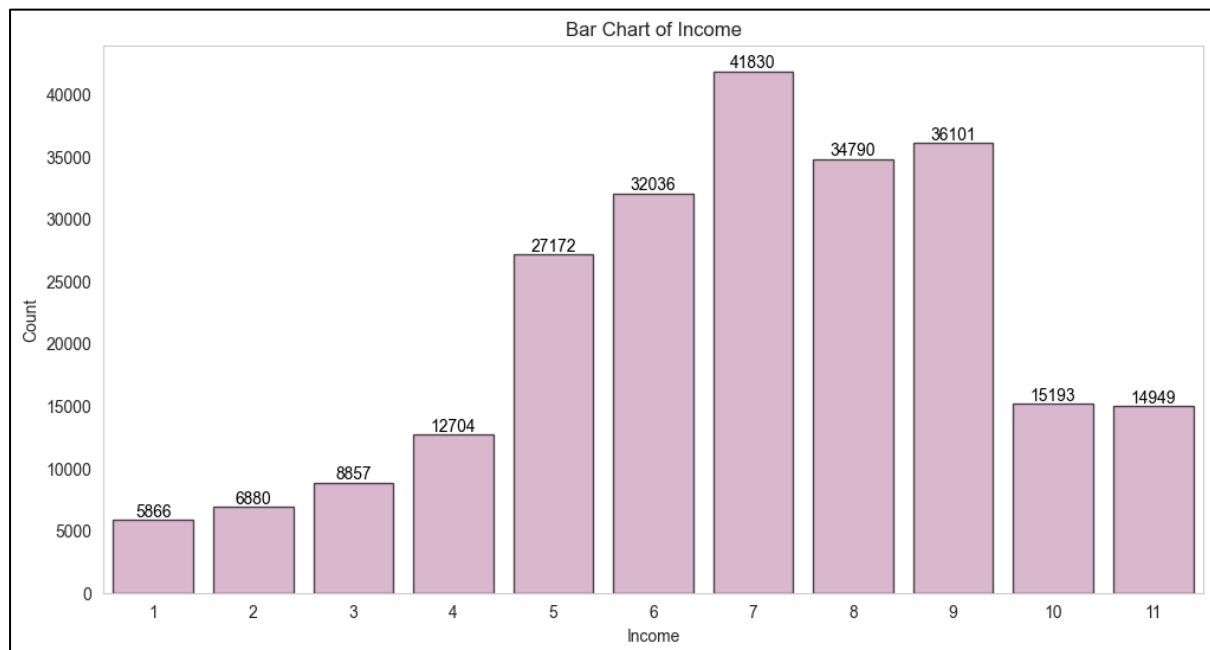


Figure 36: Bar Chart for Income

The figure above shows the distribution of the **Income** variable. The chart reveals a left-skewed distribution where the majority of individuals fall into the higher income categories (7, 8, 9) with the highest count in category "7" that have a total count of 41,830 individuals. The lower income categories (1, 2, 3) have fewer individuals, indicating that there are fewer people in the lowest income brackets. This suggests that the dataset represents a population with a relatively higher income which possibly reflecting an affluent or middle-to-upper class group.

3.1.2.2 Numerical Variables

```
import seaborn as sns
import matplotlib.pyplot as plt

# List of numerical columns
numerical_cols = ['BMI', 'MentHlth', 'PhysHlth']

sns.set_style("whitegrid")
color = "#D291BC"

fig, axes = plt.subplots(len(numerical_cols), 2, figsize=(15, 10))

# Plot histograms and boxplots side by side
for i, col in enumerate(numerical_cols):
    # Histogram (Left column)
    sns.histplot(df[col], kde=True, color=color, ax=axes[i,0], bins=30)
    axes[i,0].set_title(f'Distribution of {col}')
    axes[i,0].set_xlabel('Value')
    axes[i,0].set_ylabel('Frequency')

    # Boxplot
    sns.boxplot(y=df[col], color=color, ax=axes[i,1])
    axes[i,1].set_title(f'Boxplot of {col}')
    axes[i,1].set_ylabel('Value')

    # Calculate and display outlier count for boxplot
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
    axes[i,1].text(0.05, 0.95, f'Outliers: {len(outliers)}',
                  transform=axes[i,1].transAxes,
                  bbox=dict(facecolor='white', alpha=0.8))

plt.tight_layout()
plt.show()
```

Figure 37: Source Code of Numerical Variables by using Bar Chart

This source code is used to visualize numerical variables in the dataset through histograms and boxplots. It creates two plots side by side for each numerical column, such as 'BMI', 'MentHlth', and 'PhysHlth'. The histogram displays the distribution of values, helping to understand the frequency of different value ranges. The boxplot provides a summary of the data, showing the median, quartiles, and potential outliers. Additionally, the code calculates the outliers using the interquartile range (IQR) method and displays the count of outliers on the boxplot. These visualizations help identify the overall distribution, central tendency, and any anomalies in the numerical data, which aids in data exploration and analysis.

BMI

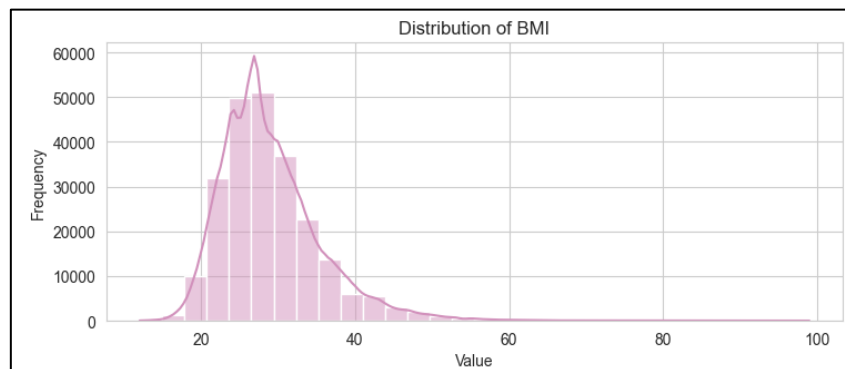


Figure 38: Distribution of BMI

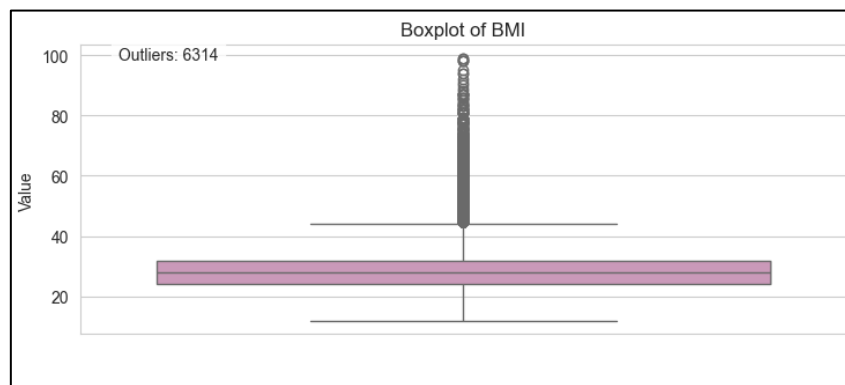


Figure 39: Boxplot of BMI

Figure above shows the distribution and boxplot of **BMI** (Body Mass Index). The distribution is right-skewed with the majority of individuals having a BMI in the normal weight range (around 20-25) as shown by the peak around a BMI of 20. This suggests that a significant portion of the population maintains a healthy weight. The boxplot further highlights the presence of outliers for the individuals with BMI values above 40. This has indicating that a smaller group of people may be struggling with obesity or weight-related health issues. These outliers could be considered for removal or further analysis during the preprocessing phase as they may skew the data and affect the results. While the majority of people are within a healthy BMI range, the outliers emphasize the need for interventions targeting those facing obesity or related conditions like diabetes or heart disease.

MenHlth

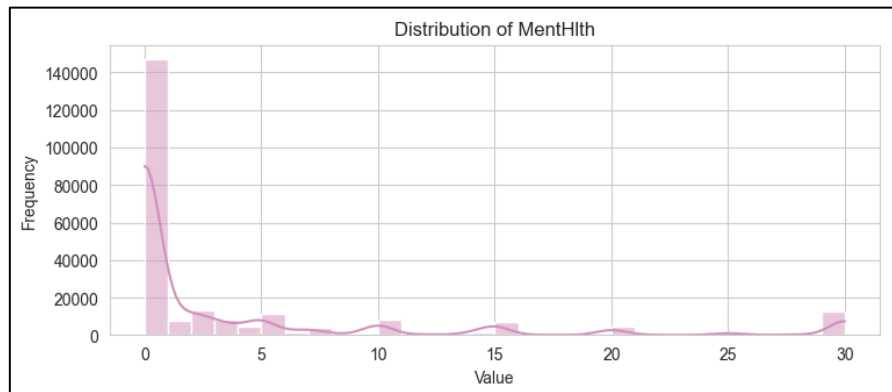


Figure 40: Distribution of MenHlth

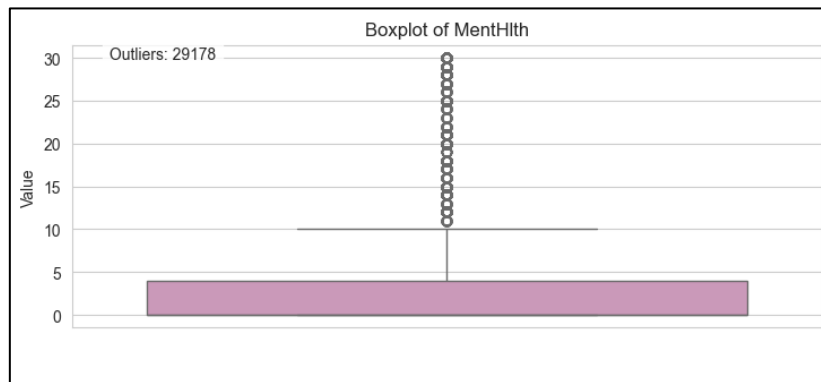


Figure 41: Boxplot of MenHlth

Figure above shows the distribution and boxplot of **MentHlth** (mental health days). The distribution is highly right skewed, indicating that most individuals report few or no mental health days. However, there is still have a small portion of the population reporting have a larger number of mental health days. This suggests that while most people experience good mental health and have a significant minority may be dealing with chronic mental health issues such as depression or anxiety. The boxplot shows a considerable number of outliers, indicating individuals who report a substantially higher number of mental health days. These outliers might need to be addressed in the preprocessing stage by removal or further analysis to avoid distorting the overall patterns in the data. These outliers also highlight the importance of addressing mental health needs through better access to mental health services, therapy and support programs.

PhysHlth

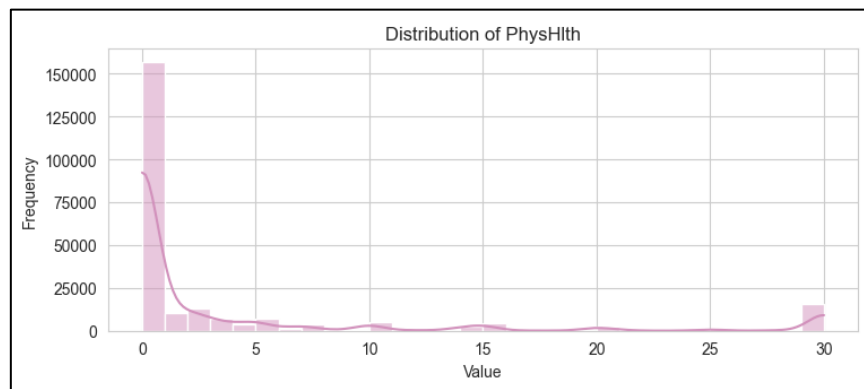


Figure 42: Distribution of PhysHlth

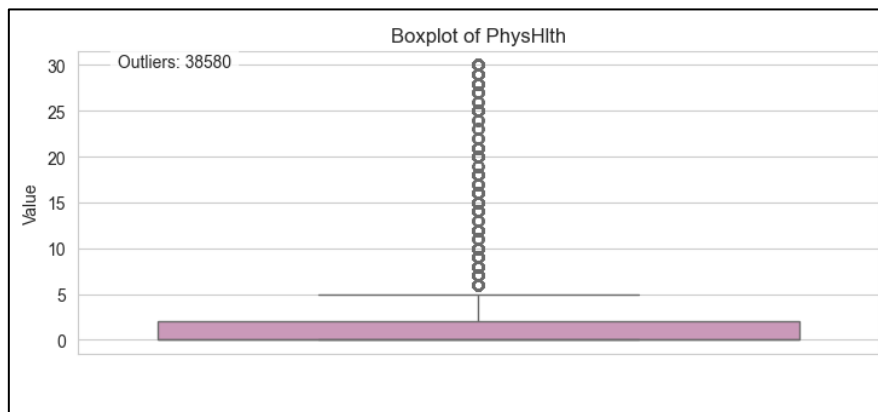


Figure 43: Boxplot of PhysHlth

Figure above shows the distribution of **PhysHlth** (physical health days). The distribution also shows a right skew with most individuals reporting few or no physical health days. However, a smaller group reports a considerably higher number of physical health days, likely due to chronic conditions, injuries or other health problems that affect their day-to-day physical well-being. The boxplot for PhysHlth highlights numerous outliers (38,580) and this is suggesting that while many individuals are physically healthy, there is still have a significant portion of the population is burdened by physical health issues. These outliers should be considered during the preprocessing phase where they could either be removed or analyzed further. Their presence emphasizes the need for focused healthcare services and support for individuals with ongoing physical health problems.

3.1.2.3 Target Variables – Diabetes_binary

The “Diabetes_binary” variable is set as the target variable to predict the diabetes.

```
import pandas as pd
import matplotlib.pyplot as plt

# Ensure 'Diabetes_binary' is treated as integer
df['Diabetes_binary'] = df['Diabetes_binary'].astype(int)

# Get the counts for the 'Diabetes_binary' column
diabetes_counts = df['Diabetes_binary'].value_counts()

# Create labels with counts and percentages for the pie chart
labels = [f"{category}\n{count} ({count/sum(diabetes_counts)*100:.1f}%)" for category, count in diabetes_counts.items()]

# Create the pie chart
plt.figure(figsize=(7, 7))
plt.pie(diabetes_counts, labels=labels, autopct='%1.1f%%',
        colors=[ "#D291BC", "#EE82EE"], startangle=90, shadow=True)
plt.title("Pie Chart of Diabetes Status (Diabetes vs No Diabetes)")
plt.axis('equal')
plt.show()

print(diabetes_counts)
print(f"Total: {diabetes_counts.sum()} entries")
```

Figure 44: Source Code of Target Variable

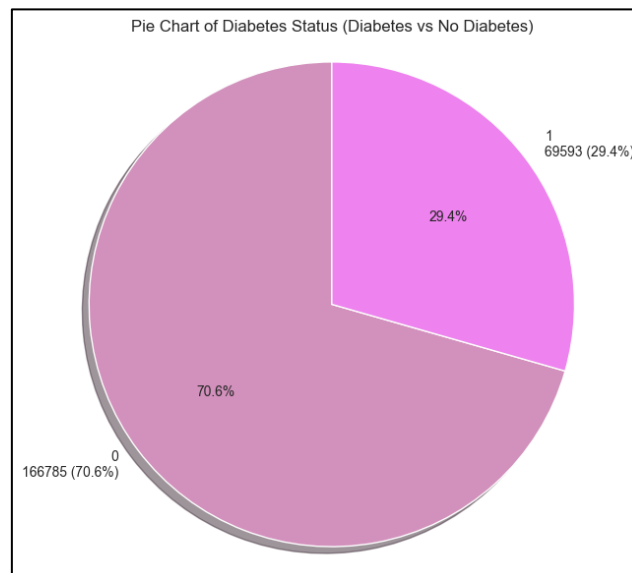


Figure 45: Pie Chart of Target Variable

Figure above shows the pie chart of the **Diabetes_binary** target variable. The pie chart shows the prevalence of diabetes in the dataset with 70.6% of individuals (166,785) classified as "No Diabetes" (0) and 29.4% of individuals (69,593) diagnosed with "Diabetes" (1). This distribution reflects an imbalanced dataset as the "No Diabetes" category significantly outnumbers the "Diabetes" category. The imbalance is an important consideration for data preprocessing as it can affect the performance of deep learning models. The model might become biased toward predicting the majority class (No Diabetes) so this might be need to solve during preprocessing stage to ensure more accurate predictions for both classes.

3.1.2.4 Correlation Heatmap

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import chi2_contingency

continuous_vars = df.select_dtypes(include=[np.number])
correlation_matrix = continuous_vars.corr()
target_correlation = correlation_matrix['Diabetes_binary'].sort_values(ascending=False)
print("Correlation with target (Diabetes_binary):")
print(target_correlation)

plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5,
            annot_kws={'size': 10}, cbar_kws={'shrink': 0.8})
plt.title('Correlation Matrix for Continuous Variables', fontsize=16)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

categorical_vars = df.select_dtypes(include=['object', 'bool']).columns
print("\nChi-Square Test Results for Categorical Variables:")

for var in categorical_vars:
    contingency_table = pd.crosstab(df[var], df['Diabetes_binary'])
    chi2, p_value, dof, expected = chi2_contingency(contingency_table)
    print(f"\nChi-Square Test for {var} vs Diabetes_binary:")
    print(f"P-Value: {p_value}")
    if p_value < 0.05:
        print(f"Significant relationship between {var} and Diabetes_binary (p < 0.05).")
    else:
        print(f"No significant relationship between {var} and Diabetes_binary (p >= 0.05).")

plt.figure(figsize=(12, 8))
for idx, var in enumerate(categorical_vars):
    plt.subplot(2, 3, idx + 1)
    sns.countplot(data=df, x=var, hue='Diabetes_binary')
    plt.title(f'{var} vs Diabetes_binary')

plt.tight_layout()
plt.show()
```

Figure 46: Source Code of Correlation Heatmap

Figure above shows the source code used to analyse the relationships between variables and the target variable, Diabetes_binary. It calculates the correlation between continuous variables and the target using the corr() function and visualizes the results in a heatmap. This helps to identify which continuous variables have strong associations with diabetes.

Besides, the code uses the Chi-Square test (chi2_contingency) to assess relationships between categorical variables and the target. A p-value below 0.05 indicates a significant relationship. Finally, count plots are generated for categorical variables against Diabetes_binary to visualize how each category relates to the target variable. The results from both the correlation matrix and Chi-Square test help in selecting relevant features for further analysis.

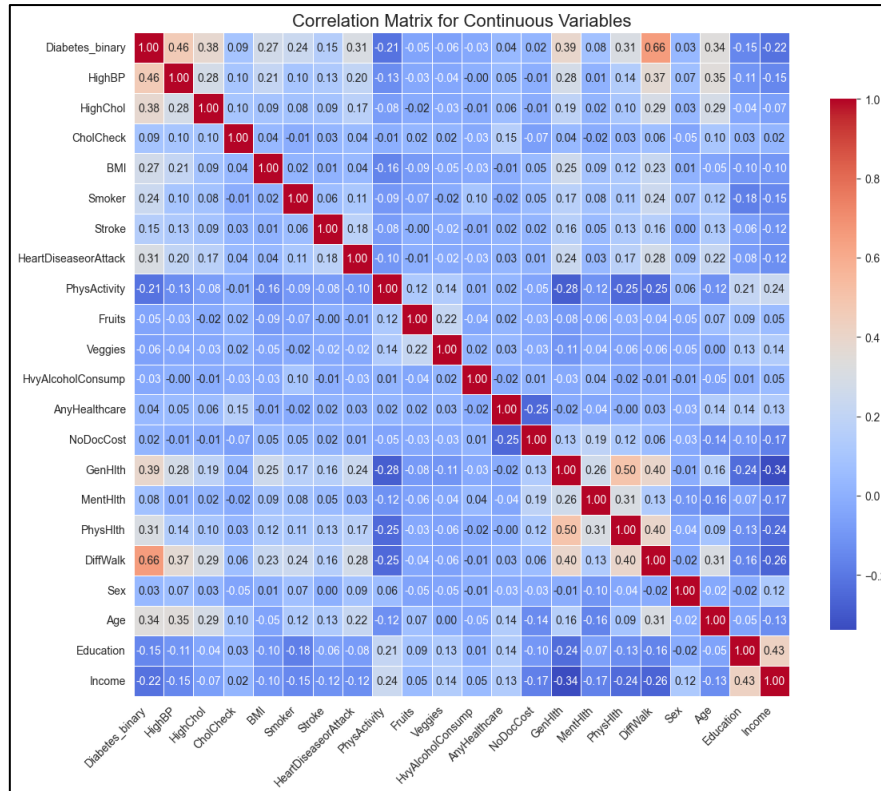


Figure 47: Correlation Heatmap

The correlation matrix for continuous variables shows the relationships between Diabetes_binary and other continuous features. The strongest correlation is DiffWalk with 0.66 and this is indicating that individuals who experience difficulty walking are more likely to have diabetes. This reflects the association between diabetes and mobility issues. BMI with 0.27 shows a moderate positive correlation and indicates that higher BMI is slightly associated with a higher likelihood of having diabetes. Similarly, HighBP (0.46) and HeartDiseaseorAttack (0.31) show moderate positive correlations and are suggesting that high blood pressure and heart disease are commonly associated with diabetes.

Besides, Age (0.34) has a moderate positive correlation with diabetes which indicates that the likelihood of having diabetes increases as individuals age. In contrast, lifestyle factors such as Fruits (-0.02), Veggies (0.02) and PhysActivity (-0.21) show weak or negative correlations with diabetes. These weak correlations suggest that fruit, vegetable consumption and physical activity have minimal impact on the likelihood of having diabetes in this dataset.

Overall, the strongest variables with diabetes are seen in health-related factors such as DiffWalk, BMI, HighBP and Age while lifestyle factors like fruit consumption, vegetable consumption, and physical activity exhibit weaker correlations.

3.2 Data Preprocessing

3.2.1 Missing Values

```
missing_values = df.isnull().sum()
print(missing_values)
```

Figure 48: Source Code of Missing Values

```
Diabetes_binary    0
HighBP             0
HighChol           0
CholCheck          0
BMI               0
Smoker            0
Stroke            0
HeartDiseaseorAttack 0
PhysActivity       0
Fruits            0
Veggies           0
HvyAlcoholConsump 0
AnyHealthcare     0
NoDocCost         0
GenHlth           0
MentHlth          0
PhysHlth          0
DiffWalk          0
Sex               0
Age              0
Education          0
Income            0
dtype: int64
```

Figure 49: Output of Missing Values

The figure above shows the output of missing value checks in the dataset where all columns have a count of 0 and is indicating that there are no missing values in the dataset.

3.2.2 Outliers

Categorical Variable - Education

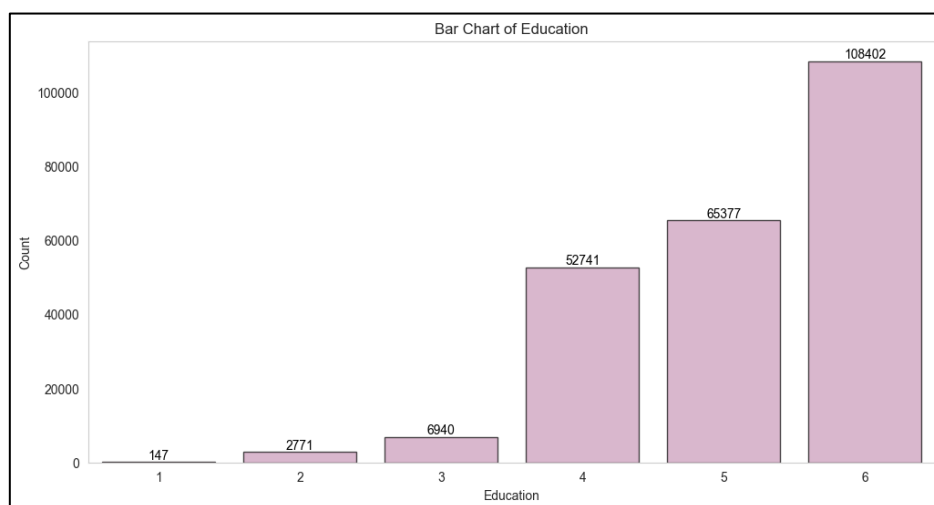


Figure 50: Bar Chart of Education before Preprocessing

As discussed during in the Graphical View of Variables section, the analysis of the education variable revealed that education level "1" contributes less than 1% of the total count and could be considered an outlier.

```
# Check value counts and diabetes rates
print("Value counts:\n", df['Education'].value_counts().sort_index())
print("\nDiabetes rates:\n", df.groupby('Education')['Diabetes_binary'].mean())
```

Figure 51: Source Code of Calculating Diabetes Rate

```
Value counts:
Education
1      147
2     2771
3     6940
4    52741
5    65377
6   108402
Name: count, dtype: int64

Diabetes rates:
Education
1      0.448980
2      0.457957
3      0.450288
4      0.369428
5      0.332166
6      0.220780
Name: Diabetes_binary, dtype: float64
```

Figure 52: Diabetes Rate for Each Education Level

Upon reviewing the diabetes rates, it was found that education level "1" has a diabetes rate of 0.448980 which is significant. As this level provides valuable insights into the dataset, it was determined that removing it would result in the loss of important information. Therefore, the decision was made to keep education level "1".

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")

# Check original distribution
print("Original Value Counts:\n", df['Education'].value_counts().sort_index())

plt.figure(figsize=(14, 6))
plt.subplot(1, 2, 1)
ax1 = sns.countplot(x='Education', data=df,
                    order=sorted(df['Education'].unique()),
                    color='█ #0291BC')

for p in ax1.patches:
    ax1.annotate(f'{int(p.get_height())}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='bottom', fontsize=10)

plt.title('Education Bar Chart (Before Merge)', pad=20)
plt.xlabel('Education Level')
plt.ylabel('Count')

# Merge and rename in one step
df['Education_level'] = df['Education'].replace(1, 2)
df.drop('Education', axis=1, inplace=True)

plt.subplot(1, 2, 2)
ax2 = sns.countplot(x='Education_level', data=df,
                    order=sorted(df['Education_level'].unique()),
                    color='█ #0291BC')

for p in ax2.patches:
    ax2.annotate(f'{int(p.get_height())}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='bottom', fontsize=10)

plt.title('Education Bar Chart (After Merge)', pad=20)
plt.xlabel('Education Level (1 merged into 2)')
plt.ylabel('Count')

plt.tight_layout()
plt.show()

print("\nFinal Education Level Counts:\n", df['Education_level'].value_counts().sort_index())

df_clean = df.copy()
```

Figure 53: Source Code of Education Distribution

Original Value Counts:		Final Education Level Counts:	
Education		Education_Level	
1	147	2	2918
2	2771	3	6940
3	6940	4	52741
4	52741	5	65377
5	65377	6	108402
6	108402		
Name: count, dtype: int64		Name: count, dtype: int64	

Figure 54: Education Counts Before and After Merge

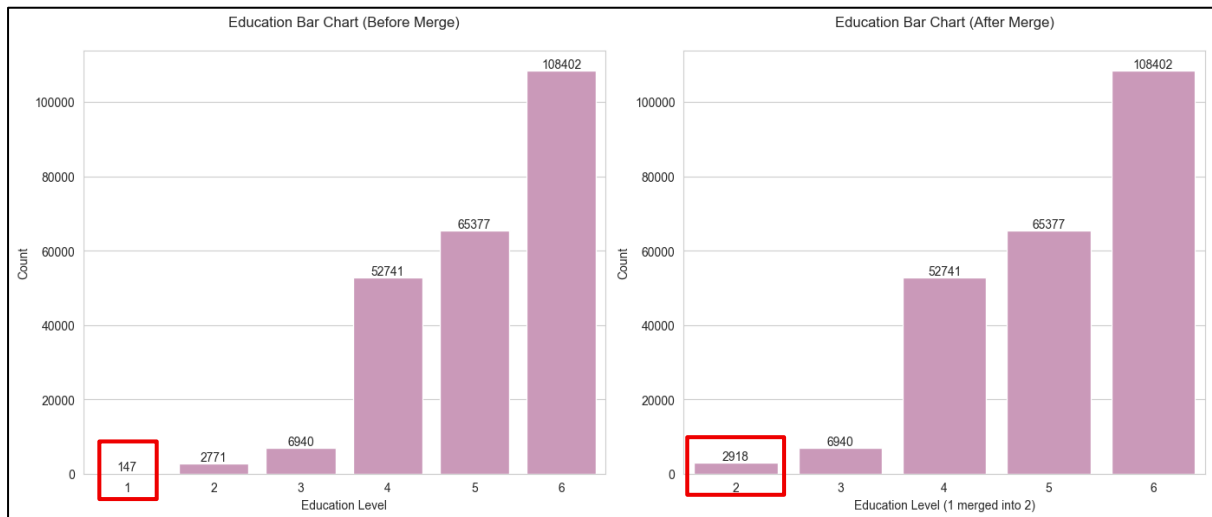


Figure 55: Education Bar Chart Before and After Merge

To retain the valuable insights while addressing the outlier issue, education level "1" was not removed. Instead, it was merged with education level "2." The figures above show the bar chart of education levels before and after the merge. The left figure shows the education distribution before the merge with education level "1" having a very low count. Besides, the right figure shows the distribution after the merge where education level "1" is combined with education level "2" and have a total count of 2918.

Numerical Variables

```
import seaborn as sns
import matplotlib.pyplot as plt

# Numerical columns
numerical_cols = ['BMI', 'MenthHlth', 'PhysHlth']

sns.set_style("whitegrid")
color = "#D291BC"
fig, axes = plt.subplots(len(numerical_cols), 2, figsize=(15, 10))

for i, col in enumerate(numerical_cols):
    # Histogram (Left column)
    sns.histplot(df[col], kde=True, color=color, ax=axes[i,0], bins=30)
    axes[i,0].set_title(f'Distribution of {col}')
    axes[i,0].set_xlabel('Value')
    axes[i,0].set_ylabel('Frequency')

    # Boxplot (Right column)
    sns.boxplot(y=df[col], color=color, ax=axes[i,1])
    axes[i,1].set_title(f'Boxplot of {col}')
    axes[i,1].set_ylabel('Value')

    # Calculate and display outlier count for boxplot
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
    axes[i,1].text(0.05, 0.95, f'Outliers: {len(outliers)}',
                  transform=axes[i,1].transAxes,
                  bbox=dict(facecolor='white', alpha=0.8))

plt.tight_layout()
plt.show()
```

Figure 56: Source Code of Display Numerical Variables

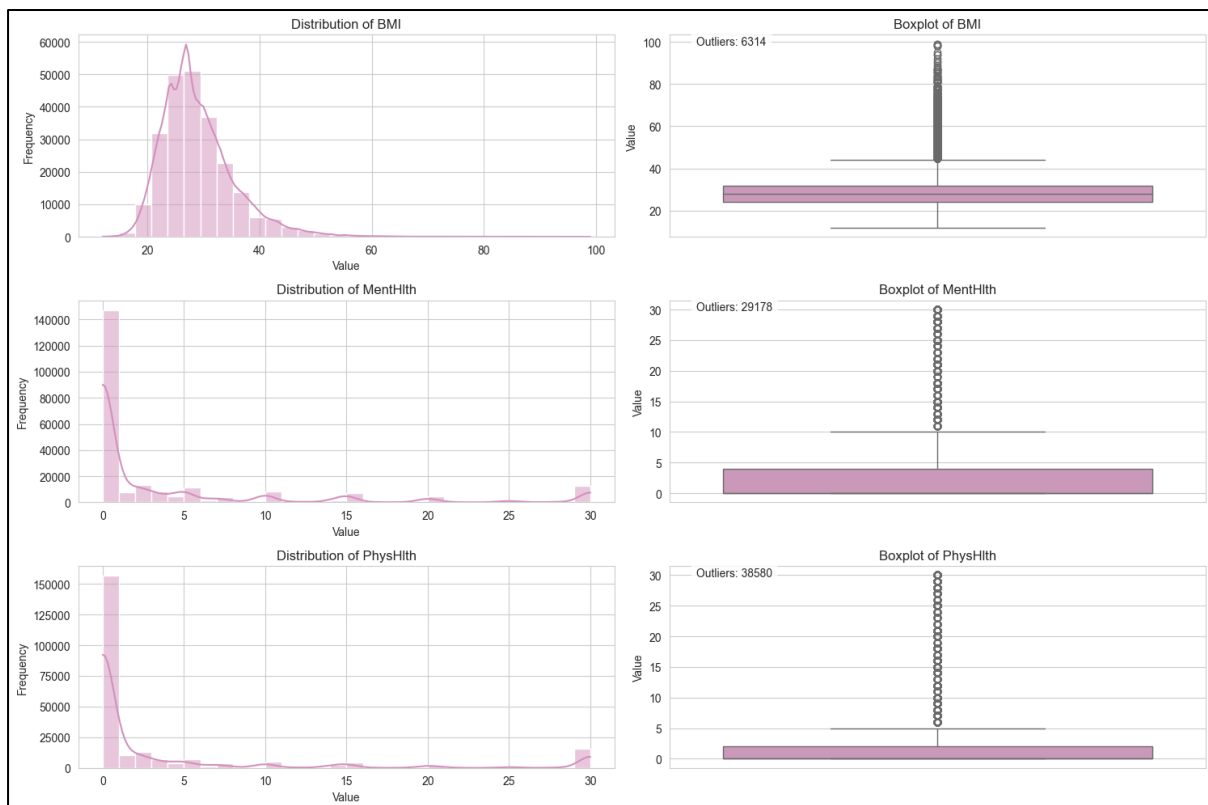


Figure 57: Display Numerical Variables

For the numerical variables that are also have been discussed in the Graphical View of Variables section, outliers have been observed in the distributions and boxplots of BMI, MentHlth and PhysHlth and are needed to be addressed.

For BMI, the distribution appears relatively normal but the boxplot reveals that there are significant outliers with 6,314 data points that fall outside the expected range. Similarly, MentHlth and PhysHlth exhibit highly skewed distributions with outliers 29,178 for MentHlth and 38,580 for PhysHlth. These outliers for MentHlth and PhysHlth may distort the analysis and should be carefully considered by either removing them or by applying appropriate transformations to handle them.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Validate outliers
print("Outliers for BMI (greater than 50):")
print(df[df['BMI'] > 50][['BMI']].describe())

print("\nOutliers for MentHlth (greater than 30):")
print(df[df['MentHlth'] > 30][['MentHlth']].describe())

print("\nOutliers for PhysHlth (greater than 30):")
print(df[df['PhysHlth'] > 30][['PhysHlth']].describe())

df_clean = df.copy()

# Apply winsorization
df_clean['BMI'] = df_clean['BMI'].clip(upper=50)
df_clean['MentHlth'] = df_clean['MentHlth'].clip(upper=30)
df_clean['PhysHlth'] = df_clean['PhysHlth'].clip(upper=30)

# Create visualizations
plt.figure(figsize=(15, 5))

# Plot BMI boxplot
plt.subplot(1, 3, 1)
sns.boxplot(y=df_clean['BMI'], color="#D291BC")
plt.title('BMI')

# Plot MentHlth boxplot
plt.subplot(1, 3, 2)
sns.boxplot(y=df_clean['MentHlth'], color="#D291BC")
plt.title('MentHlth')

# Plot PhysHlth boxplot
plt.subplot(1, 3, 3)
sns.boxplot(y=df_clean['PhysHlth'], color="#D291BC")
plt.title('PhysHlth')

plt.tight_layout()
plt.show()
```

Figure 58: Source Code of Validating Outliers

To solve the outliers in the numerical variables for BMI, MentHlth, and PhysHlth, several steps were taken to clean the data. First, outliers were identified by checking for values that exceeded specific thresholds. For BMI, values greater than 50 were considered outliers because they represent extreme cases that are likely not reflective of typical data. Similarly, for MentHlth and PhysHlth, values greater than 30 were identified as outliers since these variables track health data over a 30-day period and values exceeding this range are unusual.


```

Outliers for BMI (greater than 50):
  BMI
count  2102.000000
mean    56.831113
std     7.172753
min     51.000000
25%     52.000000
50%     55.000000
75%     59.000000
max     99.000000

Outliers for MentHlth (greater than 30):
  MentHlth
count      0.0
mean      NaN
std       NaN
min       NaN
25%       NaN
50%       NaN
75%       NaN
max       NaN

Outliers for PhysHlth (greater than 30):
  PhysHlth
count      0.0
mean      NaN
std       NaN
min       NaN
25%       NaN
50%       NaN

```

Figure 59: Output for Each Numerical Variables

Once the outliers were identified, appropriate actions were taken to remove them. For BMI, all values greater than 50 were removed as they were deemed unrealistic for the context. For MentHlth and PhysHlth, values greater than 30 were removed as these values surpassed the normal range for these health metrics. This approach could help to eliminate extreme data points that could distort the overall analysis.

After validating and removing the outliers, the winzoration method was applied to clip any remaining values that exceeded the defined thresholds. For BMI, values greater than 50 were clipped and for the values greater than 30 were clipped for MentHlth and PhysHlth. Clipping ensures that the values stay within the acceptable range and avoids the distortion caused by outliers while preserving the integrity of the data.

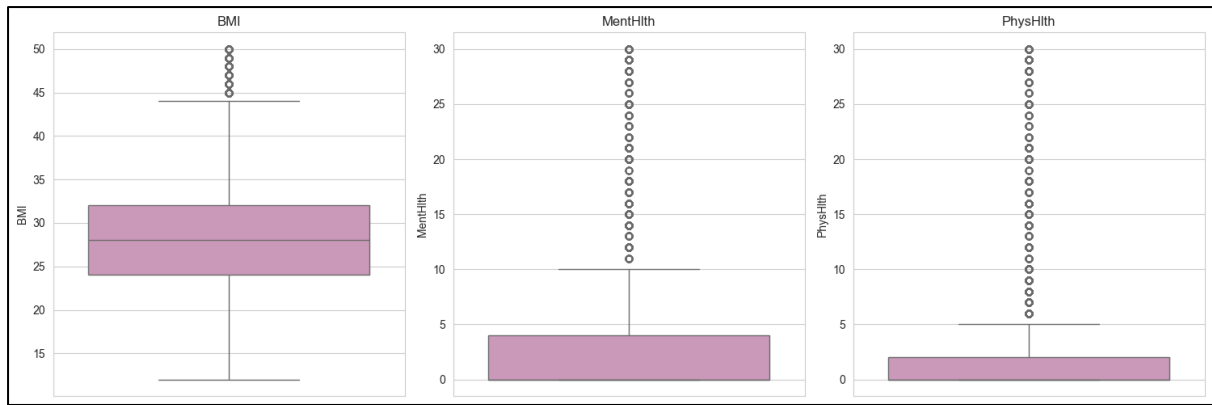


Figure 60: Boxplot of Numerical Variables after Handling Outliers

The figure above shows the boxplots of the numerical variables BMI, MentHlth and PhysHlth after handling outliers. Although some outliers remain in the distributions, they have not been removed. These remaining outliers could still represent valid data points that provide meaningful insights. The BMI boxplot shows that outliers above 50 have been successfully removed and the distribution is more concentrated after handling outliers. For MentHlth and PhysHlth, while the number of outliers has been reduced, some outliers still exist. However, given the nature of the data, these remaining outliers were kept preserving the integrity and richness of the dataset.

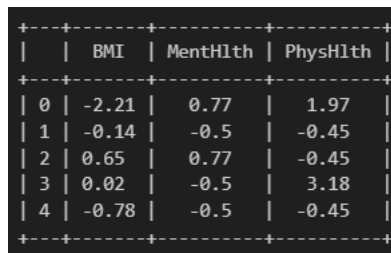
3.2.3 Data Standardization

```
from sklearn.preprocessing import StandardScaler
from tabulate import tabulate

# Numerical columns to standardize
numerical_cols = ['BMI', 'MentHlth', 'PhysHlth']
scaler = StandardScaler()

# Apply standardization to the numerical columns
df_clean[numerical_cols] = scaler.fit_transform(df_clean[numerical_cols])
scaled_data = df_clean[numerical_cols].head()
print(tabulate(scaled_data.round(2), headers='keys', tablefmt='pretty'))
```

Figure 61: Source Code of Data Standardization



	BMI	MentHlth	PhysHlth
0	-2.21	0.77	1.97
1	-0.14	-0.5	-0.45
2	0.65	0.77	-0.45
3	0.02	-0.5	3.18
4	-0.78	-0.5	-0.45

Figure 62: Output for Data Standardization

The figure above shows the application of data standardization to the numerical variables BMI, MentHlth and PhysHlth by using the StandardScaler from sklearn. The goal of standardization is to transform the data so that each variable has a mean of 0 and a standard deviation of 1. This step is important when the variables have different units or scales to ensure that no single variable dominates the analysis or modeling process due to discrepancies in their scales.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_style("whitegrid")

# Numerical columns
numerical_cols = ['BMI', 'MentHlth', 'PhysHlth']
fig, axes = plt.subplots(len(numerical_cols), 1, figsize=(12, 10))

if len(numerical_cols) == 1:
    axes = [axes]

for i, col in enumerate(numerical_cols):
    sns.histplot(df_clean[col], kde=True, color='D291BC', ax=axes[i], bins=30)
    axes[i].set_title(f'Distribution of {col} (Standardized)')
    axes[i].set_xlabel('Value')
    axes[i].set_ylabel('Frequency')

plt.tight_layout()
plt.show()
```

Figure 63: Source Code for Numerical Variables after Data Standardization

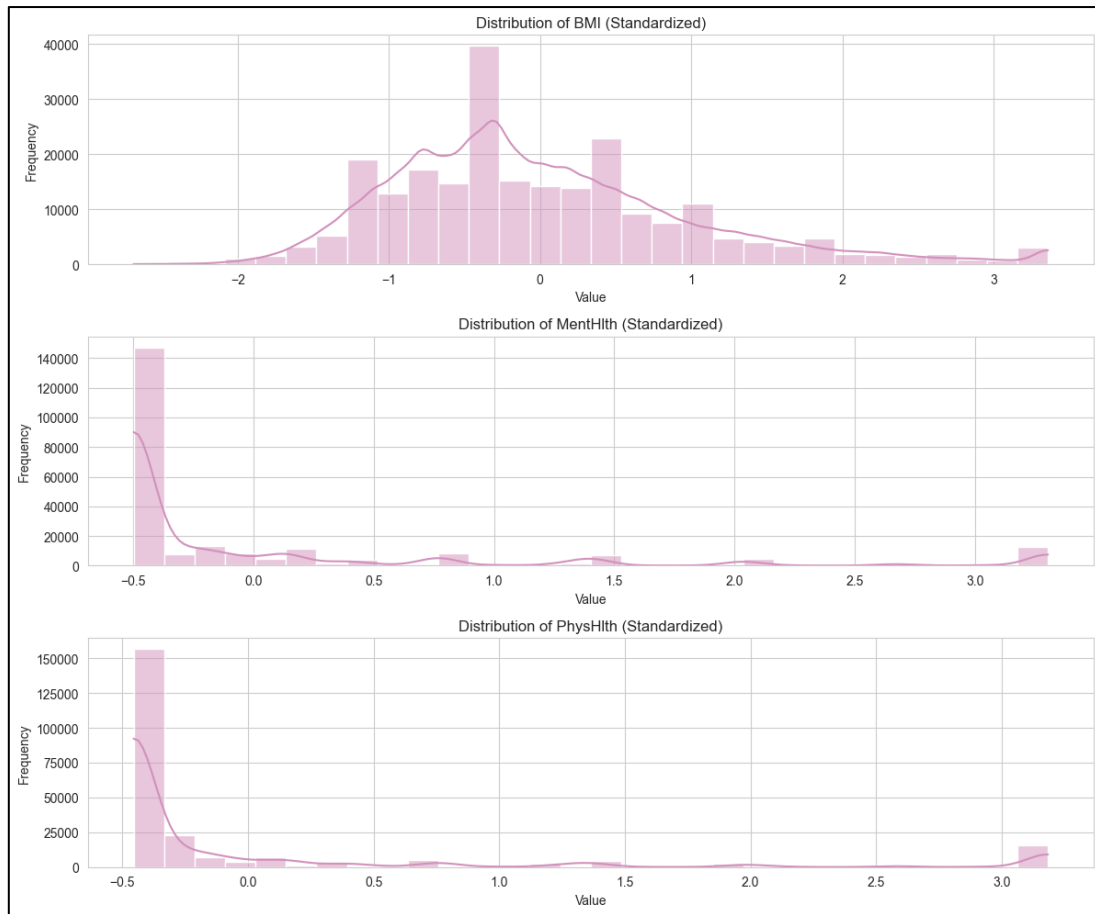


Figure 64: Distribution of Numerical Variables after Data Standardization

After applying the standardization, the output shows the transformed values for each numerical variable. For example, the standardized **BMI** values are centered around 0 with most values falling between -2 and 2. Similarly, the **MentHlth** and **PhysHlth** values show a spread around 0. These transformations allow for a more consistent comparison across variables and prepare the data ready for modeling. Overall, the standardization process was successful in preparing the data by scaling the numerical variables to a common range

3.2.4 Data Transformation

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from tabulate import tabulate

# Bin the Age column
bins = [0, 5, 8, 11, 13]
labels = [1, 2, 3, 4]
df_clean['Age_Group'] = pd.cut(df_clean['Age'], bins=bins, labels=labels, include_lowest=True).astype(int)

# Drop the original Age column
df_clean.drop('Age', axis=1, inplace=True)

# Verification
print("\nAge Group Distribution:")
print(df_clean['Age_Group'].value_counts().sort_index())

# Create Age Group bar chart
plt.figure(figsize=(10, 6))
ax = sns.countplot(x='Age_Group', data=df_clean, color='#D291BC')

for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='bottom', fontsize=12)

plt.title('Distribution of Age Groups', fontsize=14, pad=20)
plt.xlabel('Age Group (1=Youngest, 4=Oldest)', fontsize=12)
plt.ylabel('Count', fontsize=12)
plt.xticks(rotation=0)
plt.grid(axis='y', alpha=0.3)
plt.show()
```

Figure 65: Source Code of Data Transformation for Age

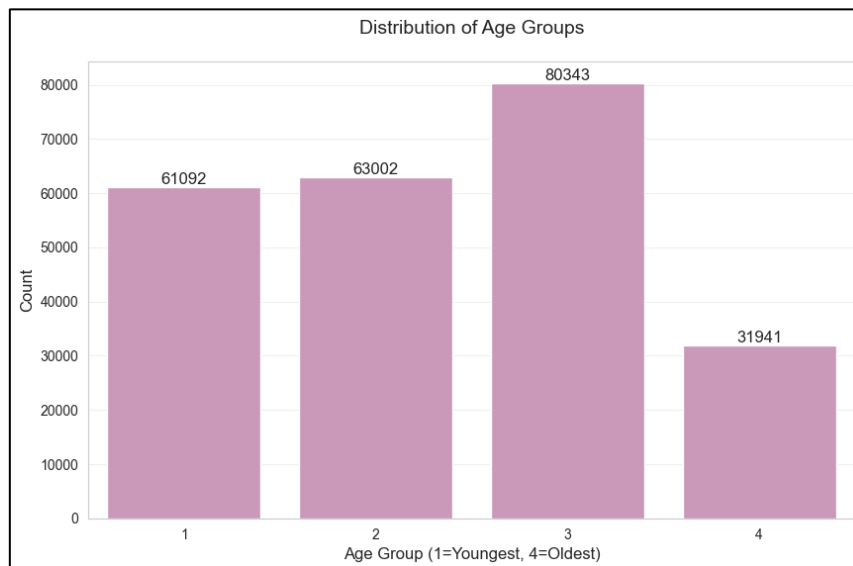


Figure 66: Distribution of Age Groups after Data Transformation

The Age column was transformed into Age Groups to enhance the prediction and accuracy of the analysis. By categorizing the age values into four distinct groups, the data becomes more structured and easier to interpret. The age groups were defined into four groups which is Group 1 for individuals aged 18-24, Group 2 for those aged 25-34, Group 3 for those aged 35-44 and Group 4 for individuals aged 45 and above. After creating the Age Group column, the original Age column was removed to avoid redundancy and maintain clarity.

This transformation improves the model's ability to detect patterns and enhance prediction accuracy by grouping individuals into more meaningful age ranges for a better trend identifications and a more reliable insight.

3.2.5 Check Data Types after Preprocessing

```
# Print data types after preprocessing
print(df_clean.dtypes)
```

Figure 67: Source Code of Data Types After Preprocessing

```
Diabetes_binary      int32
HighBP              int64
HighChol            int64
CholCheck            int64
BMI                 float64
Smoker              int64
Stroke              int64
HeartDiseaseorAttack int64
PhysActivity         int64
Fruits              int64
Veggies             int64
HvyAlcoholConsump   int64
AnyHealthcare        int64
NoDocCost            int64
GenHlth             int64
MentHlth            float64
PhysHlth            float64
DiffWalk            int64
Sex                 int64
Income              int64
Education_Level      int64
Age_Group           int32
dtype: object
```

Figure 68: Data Types After Preprocessing

After completing the preprocessing steps, it is important to verify that the data types of the variables. The table above shows the data types of each column in the dataset after preprocessing. For the numerical variables such as BMI, MentHlth and PhysHlth are correctly assigned as float64 data types that is appropriate for further analysis and modeling. The categorical variable Age_Group is correctly assigned as an int32 type which represents the binned age groups and other categorical columns like Education_Level and Sex are also correctly assigned as int64.

3.3 Data Splitting and Resampling

3.3.1 Check Target Variable Count

```
# Check Target Variable
print(df_clean['Diabetes_binary'].value_counts())
```

Figure 69: Source Code of Checking Total Numbers of Diabetes for Each Class

```
Diabetes_binary
0      166785
1       69593
Name: count, dtype: int64
```

Figure 70: Total Numbers of Diabetes for Each Class

The figure above shows the count of each class for the target variable. The target variable **Diabetes_binary** represents the binary outcome of whether an individual has diabetes (1) or not (0). It indicates that there are 166,785 instances of individuals without diabetes (0) and 69,593 instances of individuals with diabetes (1).

This distribution shows that the dataset is imbalanced with more instances of individuals without diabetes. This imbalance could potentially impact the model's performance, especially in accurately predicting the minority class, which represents individuals with diabetes. Therefore, the SMOTE (Synthetic Minority Over-sampling Technique) will be applied to solve this issue and improve the model's ability to predict the minority class accurately.

3.3.2 Data Splitting (Train-Test Split)

```
from sklearn.model_selection import train_test_split

X = df_clean.drop('Diabetes_binary', axis=1)
y = df_clean['Diabetes_binary']

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Training set size: {X_train.shape[0]}")
print(f"Test set size: {X_test.shape[0]}")
```

Figure 71: Source Code of Data Splitting

```
Training set size: 189102
Test set size: 47276
```

Figure 72: Output after Data Splitting

The data was split into training and testing sets using the `train_test_split` function from `sklearn.model_selection`. The features were separated from the target variable `Diabetes_binary` and the dataset was divided with 80% for training and 20% for testing.

The output above shows that the training set contains 189,102 samples while the test set contains 47,276 samples. This split ensures that the model is trained on a majority of the data and evaluated on a separate test set to assess its performance.

3.3.3 SMOTE Techniques on Training Data

```
from imblearn.over_sampling import SMOTE

# Apply SMOTE to the training set
smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

print("Class distribution after SMOTE:")
print(y_train_smote.value_counts())
```

Figure 73: Source Code of Smote Techniques

```
import matplotlib.pyplot as plt
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split

# Visualize the class distribution before SMOTE
ax = y_train.value_counts().plot(kind='bar', title='Class Distribution Before SMOTE', color="#D291BC")
plt.xticks([0, 1], ['No Diabetes', 'Diabetes'], rotation=0)
plt.ylabel('Count')

for p in ax.patches:
    ax.annotate(f'{p.get_height():,}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center', fontsize=10, color='black', xytext=(2, 10), textcoords='offset points')
plt.ylim(0, max(y_train.value_counts()) + 10000)
plt.grid(False)
plt.tick_params(axis="x", bottom=False)
plt.show()

# Apply SMOTE to the training set to balance the classes
smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

# Visualize the class distribution after SMOTE
ax = y_train_smote.value_counts().plot(kind='bar', title='Class Distribution After SMOTE', color="#D291BC")
plt.xticks([0, 1], ['No Diabetes', 'Diabetes'], rotation=0)
plt.ylabel('Count')

for p in ax.patches:
    ax.annotate(f'{p.get_height():,}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center', fontsize=10, color='black', xytext=(2, 10), textcoords='offset points')
plt.ylim(0, max(y_train_smote.value_counts()) + 10000)
plt.grid(False)
plt.tick_params(axis="x", bottom=False)
plt.show()
```

Figure 74: Source Code of Smote Techniques


```

Class distribution after SMOTE:
Diabetes_binary
1    133406
0    133406
Name: count, dtype: int64

```

Figure 75: Output After Smote Techniques

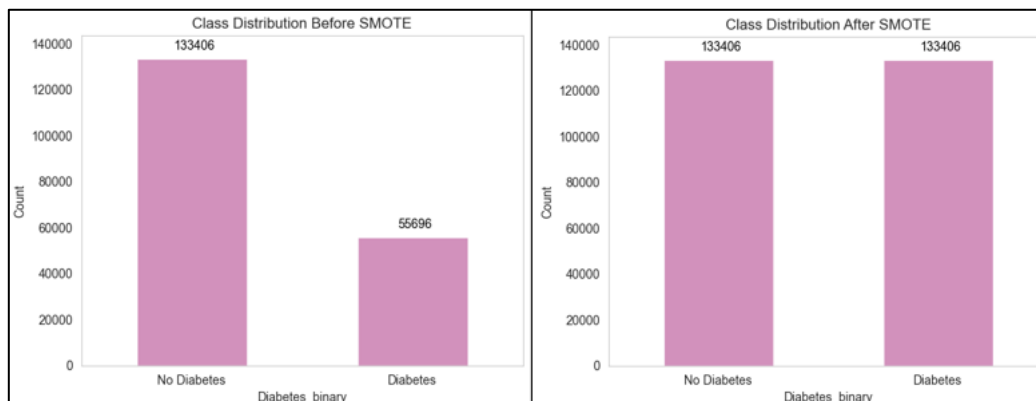


Figure 76: Bar Chart for Diabetes Count Before and After Class Imbalance

In this section, the **SMOTE (Synthetic Minority Over-sampling Technique)** technique was applied to solve the class imbalance in the training data. SMOTE is used to generate synthetic samples for the minority class by creating new and similar data points based on the existing minority class instances.

The class distribution before SMOTE shows an imbalance between the classes with No Diabetes having 133,406 instances and Diabetes having only 55,696 instances. This imbalance can lead to poor model performance for predicting the minority class.

After applying SMOTE, the class distribution was balanced with both No Diabetes and Diabetes now having 133,406 instances each. This equal distribution ensures that the model will have an equal focus on both classes which can help in improving the model's ability to predict the diabetes people more accurately.

The bar charts before and after SMOTE clearly show the impact of SMOTE on balancing the class distribution. This balance helps improve the model's performance and accuracy by reducing the bias towards the majority class during model building and tuning.

3.4 Summary of Data Preparation

The Exploratory Data Analysis (EDA) and data preprocessing processes for the diabetes prediction dataset have been thoroughly conducted. The primary goal was to explore the dataset, understand the relationships between the variables and prepare the data for model building and tuning.

The data understanding phase involved performing initial exploration to verify its structure. The dataset consists of 236,378 rows and 22 columns with all columns containing integer values and no missing data. Descriptive statistics were generated to provide insights into the distribution of values across variables. Graphical representations were created for both categorical and numerical variables, which revealed key patterns such as the prevalence of high blood pressure, high cholesterol, smoking habits and other health indicators. The analysis of the `Diabetes_binary` target variable showed an imbalanced dataset with a higher number of individuals classified as "No Diabetes" compared to those with "Diabetes."

During the preprocessing phase, outliers were identified in numerical variables such as BMI, `MentHlth` and `PhysHlth`. These outliers were solved by removing extreme values and using the winzoration method to clip the remaining outliers to ensure the data remained within reasonable bounds. Besides, the data was standardized using the `StandardScaler`, which transformed the numerical variables such as BMI, `MentHlth` and `PhysHlth` to have a mean of 0 and a standard deviation of 1. This ensured that all variables were on a comparable scale to prevent any single variable from dominating the model due to discrepancies in their ranges. Additionally, the Age variable was transformed into categorical age groups to improve the interpretability and predictive accuracy of the model.

To handle the class imbalance identified in the target variable `Diabetes_binary`, the SMOTE (Synthetic Minority Over-sampling Technique) was applied. This technique generated synthetic samples for the minority class which is the individuals with diabetes. So, this will result for a balanced distribution between the "No Diabetes" and "Diabetes" classes. This step was important to ensure that the model would not be biased towards the majority class and helps in improving the model's ability to predict the minority class more accurately.

Finally, the dataset was split into training and testing sets with 80% for training and 20% for testing. These preprocessing steps have successfully prepared the data for the next phase of the project, where deep learning models will be trained and evaluated.

In conclusion, the data has been thoroughly cleaned, balanced and transformed to ensure that it is in an optimal state for further analysis and modelling. These steps lay the foundation for building and evaluating a deep learning model to predict diabetes with improved accuracy and fairness.

4.0 Model Building

It is important to note that the model training process uses the binary cross-entropy loss function as the primary objective for optimization. This loss function calculates the difference between predicted and true labels and the model's parameters are adjusted to minimize this loss. As a result, the learning curve of loss reflects the behavior of the binary cross-entropy loss to show how the model converges during training.

However, while the binary cross-entropy loss is used for optimization, the F1 score is employed as an evaluation metric during training. The F1 score is a harmonic mean of precision and recall providing a more balanced evaluation which is important for this dataset due to its imbalanced class distribution. By using the F1 score, it ensures that the model is not just accurate overall but also performs well in handling the minority class, which is important for real-world applications where imbalanced data is common.

It is important to emphasize that the F1 score does not directly influence the loss calculation or model optimization. Instead, it serves as a monitoring tool that helps track the model's performance from a practical standpoint. This approach ensures that the model is optimized for classification accuracy through binary cross-entropy loss while also being evaluated holistically using the F1 score.

In summary, the learning curve of loss is based on the binary cross-entropy loss to standard in classification tasks, while the F1 score serves as a complementary metric to assess the model's ability to balance precision and recall for the imbalanced datasets.

4.1 Deep Neural Networks Base Model (DNNs)

4.1.1 Model Initialization

```
import numpy as np
import pandas as pd
import tensorflow as tf
import random
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc_curve, roc_auc_score, f1_score
import matplotlib.pyplot as plt
import seaborn as sns
from tensorflow.keras import backend as K
from matplotlib.ticker import FuncFormatter

# Custom F1-Score Metric
def precision_metric(y_true, y_pred):
    y_true = K.flatten(K.cast(y_true, 'float32'))
    y_pred = K.flatten(K.round(y_pred))
    tp = K.sum(y_true * y_pred)
    fp = K.sum((1 - y_true) * y_pred)
    return tp / (tp + fp + K.epsilon())

def recall_metric(y_true, y_pred):
    y_true = K.flatten(K.cast(y_true, 'float32'))
    y_pred = K.flatten(K.round(y_pred))
    tp = K.sum(y_true * y_pred)
    fn = K.sum(y_true * (1 - y_pred))
    return tp / (tp + fn + K.epsilon())

def f1_metric(y_true, y_pred):
    y_true = K.flatten(K.cast(y_true, 'float32'))
    y_pred = K.flatten(K.round(y_pred))

    tp = K.sum(y_true * y_pred)
    fp = K.sum((1 - y_true) * y_pred)
    fn = K.sum(y_true * (1 - y_pred))

    # Avoid division by zero
    precision = tp / (tp + fp + K.epsilon())
    recall = tp / (tp + fn + K.epsilon())

    return 2 * (precision * recall) / (precision + recall + K.epsilon())
```

Figure 77: Source Code of Base Model (DNNs)

Figure above shows the code in defining Deep Neural Network (DNN) base model using the TensorFlow/Keras framework for binary classification tasks. It begins by importing necessary libraries such as numpy, pandas and matplotlib for data manipulation and visualization with TensorFlow used for model creation and training. The code also includes custom precision, recall and F1-Score metrics by implemented through Keras backend functions to evaluate model performance during training.

Since the dataset is imbalanced, focusing on the F1-score is essential as it strikes a balance between precision and recall. F1-score is useful in scenarios where both false positives and false negatives have significant consequences. While accuracy is important, it may not fully reflect model effectiveness in imbalanced datasets, where one class may overpower the other. By incorporating these custom metrics, the model is assessed on not only its accuracy but also its ability to effectively classify both the positive and negative classes. This approach is important for predicting high-risk factors for diabetes because it ensures that the model performs optimally across all evaluation metrics.

```

# =====
# DNN BASE MODEL ARCHITECTURE
# =====
print("\n" + "="*50)
print("BUILDING BASE DNN MODEL ARCHITECTURE")
print("="*50)

# Deep Neural Network (DNN)
model = Sequential()

# Input layer
input_shape = X_train_smote.shape[1]
print(f"\nInput shape: {input_shape} features")

# Use Input layer to define the input shape
model.add(Input(shape=(input_shape,)))

# First hidden layer
model.add(Dense(64, activation='relu'))
print("Added Dense layer with 64 units and ReLU activation")

# Second hidden layer
model.add(Dense(32, activation='relu'))
print("Added Dense layer with 32 units and ReLU activation")

# Output layer (binary classification)
model.add(Dense(1, activation='sigmoid'))
print("Added Output layer with 1 unit and sigmoid activation")

# Compile the model
model.compile(loss='binary_crossentropy',
              optimizer=Adam(learning_rate=0.001),
              metrics=['accuracy', precision_metric, recall_metric, f1_metric])

# Model architecture
print("\nModel Architecture Summary:")
model.summary()

```

Figure 78: Source Code of Model Architecture for Base Model (DNNs)

```

=====
BUILDING BASE DNN MODEL ARCHITECTURE
=====

Input shape: 21 features
Added Dense layer with 64 units and ReLU activation
Added Dense layer with 32 units and ReLU activation
Added Output layer with 1 unit and sigmoid activation

Model Architecture Summary:

Model: "sequential_3"

```

Layer (type)	Output Shape	Param #
dense_8 (Dense)	(None, 64)	1,408
dense_9 (Dense)	(None, 32)	2,080
dense_10 (Dense)	(None, 1)	33

```

Total params: 3,521 (13.75 KB)

Trainable params: 3,521 (13.75 KB)

Non-trainable params: 0 (0.00 B)

```

Figure 79: Model Architecture for Base Model (DNNs)

The architecture itself consists of three primary layers which are an input layer, two hidden layers and an output layer. The input layer is dynamically determined by the number of features in the dataset to ensure that the model is flexible to different data shapes. The first hidden layer contains 64 units with a ReLU activation function and the second hidden layer consists of 32

units by using the ReLU activation. This helps the network capture complex patterns and relationships in the data. The output layer consists of a single unit with a sigmoid activation function, which is typical for binary classification tasks which helps in outputting probabilities between 0 and 1.

Then, the model is compiled using binary cross-entropy as the loss function which is suitable for binary classification problems. Besides, the Adam optimizer is used with a learning rate of 0.001 that is a common and effective choice for training deep learning models. Along with the standard accuracy metric, the custom metrics precision, recall and F1-score are used to evaluate the model's performance to provide a more comprehensive understanding of its ability to classify the positive and negative classes.

Finally, after building and compiling the model, a model summary is printed which shown in Figure 79. This summary provides an overview of the architecture which has including the number of parameters and their trainability. In this case, the total number of parameters is 3,521, with all parameters being trainable for allowing the model to adjust its weights based on the data it processes during training.

4.1.2 Training Process

```
# =====  
# BASE MODEL TRAINING  
# =====  
print("\n" + "="*50)  
print("TRAINING THE BASE DNN MODEL")  
print("="*50)  
  
# Add EarlyStopping to avoid overfitting  
early_stopping = EarlyStopping(  
    monitor='val_f1_metric',  
    mode='max',  
    patience=5,  
    restore_best_weights=True  
)  
  
# Train the model  
print("\nStarting model training...")  
history = model.fit(X_train_smote, y_train_smote,  
                    epochs=50,  
                    batch_size=32,  
                    validation_data=(X_test, y_test),  
                    callbacks=[early_stopping],  
                    verbose=1)
```

Figure 80: Source Code of Training Base Model (DNNs)

The source code above focuses on training the base Deep Neural Network (DNN) model for binary classification. It starts by setting up an EarlyStopping callback to prevent overfitting during training. This callback monitors the validation F1-score and halts training if the F1-score does not improve for 5 consecutive epochs. Then, the model is trained using the fit() function with training data (X_train_smote and y_train_smote) and validation data (X_test and y_test). Besides, the training process runs for a maximum of 50 epochs with a batch size of 32. The verbose=1 argument ensures that the progress of training is displayed which includes the loss and accuracy for each epoch. The EarlyStopping callback ensures that the best model based on validation performance and is selected by restoring the weights from the epoch with the highest validation F1-score.

```
=====
TRAINING THE BASE DNN MODEL
=====

Starting model training...
Epoch 1/50
8338/8338 — 17s 2ms/step - accuracy: 0.8564 - f1_metric: 0.8493 - loss: 0.3424 - precision_metric: 0.8710 - recall_metric: 0.8384 -
Epoch 2/50
8338/8338 — 19s 2ms/step - accuracy: 0.8637 - f1_metric: 0.8554 - loss: 0.3270 - precision_metric: 0.8855 - recall_metric: 0.8356 -
Epoch 3/50
8338/8338 — 14s 2ms/step - accuracy: 0.8653 - f1_metric: 0.8566 - loss: 0.3235 - precision_metric: 0.8900 - recall_metric: 0.8338 -
Epoch 4/50
8338/8338 — 17s 2ms/step - accuracy: 0.8658 - f1_metric: 0.8568 - loss: 0.3216 - precision_metric: 0.8919 - recall_metric: 0.8323 -
Epoch 5/50
8338/8338 — 21s 3ms/step - accuracy: 0.8663 - f1_metric: 0.8578 - loss: 0.3203 - precision_metric: 0.8919 - recall_metric: 0.8339 -
Epoch 6/50
8338/8338 — 18s 2ms/step - accuracy: 0.8670 - f1_metric: 0.8582 - loss: 0.3197 - precision_metric: 0.8945 - recall_metric: 0.8327 -
Epoch 7/50
8338/8338 — 16s 2ms/step - accuracy: 0.8672 - f1_metric: 0.8586 - loss: 0.3188 - precision_metric: 0.8946 - recall_metric: 0.8330 -
Epoch 8/50
8338/8338 — 16s 2ms/step - accuracy: 0.8677 - f1_metric: 0.8589 - loss: 0.3181 - precision_metric: 0.8944 - recall_metric: 0.8339 -
```

Figure 81: Model Training for Base Model (DNNs)


```

val_accuracy: 0.8874 - val_f1_metric: 0.8047 - val_loss: 0.3056 - val_precision_metric: 0.7947 - val_recall_metric: 0.8338
val_accuracy: 0.8819 - val_f1_metric: 0.7998 - val_loss: 0.3097 - val_precision_metric: 0.7731 - val_recall_metric: 0.8486
val_accuracy: 0.8906 - val_f1_metric: 0.8076 - val_loss: 0.3087 - val_precision_metric: 0.8075 - val_recall_metric: 0.8266
val_accuracy: 0.8836 - val_f1_metric: 0.8024 - val_loss: 0.3117 - val_precision_metric: 0.7761 - val_recall_metric: 0.8492
val_accuracy: 0.8829 - val_f1_metric: 0.8010 - val_loss: 0.3150 - val_precision_metric: 0.7753 - val_recall_metric: 0.8483
val_accuracy: 0.8870 - val_f1_metric: 0.8062 - val_loss: 0.3079 - val_precision_metric: 0.7880 - val_recall_metric: 0.8443
val_accuracy: 0.8840 - val_f1_metric: 0.8032 - val_loss: 0.3170 - val_precision_metric: 0.7771 - val_recall_metric: 0.8511
val_accuracy: 0.8858 - val_f1_metric: 0.8047 - val_loss: 0.3086 - val_precision_metric: 0.7848 - val_recall_metric: 0.8457

```

Figure 82: Model Training for Base Model (DNNs)

The figures above show the training progress of the DNN base model which includes both training and validation metrics for each epoch. During training, the model's accuracy, F1-score, precision and recall values are calculated for both the training and validation datasets. The model's performance is monitored throughout the training process to avoid overfitting using the EarlyStopping callback. This ensures the model's parameters are updated in a way that minimizes the validation loss and maximizes the validation F1-score.

As shown in the figure above, as the epochs increase, the training accuracy, precision, recall and F1-score all improve for both the training and validation datasets. This indicates that the model is learning effectively and generalizing well to unseen data. The validation metrics such as F1-score are especially useful here since they give a better sense of how well the model is performing in a class-imbalanced dataset like predicting diabetes. Additionally, the validation loss is gradually decreasing which indicates that the model is becoming more accurate and reliable over time.

In conclusion, the training and validation results demonstrate that the DNN model is progressing well in terms of both accuracy and the custom metrics of precision, recall and F1-score.

4.1.3 Evaluation Results

```
# =====  
# MODEL EVALUATION  
# =====  
print("\n" + "-"*50)  
print("MODEL EVALUATION")  
print("-"*50)  
  
# Evaluate the model on the test data  
y_pred = model.predict(X_test)  
y_pred_class = (y_pred > 0.5).astype(int)  
  
# Calculate accuracy and print metrics  
accuracy = accuracy_score(y_test, y_pred_class)  
f1 = f1_score(y_test, y_pred_class)  
  
print(f"\nAccuracy: {accuracy:.4f}")  
print(f"F1-Score: {f1:.4f}")  
  
# Confusion Matrix  
cm = confusion_matrix(y_test, y_pred_class)  
print("\nConfusion Matrix:")  
print(cm)  
  
# Classification Report  
print("\nClassification Report:")  
print(classification_report(y_test, y_pred_class))
```

Figure 83: Source Code of Model Evaluation for Base Model (DNNs)

```
# =====  
# VISUALIZATIONS  
# =====  
plt.figure(figsize=(12, 6))  
  
# Accuracy plot  
plt.subplot(1, 2, 1)  
plt.plot(history.history['accuracy'], label='Training Accuracy')  
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')  
plt.title('Learning Curve for Base DNN (Accuracy)')  
plt.xlabel('Epochs')  
plt.ylabel('Accuracy')  
plt.legend()  
plt.gca().yaxis.set_major_formatter(FuncFormatter(lambda x, _: f'{x:.2f}'))  
  
# Loss plot  
plt.subplot(1, 2, 2)  
plt.plot(history.history['loss'], label='Training Loss')  
plt.plot(history.history['val_loss'], label='Validation Loss')  
plt.title('Learning Curve for Base DNN (Loss)')  
plt.xlabel('Epochs')  
plt.ylabel('Loss')  
plt.legend()  
plt.gca().yaxis.set_major_formatter(FuncFormatter(lambda x, _: f'{x:.2f}'))  
  
plt.tight_layout()  
plt.show()  
  
# 2. Confusion Matrix  
plt.figure(figsize=(8, 6))  
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',  
            xticklabels=['Predicted Negative', 'Predicted Positive'],  
            yticklabels=['Actual Negative', 'Actual Positive'])  
plt.title('Confusion Matrix for Base DNN Model')  
plt.xlabel('Predicted Label')  
plt.ylabel('True Label')  
plt.show()  
  
# Calculate ROC AUC  
fpr, tpr, thresholds = roc_curve(y_test, y_pred)  
roc_auc = roc_auc_score(y_test, y_pred)  
  
# Plot ROC curve  
plt.figure(figsize=(8, 6))  
plt.plot(fpr, tpr, color='b', label=f'ROC curve (AUC = {roc_auc:.2f})')  
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')  
plt.xlabel('False Positive Rate')  
plt.ylabel('True Positive Rate')  
plt.title('ROC Curve for Base DNN Model')  
plt.legend(loc='lower right')  
plt.show()
```

Figure 84: Source Code of Model Evaluation for Base Model (DNNs)

The code above evaluates the performance of the trained Deep Neural Network (DNN) base model on the test data by predicting the outcomes and converting the predicted probabilities into class labels by using a threshold of 0.5. It calculates and prints the accuracy and F1-score, where accuracy shows the overall prediction correctness and the F1-score provides a balanced metric between precision and recall which are important in class-imbalanced tasks. Additionally, the confusion matrix is generated to assess the true and false positive and negative predictions, and the classification report is printed to provide a detailed breakdown of precision, recall and F1-score for each class. Additionally, the code also includes visualizations such as the learning curve for accuracy and loss over the epochs and the confusion matrix as a heatmap for better analysis. Lastly, the ROC curve is plotted, and the AUC score is calculated to evaluate the model's ability to differentiate between the two classes that offers insights into its overall performance.

4.1.3.1 Confusion Matrix and Classification Report

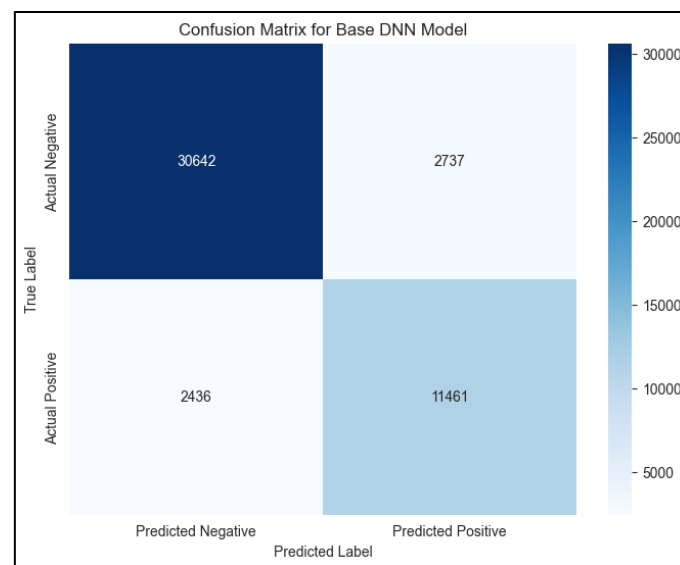


Figure 85: Confusion Matrix for Base Model (DNNs)

The figure above shows the confusion matrix for the Base DNN Model. It represents the model's performance of true positive, true negative, false positive and false negative predictions. The confusion matrix shows that the model has correctly predicted 30,642 true negatives (non-diabetic individuals) and 11,461 true positives (diabetic individuals). However, it has made 2,737 false positives (non-diabetic individuals classified as diabetic) and 2,436 false negatives (diabetic individuals classified as non-diabetic). This matrix helps visualize the balance between the actual and predicted values to provide insights into the model's error distribution.

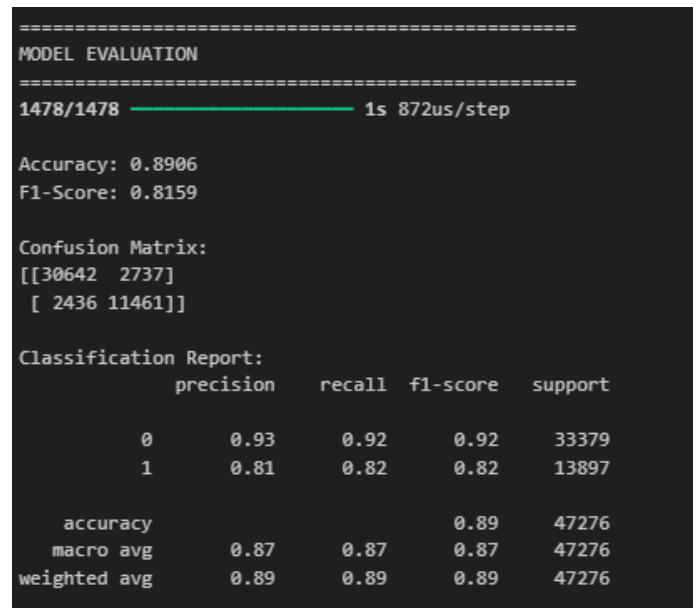


Figure 86: Classification Report for Base Model (DNNs)

The figure above shows the classification report for the DNN Base Model. The precision for non-diabetic individuals is 93% which shows there is predicted non-diabetic cases are correctly identified. For diabetic individuals, the precision is 81% which indicates there is 81% of the predicted diabetic cases are correct. The recall for non-diabetic individuals is 92% and this indicates that 92% of the actual non-diabetic cases are correctly classified, while for diabetic individuals there is 82% of the actual diabetic cases are correctly identified. The F1-score for non-diabetic individuals is 92% and for diabetic individuals is 82%. The overall accuracy of the model is 89% and this is indicating strong overall model performance. These metrics suggest that the model is performing well especially for non-diabetic individuals thus there is still have improvement in detecting diabetic individuals.

4.1.3.2 Learning Curve (Accuracy)

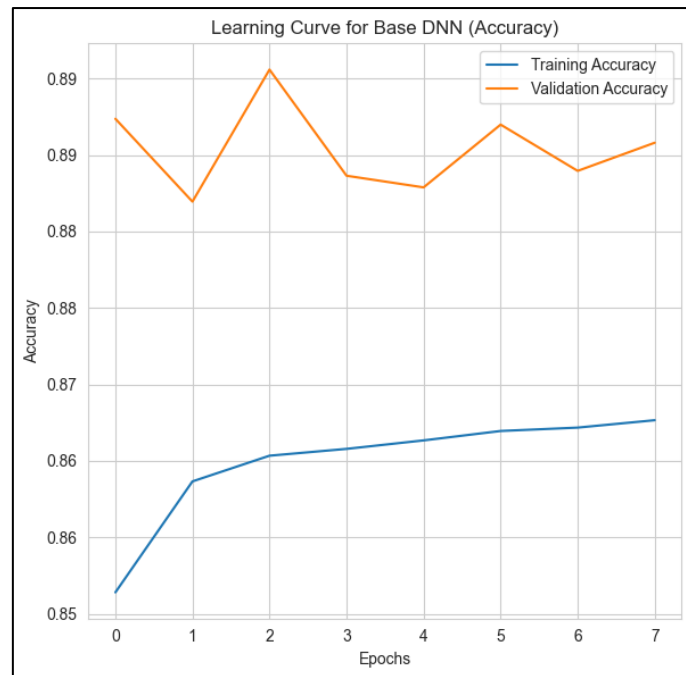


Figure 87: Learning Curve (Accuracy) for Base Model (DNNs)

The accuracy curve provides insight into how well the model is performing in terms of correctly classifying samples over the course of training. As shown in the figure above, the training accuracy starts at approximately 86%, indicating that the model initially struggles with correct classification. However, it steadily increases with each epoch and is reaching about 89% by the end of the training. This consistent improvement in training accuracy suggests that the model is effectively learning the patterns from the training data.

The validation accuracy reflects how well the model generalizes to unseen data and, although it fluctuates slightly, it shows a consistent increase over time. These fluctuations could be due to small changes in the model's generalization ability from epoch to epoch. Despite these minor oscillations, the validation accuracy reaches a similar value and is indicating that the model is not overfitting to the training data but rather generalizing well. The close proximity between the training and validation accuracy values suggests that the model is performing effectively across both datasets. However, the minor fluctuations in validation accuracy indicate that further fine-tuning could help reduce this variance and improve consistency.

In summary, the accuracy learning curve suggests steady and consistent model improvement which has indicating that the model is effectively learning to classify the data with minimal overfitting or underfitting. However, additional optimization may help reduce validation accuracy fluctuations for more stable performance.

4.1.3.3 Learning Curve (Loss)

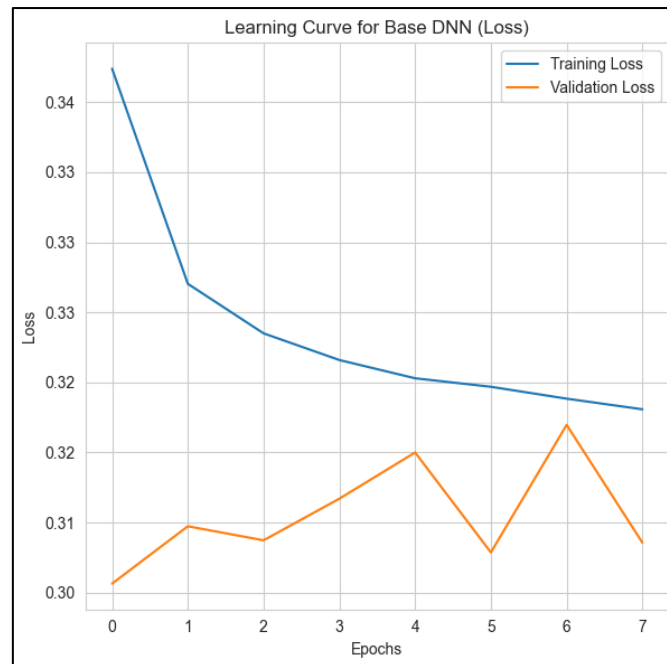


Figure 88: Learning Curve (Loss) for Base Model (DNNs)

The loss curve shows the model's error rate over time, which helps to assess how well the model is reducing its error during training. In the figure above, the training loss starts relatively high which is indicating that the model's initial predictions are far from accurate. However, as the epochs progress, the training loss decreases significantly and is stabilizing at around 0.31 after the initial drop. This sharp decline at the start is expected as the model begins to learn and adapt to the data, and the flattening of the curve indicates that the model is starting to converge to a good solution.

The validation loss which are used to measure the model's error on unseen data also shows a similar pattern. It decreases initially and then flattens out with small fluctuations observed in the later epochs. The stabilization of the validation loss close to the training loss indicates that the model is not overfitting. However, the minor fluctuations in the validation loss could indicate that the model might benefit from further adjustments such as a smaller learning rate or better regularization.

In summary, the loss curve indicates that the model is learning effectively and generalizing well to the validation data. While the loss values stabilize, the slight oscillations in the validation loss suggest potential room for fine-tuning to improve stability.

4.1.3.4 ROC and AUC Score

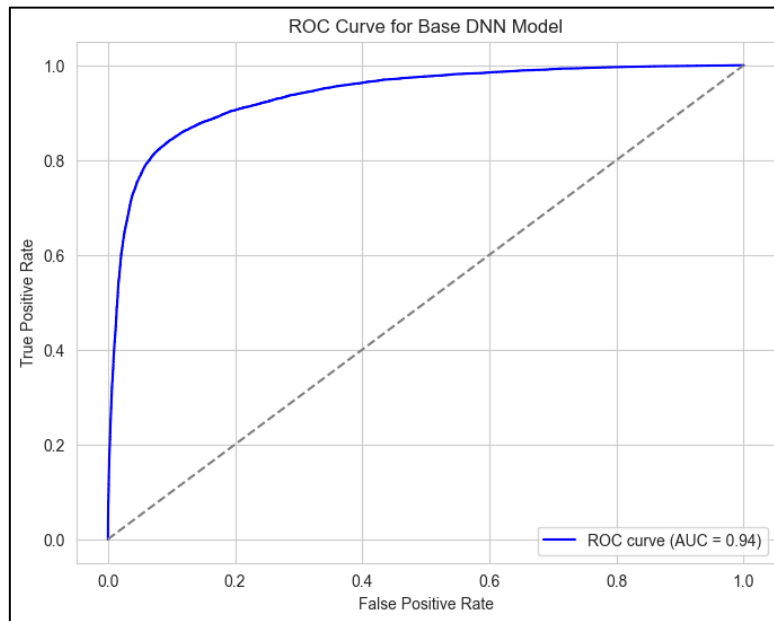


Figure 89 : ROC and AUC Score for Base Model (DNNs)

The ROC curve shown in the figure represents the model's ability to distinguish between the positive and negative classes. With a high True Positive Rate (TPR) and a low False Positive Rate (FPR), the model achieves an AUC score of 0.94, which indicates excellent classification performance. The ROC curve clearly shows that the model performs well in distinguishing between classes as the curve is far above the diagonal line. The high AUC score suggests that the model has a strong capacity to correctly identify both positive and negative samples in the dataset. This is very important for applications like diabetes risk prediction to distinguish between high-risk and low-risk individuals.

4.2 Convolutional Neural Network (CNN)

4.2.1 Model Initialization

```
import numpy as np
import pandas as pd
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatten, Dense, Dropout, Reshape, Input
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc_curve, roc_auc_score, f1_score
import matplotlib.pyplot as plt
from tensorflow.keras import backend as K
from matplotlib.ticker import FuncFormatter

def precision_metric(y_true, y_pred):
    y_true = K.flatten(K.cast(y_true, 'float32'))
    y_pred = K.flatten(K.round(y_pred))
    tp = K.sum(y_true * y_pred)
    fp = K.sum((1 - y_true) * y_pred)
    return tp / (tp + fp + K.epsilon())

def recall_metric(y_true, y_pred):
    y_true = K.flatten(K.cast(y_true, 'float32'))
    y_pred = K.flatten(K.round(y_pred))
    tp = K.sum(y_true * y_pred)
    fn = K.sum(y_true * (1 - y_pred))
    return tp / (tp + fn + K.epsilon())

def f1_metric(y_true, y_pred):
    y_true = K.flatten(K.cast(y_true, 'float32'))
    y_pred = K.flatten(K.round(y_pred))

    tp = K.sum(y_true * y_pred)
    fp = K.sum((1 - y_true) * y_pred)
    fn = K.sum(y_true * (1 - y_pred))

    precision = tp / (tp + fp + K.epsilon())
    recall = tp / (tp + fn + K.epsilon())

    return 2 * (precision * recall) / (precision + recall + K.epsilon())
```

Figure 90: Source Code for Convolutional Neural Network Base Model (CNN)

Figure above shows the code in defining Convolutional Neural Network (CNN) Base Model using the TensorFlow/Keras framework for binary classification tasks. It begins by importing necessary libraries such as numpy, pandas and matplotlib for data manipulation and visualization with TensorFlow used for model creation and training. The code also includes custom precision, recall and F1-Score metrics by implemented through Keras backend functions to evaluate model performance during training.

Since the dataset is imbalanced, focusing on the F1-score is essential as it strikes a balance between precision and recall. F1-score is useful in scenarios where both false positives and false negatives have significant consequences. While accuracy is important, it may not fully reflect model effectiveness in imbalanced datasets, where one class may overpower the other. By incorporating these custom metrics, the model is assessed on not only its accuracy but also its ability to effectively classify both the positive and negative classes. This approach is important for predicting high-risk factors for diabetes because it ensures that the model performs optimally across all evaluation metrics.


```

# =====
# CNN MODEL ARCHITECTURE
# =====
print("\n" + "="*50)
print("BUILDING BASE CNN MODEL ARCHITECTURE")
print("="*50)

# Build the CNN model
model = Sequential()

# Input shape for 1D convolution
input_shape = X_train_smote.shape[1:]
print(f"\nInput shape: {input_shape} features")

model.add(Input(shape=input_shape))
model.add(Reshape((input_shape[0], 1)))
print(f"Added Input layer with shape {input_shape}")
print(f"Added Reshape layer to convert {input_shape[0]} features to (features, 1) for Conv1D")

# First Conv1D layer
model.add(Conv1D(filters=64, kernel_size=3, activation='relu'))
print("Added Conv1D layer with 64 filters, kernel_size=3, and ReLU activation")

# Pooling layer
model.add(MaxPooling1D(pool_size=2))
print("Added MaxPooling1D layer with pool_size=2")

# Second Conv1D layer
model.add(Conv1D(filters=32, kernel_size=2, activation='relu'))
print("Added Conv1D layer with 32 filters, kernel_size=2, and ReLU activation")

# Flatten layer to transition to dense layers
model.add(Flatten())
print("Added Flatten layer")

# Dense layer
model.add(Dense(32, activation='relu'))
print("Added Dense layer with 32 units and ReLU activation")

# Dropout for regularization
model.add(Dropout(0.5))
print("Added Dropout layer with rate=0.5")

# Output layer
model.add(Dense(1, activation='sigmoid'))
print("Added Output layer with 1 unit and sigmoid activation")

# Compile the model
model.compile(loss='binary_crossentropy',
              optimizer=Adam(learning_rate=0.001),
              metrics=['accuracy', precision_metric, recall_metric, f1_metric])

# Model architecture
print("\nModel Architecture Summary:")
model.summary()

```

Figure 91: Source Code of Model Architecture for Base Model (CNN)

```

=====
BUILDING BASE CNN MODEL ARCHITECTURE
=====

Input shape: (21,) features
Added Input layer with shape (21,)
Added Reshape layer to convert 21 features to (features, 1) for Conv1D
Added Conv1D layer with 64 filters, kernel_size=3, and ReLU activation
Added MaxPooling1D layer with pool_size=2
Added Conv1D layer with 32 filters, kernel_size=2, and ReLU activation
Added Flatten layer
Added Dense layer with 32 units and ReLU activation
Added Dropout layer with rate=0.5
Added Output layer with 1 unit and sigmoid activation

Model Architecture Summary:

Model: "sequential_1"


```

Layer (type)	Output Shape	Param #
reshape (Reshape)	(None, 21, 1)	0
conv1d (Conv1D)	(None, 19, 64)	256
max_pooling1d (MaxPooling1D)	(None, 9, 64)	0
conv1d_1 (Conv1D)	(None, 8, 32)	4,128
flatten (Flatten)	(None, 256)	0
dense_3 (Dense)	(None, 32)	8,224
dropout (Dropout)	(None, 32)	0
dense_4 (Dense)	(None, 1)	33

```

Total params: 12,641 (49.38 KB)

Trainable params: 12,641 (49.38 KB)

Non-trainable params: 0 (0.00 B)

```

Figure 92: Model Architecture for Base Model (CNN)

The code above builds the base CNN (Convolutional Neural Network) model for predicting diabetes risk using the TensorFlow/Keras framework. The model starts with an input layer that accepts the data and followed by a reshape layer to adjust the data into a suitable format for 1D convolution. The first convolutional layer uses 64 filters with a kernel size of 3 and applies ReLU activation to learn spatial features from the data. This is followed by a MaxPooling layer to reduce the spatial dimensions and down-sample the features.

The second convolutional layer with 32 filters and kernel size 2 continues learning more complex features. After the convolutional layers, a Flatten layer is applied to reshape the output into a 1D array which is suitable for the fully connected (dense) layers.

Then, the model has a dense layer with 32 units and ReLU activation to further learn non-linear combinations of features. A Dropout layer with a rate of 0.5 is used for regularization to prevent overfitting. Finally, the model ends with an output layer with 1 unit and a sigmoid activation function, suitable for binary classification.

Finally, the model is compiled using binary cross-entropy loss, the Adam optimizer with a learning rate of 0.001 and performance metrics including accuracy, precision, recall and F1-score. These metrics are important for evaluating how well the model performs in classifying both diabetic and non-diabetic individuals especially for the imbalanced dataset. The architecture uses a sequential model where each layer is added one after the other to progressively learn complex features.

4.2.2 Training Process

```
# =====  
# MODEL TRAINING  
# =====  
print("\n" + "="*50)  
print("TRAINING THE CNN MODEL")  
print("="*50)  
  
# Add EarlyStopping to avoid overfitting  
early_stopping = EarlyStopping(  
    monitor='val_f1_metric',  
    mode='max',  
    patience=5,  
    restore_best_weights=True  
)  
  
# Train the model  
print("\nStarting model training...")  
history = model.fit(X_train_smote, y_train_smote,  
                    epochs=50,  
                    batch_size=32,  
                    validation_data=(X_test, y_test),  
                    callbacks=[early_stopping],  
                    verbose=1)
```

Figure 93: Source Code for Training Base Model (CNN)

The code above defines the training process for the CNN model. First, it implements early stopping which is a technique used to prevent overfitting by monitoring the validation F1 score during training. If the F1 score does not improve for 5 consecutive epochs, the training will stop and the model with the best weights is restored.

The model is then trained using the fit() method, where the training data (X_train_smote and y_train_smote) is used to learn from the patterns. The training runs for 50 epochs with a batch size of 32 and the model's performance is validated on the test data (X_test and y_test) during each epoch. The verbose=1 option allows for progress information to be printed during training.

Overall, this step helps ensure that the model is trained efficiently. The early stopping mechanism prevents overfitting to the training data while allowing the model to generalize well to unseen data.

```

=====
TRAINING THE CNN MODEL
=====

Starting model training...
Epoch 1/50
8338/8338 ----- 28s 3ms/step - accuracy: 0.8514 - f1_metric: 0.8422 - loss: 0.3660 - precision_metric: 0.8743 - recall_metric: 0.8218 -
Epoch 2/50
8338/8338 ----- 31s 4ms/step - accuracy: 0.8572 - f1_metric: 0.8479 - loss: 0.3542 - precision_metric: 0.8836 - recall_metric: 0.8236 -
Epoch 3/50
8338/8338 ----- 31s 4ms/step - accuracy: 0.8584 - f1_metric: 0.8489 - loss: 0.3517 - precision_metric: 0.8864 - recall_metric: 0.8227 -
Epoch 4/50
8338/8338 ----- 31s 4ms/step - accuracy: 0.8591 - f1_metric: 0.8492 - loss: 0.3505 - precision_metric: 0.8877 - recall_metric: 0.8220 -
Epoch 5/50
8338/8338 ----- 31s 4ms/step - accuracy: 0.8594 - f1_metric: 0.8499 - loss: 0.3494 - precision_metric: 0.8882 - recall_metric: 0.8230 -
Epoch 6/50
8338/8338 ----- 30s 4ms/step - accuracy: 0.8597 - f1_metric: 0.8498 - loss: 0.3489 - precision_metric: 0.8887 - recall_metric: 0.8225 -
Epoch 7/50
8338/8338 ----- 40s 3ms/step - accuracy: 0.8604 - f1_metric: 0.8506 - loss: 0.3474 - precision_metric: 0.8896 - recall_metric: 0.8232 -
Epoch 8/50
8338/8338 ----- 30s 4ms/step - accuracy: 0.8612 - f1_metric: 0.8516 - loss: 0.3478 - precision_metric: 0.8904 - recall_metric: 0.8241 -
Epoch 9/50
8338/8338 ----- 30s 4ms/step - accuracy: 0.8606 - f1_metric: 0.8512 - loss: 0.3471 - precision_metric: 0.8884 - recall_metric: 0.8249 -
Epoch 10/50
8338/8338 ----- 30s 4ms/step - accuracy: 0.8613 - f1_metric: 0.8515 - loss: 0.3463 - precision_metric: 0.8905 - recall_metric: 0.8237 -
Epoch 11/50
8338/8338 ----- 29s 4ms/step - accuracy: 0.8614 - f1_metric: 0.8519 - loss: 0.3458 - precision_metric: 0.8900 - recall_metric: 0.8249 -
Epoch 12/50
8338/8338 ----- 29s 4ms/step - accuracy: 0.8614 - f1_metric: 0.8519 - loss: 0.3458 - precision_metric: 0.8900 - recall_metric: 0.8249 -

```

Figure 94: Model Training for Base Model (CNN)

```

val_accuracy: 0.8670 - val_f1_metric: 0.7823 - val_loss: 0.3595 - val_precision_metric: 0.7360 - val_recall_metric: 0.8564
val_accuracy: 0.8705 - val_f1_metric: 0.7861 - val_loss: 0.3327 - val_precision_metric: 0.7445 - val_recall_metric: 0.8539
val_accuracy: 0.8755 - val_f1_metric: 0.7917 - val_loss: 0.3276 - val_precision_metric: 0.7575 - val_recall_metric: 0.8499
val_accuracy: 0.8842 - val_f1_metric: 0.8000 - val_loss: 0.3218 - val_precision_metric: 0.7881 - val_recall_metric: 0.8314
val_accuracy: 0.8818 - val_f1_metric: 0.7982 - val_loss: 0.3183 - val_precision_metric: 0.7787 - val_recall_metric: 0.8386
val_accuracy: 0.8847 - val_f1_metric: 0.7999 - val_loss: 0.3105 - val_precision_metric: 0.7896 - val_recall_metric: 0.8299
val_accuracy: 0.8770 - val_f1_metric: 0.7940 - val_loss: 0.3263 - val_precision_metric: 0.7591 - val_recall_metric: 0.8522
val_accuracy: 0.8610 - val_f1_metric: 0.7770 - val_loss: 0.3458 - val_precision_metric: 0.7193 - val_recall_metric: 0.8673
val_accuracy: 0.8854 - val_f1_metric: 0.8006 - val_loss: 0.3192 - val_precision_metric: 0.7938 - val_recall_metric: 0.8266
val_accuracy: 0.8806 - val_f1_metric: 0.7964 - val_loss: 0.3097 - val_precision_metric: 0.7734 - val_recall_metric: 0.8406
val_accuracy: 0.8806 - val_f1_metric: 0.7969 - val_loss: 0.3240 - val_precision_metric: 0.7736 - val_recall_metric: 0.8415

```

Figure 95: Model Training for Base Model (CNN)

The figure above shows the CNN model training process by displaying key metrics such as accuracy, F1-score, precision, recall and loss for each epoch. The training process begins with lower accuracy and F1-score values which are gradually improving as the model learns and adapts. For example, accuracy improves from around 85.14% to 86.14%, with the F1-score increasing steadily over the epochs. This shows that the model is learning to distinguish between diabetic and non-diabetic individuals more effectively.

The validation metrics for each epoch show that the validation accuracy fluctuates slightly but exhibits an overall upward trend. This indicates the model's ability to generalize well to unseen data. Additionally, the validation loss decreases, suggesting that the model is effectively minimizing its error. The F1-score for the validation data mirrors the trend seen in the training F1-score which reflects a balanced performance in terms of precision and recall. However, fluctuations in both the validation accuracy and F1-score suggest that there may be further tuning to improve the model's consistency and performance.

The training and validation metrics show the model's improving ability to identify both diabetic and non-diabetic individuals. The F1-score is used for balancing precision and recall that is important for diabetes prediction, where false positives and negatives are significant. The model shows effective learning with validation metrics supporting this. The further fine-tuning could help stabilize fluctuations seen in later epochs.

4.2.3 Evaluation Results

```
# =====  
# MODEL EVALUATION  
# =====  
print("\n" + "="*50)  
print("MODEL EVALUATION")  
print("="*50)  
  
# Evaluate the model on the test data  
y_pred = model.predict(X_test)  
y_pred_class = (y_pred > 0.5).astype(int)  
  
# Calculate accuracy and print metrics  
accuracy = accuracy_score(y_test, y_pred_class)  
f1 = f1_score(y_test, y_pred_class)  
  
print(f"\nAccuracy: {accuracy:.4f}")  
print(f"F1-Score: {f1:.4f}")  
  
# Confusion Matrix  
cm = confusion_matrix(y_test, y_pred_class)  
print("\nConfusion Matrix:")  
print(cm)  
  
# Classification Report  
print("\nClassification Report:")  
print(classification_report(y_test, y_pred_class))
```

Figure 96: Source Code of Model Evaluation for Base Model (CNN)

```

# =====
# VISUALIZATIONS
# =====
plt.figure(figsize=(12, 6))

# Accuracy plot
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Learning Curve for Base CNN (Accuracy)')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.gca().yaxis.set_major_formatter(FuncFormatter(lambda x, _: f'{x:.2f}'))

# Loss plot
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Learning Curve for Base CNN (Loss)')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.gca().yaxis.set_major_formatter(FuncFormatter(lambda x, _: f'{x:.2f}'))

plt.tight_layout()
plt.show()

# Confusion Matrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm,
            annot=True,
            fmt='d',
            cmap='Blues',
            xticklabels=['Predicted Negative', 'Predicted Positive'],
            yticklabels=['Actual Negative', 'Actual Positive'])
plt.title('Confusion Matrix for Base CNN Model')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()

# Calculate ROC AUC
fpr, tpr, thresholds = roc_curve(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_pred)

# Plot ROC curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='b', label=f'ROC curve (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for Base CNN Model')
plt.legend(loc='lower right')
plt.show()

```

Figure 97: Source Code of Model Evaluation for Base Model (CNN)

The code above evaluates the performance of the trained Convolutional Neural Network (CNN) base model on the test data by predicting the outcomes and converting the predicted probabilities into class labels by using a threshold of 0.5. It calculates and prints the accuracy and F1-score, where accuracy shows the overall prediction correctness and the F1-score provides a balanced metric between precision and recall which are important in class-imbalanced tasks. Additionally, the confusion matrix is generated to assess the true and false positive and negative predictions, and the classification report is printed to provide a detailed breakdown of precision, recall and F1-score for each class. Additionally, the code also includes visualizations such as the learning curve for accuracy and loss over the epochs and the confusion matrix as a heatmap for better analysis. Lastly, the ROC curve is plotted, and the AUC score is calculated to evaluate the model's ability to differentiate between the two classes that offers insights into its overall performance.

4.2.3.1 Confusion Matrix and Classification Report

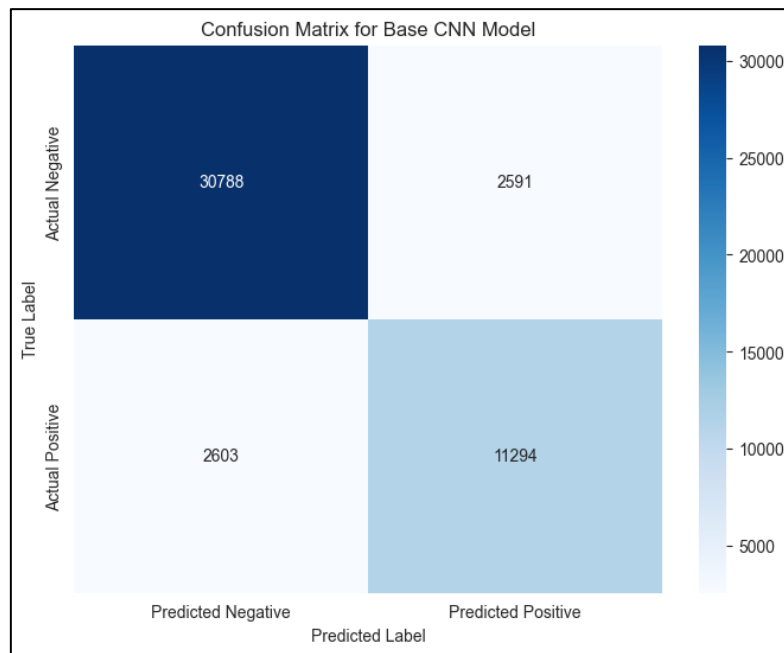


Figure 98: Confusion Matrix for Base Model (CNN)

The figure above displays the confusion matrix for the Base CNN Model. It shows the model's performance in terms of true positives, true negatives, false positives and false negatives. The model has correctly predicted 30,788 true negatives (non-diabetic individuals) and 11,294 true positives (diabetic individuals). However, it made 2,603 false positives (non-diabetic individuals predicted as diabetic) and 2,591 false negatives (diabetic individuals predicted as non-diabetic). This matrix provides insight into how well the model distinguishes between diabetic and non-diabetic individuals. This highlights the error distribution and balance between the two classes.

```

=====
MODEL EVALUATION
=====
1478/1478 ————— 2s 1ms/step

Accuracy: 0.8901
F1-Score: 0.8130

Confusion Matrix:
[[30788 2591]
 [ 2603 11294]]

Classification Report:

```

	precision	recall	f1-score	support
0	0.92	0.92	0.92	33379
1	0.81	0.81	0.81	13897
accuracy			0.89	47276
macro avg	0.87	0.87	0.87	47276
weighted avg	0.89	0.89	0.89	47276

Figure 99: Classification Report for Base Model (CNN)

The figure above shows the classification report for the CNN Base Model which evaluates its performance for predicting non-diabetic (0) and diabetic (1) individuals. The precision for non-diabetic individuals (class 0) shows that there is 92% of the predicted non-diabetic cases are correctly identified. For diabetic individuals (class 1), precision is 81% is indicating a lower success rate in identifying diabetic individuals compared to non-diabetic ones. The recall for class 0 has 92% of actual non-diabetic cases are correctly classified, while recall for class 1 has 81% of the actual diabetic individuals are correctly identified. Besides, there is 92% of F1-score for non-diabetic individuals and 81% for diabetic individuals which is reflecting the balance between precision and recall.

This CNN Base Model's classification report highlights that the model is well-balanced in identifying non-diabetic individuals to ensure minimal false positives. It also shows that while the model performs well for detecting diabetic individuals, there is still room for improvement in its ability to identify all diabetic cases (true positives). The overall accuracy of 89% indicates strong model performance. This suggests the model is effective for predicting diabetes risk, but further fine-tuning is needed to improve predictions for diabetic individuals.

4.2.3.2 Learning Curve (Accuracy)

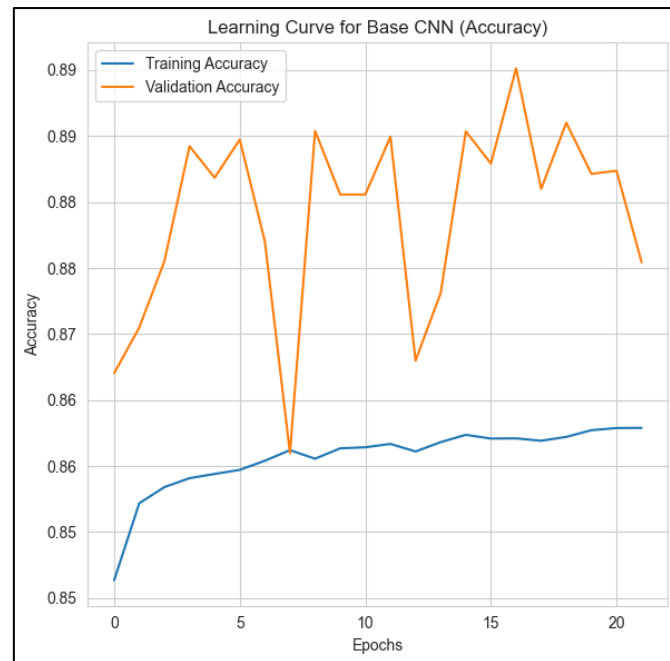


Figure 100: Learning Curve (Accuracy) for Base Model (CNN)

The Learning Curve (Accuracy) indicates a steady improvement in the model's performance over time. The training accuracy starts at approximately 85% and increases gradually to around 88%. This suggests that the model is effectively learning from the data by adjusting its weights to reduce prediction errors. The validation accuracy follows a similar trend with starting lower but increasing steadily. This indicates the model is not overfitting despite the fluctuations in the early epochs. The fluctuations in the validation accuracy at the beginning may be a result of the model's sensitivity to the initial random weights or the learning rate, but it generally stabilizes over time. The relatively close alignment between the training and validation accuracy suggests that the model is generalizing well to unseen data.

4.2.3.3 Learning Curve (Loss)

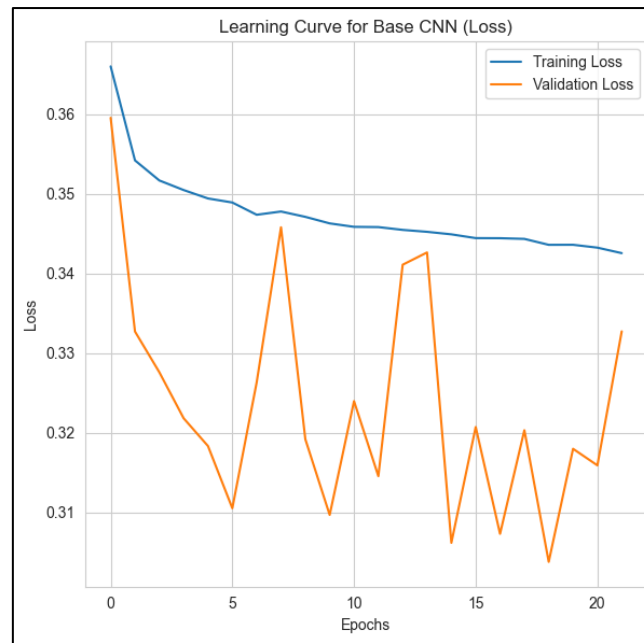


Figure 101: Learning Curve (Loss) for Base Model (CNN)

The Learning Curve (Loss) presents a more nuanced view of the model's training. The training loss decreases rapidly in the early epochs which is reflecting the model's ability to reduce error and improve prediction accuracy. This rapid decrease suggests that the model is quickly adjusting its parameters and learning from the data. However, the validation loss shows more fluctuations and decreases more slowly. This fluctuation in the validation loss is indicative of the model's occasional overfitting to the training data, especially in later epochs, where the validation loss could slightly increase. Despite these fluctuations, the validation loss generally trends downward which is indicating that the model is learning to generalize better. The fact that the validation loss doesn't steadily increase is a positive sign which indicates that the model is not overfitting significantly. However, there is still room for further optimization to improve its performance.

4.2.3.4 ROC and AUC Score

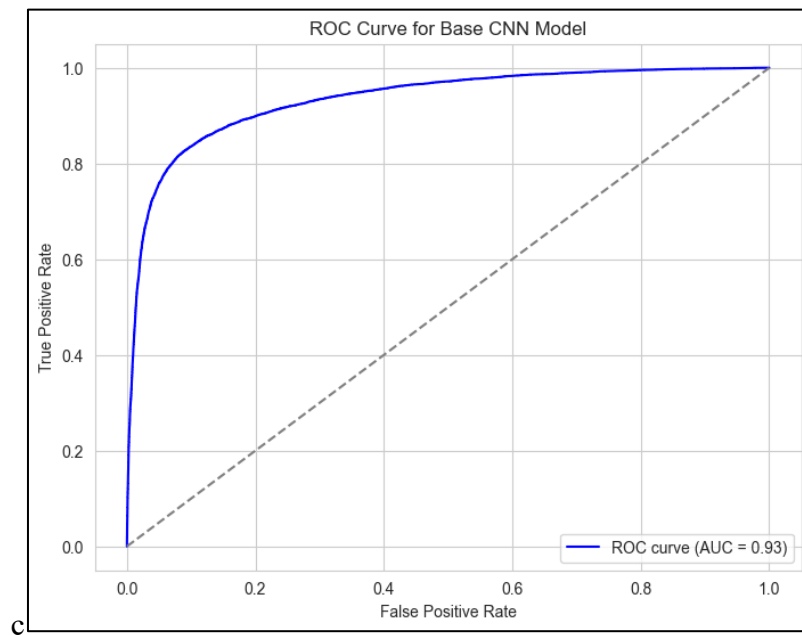


Figure 102: ROC and AUC Score for Base Model (CNN)

The ROC curve for the Base CNN model demonstrates its ability to distinguish between diabetic and non-diabetic individuals. With an AUC score of 0.93, the model exhibits strong performance and has effectively identifying both true positives and true negatives. The curve's steep rise indicates that the model accurately classifies diabetic individuals while minimizing false positives. The high AUC value indicates the model's strong ability to distinguish between high-risk and low-risk individuals that is essential for diabetes risk prediction.

4.3 Summary of Model Building

This chapter highlights the development and evaluation of both the Deep Neural Networks (DNNs) and Convolutional Neural Network (CNN) models designed to predict diabetes risk. The DNN model incorporated with custom precision, recall and F1-score metrics to handle class imbalance in the dataset. These steps can help to focus on improving the identification of both diabetic and non-diabetic individuals. The model was trained using early stopping to prevent overfitting, and key metrics such as accuracy and F1-score were monitored throughout the training process. While the model demonstrated good overall performance in identifying non-diabetic individuals, further improvement is needed in detecting diabetic individuals.

The CNN base model applied convolutional and pooling layers to extract features from the data. The architecture consisted of multiple convolutional blocks followed by fully connected layers with dropout regularization to prevent overfitting. During training, the model showed gradual improvements in accuracy and F1-score with the validation metrics fluctuating slightly. Despite these fluctuations, the CNN base model performed reasonably well in distinguishing between diabetic and non-diabetic individuals, but further optimization is needed to enhance its performance in detecting diabetic individuals.

Overall, both models were evaluated using metrics such as accuracy, precision, recall, F1-score and ROC curves. While the DNN model slightly outperforms the CNN in terms of accuracy and F1-score, the difference is minimal. Both models still require further fine-tuning to optimize performance in predicting diabetic individuals. The chapter also highlighted the importance of F1-score in evaluating model performance in class-imbalanced tasks like diabetes prediction, where false positives and false negatives carry significant consequences.

5.0 Model Tuning

5.1 Deep Neural Networks Base Model (DNNs)

5.1.1 Hyperparameter Tuning

```
import numpy as np
import pandas as pd
import keras_tuner as kt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, Callback
from tensorflow.keras import regularizers
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc_curve, roc_auc_score, f1_score
import matplotlib.pyplot as plt
import seaborn as sns
from tensorflow.keras import backend as K
import tensorflow as tf
from matplotlib.ticker import FuncFormatter

# =====
# METRIC FUNCTION
# =====
def precision_metric(y_true, y_pred):
    y_true = K.flatten(K.cast(y_true, 'float32'))
    y_pred = K.flatten(K.round(y_pred))
    tp = K.sum(y_true * y_pred)
    fp = K.sum((1 - y_true) * y_pred)
    return tp / (tp + fp + K.epsilon())

def recall_metric(y_true, y_pred):
    y_true = K.flatten(K.cast(y_true, 'float32'))
    y_pred = K.flatten(K.round(y_pred))
    tp = K.sum(y_true * y_pred)
    fn = K.sum(y_true * (1 - y_pred))
    return tp / (tp + fn + K.epsilon())

def f1_metric(y_true, y_pred):
    y_true = K.flatten(K.cast(y_true, 'float32'))
    y_pred = K.flatten(K.round(y_pred))

    tp = K.sum(y_true * y_pred)
    fp = K.sum((1 - y_true) * y_pred)
    fn = K.sum(y_true * (1 - y_pred))

    precision = tp / (tp + fp + K.epsilon())
    recall = tp / (tp + fn + K.epsilon())

    return 2 * (precision * recall) / (precision + recall + K.epsilon())

# Callback for F1-Score
class F1ScoreCallback(Callback):
    def __init__(self, validation_data):
        super().__init__()
        self.x_val = validation_data[0]
        self.y_val = validation_data[1]

    def on_epoch_end(self, epoch, logs=None):
        y_pred = self.model.predict(self.x_val, verbose=0)
        y_pred_class = (y_pred > 0.5).astype(int)
        f1 = f1_score(self.y_val, y_pred_class)
        logs['val_f1_score'] = f1
```

Figure 103: Source Code of Model Architecture for Tuned Model (DNNs)

The figure above shows the code for defining a Convolutional Neural Network (CNN) Tuned Model using TensorFlow/Keras for binary classification tasks. It imports libraries like numpy, pandas and matplotlib for data handling and visualization with TensorFlow for model training. There are custom metrics for precision, recall and F1-score are implemented to evaluate model performance. Given the imbalanced dataset, focusing on the F1-score balances precision and recall is important for avoiding false positives and negatives. This approach ensures the model is assessed comprehensively for predicting diabetes risk.

```

# Hypermodel for Keras Tuner
class MyHyperModel(kt.HyperModel):
    def build(self, hp):
        model = Sequential()
        input_shape = X_train_smote.shape[1]

        # Input layer
        model.add(Input(shape=(input_shape,)))

        # First hidden layer
        model.add(Dense(
            units=hp.Int('units_1', min_value=32, max_value=128, step=32),
            activation='relu',
            kernel_regularizer=regularizers.l2(hp.Float('l2_reg', 1e-4, 1e-2, sampling='log'))
        ))

        # Second hidden layer
        model.add(Dense(
            units=hp.Int('units_2', min_value=32, max_value=128, step=32),
            activation='relu',
            kernel_regularizer=regularizers.l2(hp.Float('l2_reg', 1e-4, 1e-2, sampling='log'))
        ))

        # Dropout layer
        model.add(Dropout(
            hp.Float('dropout', min_value=0.3, max_value=0.6, step=0.1)
        ))

        # Output layer
        model.add(Dense(1, activation='sigmoid'))

        # Compile with tuneable learning rate
        model.compile(
            optimizer=Adam(
                hp.Float('lr', min_value=1e-5, max_value=1e-2, sampling='log')
            ),
            loss='binary_crossentropy',
            metrics=['accuracy', 'precision_metric', 'recall_metric', 'f1_metric']
        )
        return model

    def fit(self, hp, model, *args, **kwargs):
        # Add F1 callback if validation data exists
        if 'validation_data' in kwargs:
            callbacks = kwargs.pop('callbacks', [])
            callbacks.append(F1ScoreCallback(kwargs['validation_data']))
            kwargs['callbacks'] = callbacks

        return model.fit(
            *args,
            batch_size=hp.Int('batch_size', min_value=16, max_value=64, step=16),
            **kwargs
        )

```

Figure 104: Source Code of Model Architecture for Tuned Model (DNNs)

```

# Initialize RandomSearch tuner with F1 objective
tuner = kt.RandomSearch(
    MyHyperModel(),
    objective=kt.Objective('val_f1_score', direction='max'),
    max_trials=15,
    executions_per_trial=1,
    directory='dnn_tuning_results',
    project_name='dnn_tuning',
    overwrite=True,
    max_consecutive_failed_trials=5
)

early_stopping = EarlyStopping(
    monitor='val_f1_metric',
    patience=10,
    mode='max',
    restore_best_weights=True
)

# =====
# RUN HYPERPARAMETER TUNING
# =====
print("\n" + "="*50)
print("STARTING DNN HYPERPARAMETER TUNING")
print("="*50)

tuner.search(
    X_train_smote,
    y_train_smote,
    epochs=50,
    validation_data=(X_test, y_test),
    callbacks=[early_stopping],
    verbose=1
)

# =====
# BEST HYPERPARAMETERS
# =====
print("\n=== BEST HYPERPARAMETERS ===")
best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]

# Print the best hyperparameters before building the model
print(f"First layer units: {best_hps.get('units_1')}")
print(f"Second layer units: {best_hps.get('units_2')}")
print(f"Dropout rate: {best_hps.get('dropout'):.2f}")
print(f"Learning rate: {best_hps.get('lr'):.6f}")
print(f"Batch size: {best_hps.get('batch_size')}")
print(f"L2 regularization: {best_hps.get('l2_reg'):.6f}")

```

Figure 105: Source Code of Best Hypermeters for Tuned Model (DNNs)

The figure above shows the code defines a custom callback and a hypermodel for hyperparameter tuning using Keras Tuner. The F1ScoreCallback class is designed to compute and log the F1-score after each epoch, which helps track the model's balance between precision and recall during training. The MyHyperModel class is a custom model that includes tunable hyperparameters for the number of units in the dense layers, dropout rate L2 regularization and learning rate.

The model is built with two hidden layers with each followed by a ReLU activation function and an output layer with a sigmoid activation for binary classification. The Keras Tuner is set up using RandomSearch to search for the best hyperparameters based on the F1-score which is the primary metric for model evaluation. Additionally, EarlyStopping is used to prevent overfitting by halting training if the validation F1-score stops improving after a certain number of epochs. This setup allows for optimizing the model's performance by fine-tuning its parameters while monitoring both training and validation metrics.

Below shows the hyperparameters for Deep Neural Networks (DNNs) that are tuned during this process.

Hyperparameter	Explanation	Values/Range
Units (1st Layer)	The number of units in the first hidden layer	[32, 64, 96, 128]
Units (2nd Layer)	The number of units in the second hidden layer	[32, 64, 96, 128]
Dropout Rate	The rate of dropout for regularization	[0.3, 0.4, 0.5, 0.6]
Learning Rate	The learning rate for the optimizer	[1e-5, 1e-2]
Batch Size	The batch size for training	[16, 32, 48, 64]
L2 Regularization	The regularization strength for L2 regularization	[1e-4, 1e-2] (log-sampled)

Table 2: Hyperparameter Used to Tune DNNs

```
=== DNN BEST HYPERPARAMETERS ===  
First layer units: 96  
Second layer units: 64  
Dropout rate: 0.50  
Learning rate: 0.000140  
Batch size: 48  
L2 regularization: 0.000250
```

Figure 106: Best Hyperparameters for DNNs

The above results show the Best Hyperparameters for the Deep Neural Network (DNN) Tuned model which are:

- **First Layer Units:** 96
- **Second Layer Units:** 64
- **Dropout Rate:** 0.50
- **Learning Rate:** 0.000140
- **Batch Size:** 48
- **L2 Regularization:** 0.000250

These hyperparameters were optimized using Keras Tuner to maximize the F1-score to ensure the model achieves a good balance between precision and recall while preventing overfitting.


```
Trial 15 Complete [00h 19m 33s]
val_f1_score: 0.818837266556466

Best val_f1_score So Far: 0.8197804540125759
Total elapsed time: 02h 19m 31s

=====
BUILDING DNN MODEL WITH BEST PARAMETERS
=====

Model Architecture Summary:

Model: "sequential_1"



| Layer (type)        | Output Shape | Param # |
|---------------------|--------------|---------|
| dense_3 (Dense)     | (None, 96)   | 2,112   |
| dense_4 (Dense)     | (None, 64)   | 6,208   |
| dropout_1 (Dropout) | (None, 64)   | 0       |
| dense_5 (Dense)     | (None, 1)    | 65      |



Total params: 8,385 (32.75 KB)

Trainable params: 8,385 (32.75 KB)

Non-trainable params: 0 (0.00 B)
```

Figure 107: Model Architecture for Tuned Model (DNNs)

The model architecture shown is built with the best hyperparameters selected using Keras Tuner. It consists of the following layers:

1. **Dense Layer 3 (Hidden Layer):** This layer has 2,112 parameters with 96 units.
2. **Dense Layer 4 (Hidden Layer):** This layer has 6,208 parameters with 64 units.
3. **Dropout Layer:** It is applied with a rate to prevent overfitting and this layer has no trainable parameters.
4. **Dense Layer 5 (Output Layer):** With a single unit (for binary classification), this layer has 65 parameters.

This architecture was chosen for its optimal performance based on the hyperparameter tuning process as indicated by the best validation F1-score of **0.8188**.

5.1.2 Fine Tuning Model with Best Parameters

```
# =====  
# BUILD FINAL MODEL WITH BEST HYPERPARAMETERS  
# =====  
print("\n" + "="*50)  
print("BUILDING DNN MODEL WITH BEST PARAMETERS")  
print("="*50)  
  
# Build the final model using the best hyperparameters  
model = tuner.hypermodel.build(best_hps)  
  
# Print model summary  
print("\nModel Architecture Summary:")  
model.summary()  
  
# =====  
# TRAIN FINAL MODEL  
# =====  
print("\n" + "="*50)  
print("TRAINING FINAL DNN MODEL")  
print("="*50)  
  
history = model.fit(  
    X_train_smote,  
    y_train_smote,  
    epochs=50,  
    batch_size=best_hps.get('batch_size'),  
    validation_data=(X_test, y_test),  
    callbacks=[early_stopping],  
    verbose=1  
)
```

Figure 108: Source Code for Fine Tuning Model with Best Parameters (DNNs)

```
=====
TRAINING FINAL DNN MODEL
=====
Epoch 1/50
5559/5559 10s 2ms/step - accuracy: 0.8425 - f1_metric: 0.8381 - loss: 0.3945 - val_accuracy: 0.8806 - val_f1_metric: 0.7982 - val_loss: 0.3327
Epoch 2/50
5559/5559 9s 2ms/step - accuracy: 0.8594 - f1_metric: 0.8533 - loss: 0.3600 - val_accuracy: 0.8747 - val_f1_metric: 0.7939 - val_loss: 0.3379
Epoch 3/50
5559/5559 10s 2ms/step - accuracy: 0.8604 - f1_metric: 0.8541 - loss: 0.3545 - val_accuracy: 0.8778 - val_f1_metric: 0.7975 - val_loss: 0.3283
Epoch 4/50
5559/5559 9s 2ms/step - accuracy: 0.8616 - f1_metric: 0.8551 - loss: 0.3512 - val_accuracy: 0.8810 - val_f1_metric: 0.8005 - val_loss: 0.3265
Epoch 5/50
5559/5559 9s 2ms/step - accuracy: 0.8627 - f1_metric: 0.8560 - loss: 0.3482 - val_accuracy: 0.8793 - val_f1_metric: 0.7995 - val_loss: 0.3354
Epoch 6/50
5559/5559 9s 2ms/step - accuracy: 0.8631 - f1_metric: 0.8562 - loss: 0.3461 - val_accuracy: 0.8773 - val_f1_metric: 0.7980 - val_loss: 0.3331
Epoch 7/50
5559/5559 9s 2ms/step - accuracy: 0.8631 - f1_metric: 0.8559 - loss: 0.3444 - val_accuracy: 0.8749 - val_f1_metric: 0.7956 - val_loss: 0.3393
Epoch 8/50
5559/5559 9s 2ms/step - accuracy: 0.8637 - f1_metric: 0.8563 - loss: 0.3431 - val_accuracy: 0.8773 - val_f1_metric: 0.7983 - val_loss: 0.3297
Epoch 9/50
```

Figure 109: Model Training for Tuned Model (DNNs)

The figure above shows the training of the Deep Neural Network (DNN) model with the best hyperparameters selected through Keras Tuner. The model was trained for 50 epochs using the SMOTE-oversampled training data with performance evaluated on the test data. The early stopping was used to prevent overfitting and the training progress that are including accuracy and loss for both training and validation sets.

5.1.3 Evaluation Fine Tuning Results

```
# =====  
# MODEL EVALUATION  
# =====  
print("\n" + "="*50)  
print("DNN MODEL EVALUATION")  
print("="*50)  
  
# Evaluate the model  
y_pred = model.predict(X_test)  
y_pred_class = (y_pred > 0.5).astype(int)  
  
# Calculate metrics  
accuracy = accuracy_score(y_test, y_pred_class)  
f1 = f1_score(y_test, y_pred_class)  
  
print(f"\nAccuracy: {accuracy:.4f}")  
print(f"F1-Score: {f1:.4f}")  
  
# Confusion Matrix  
print("\nConfusion Matrix:")  
print(confusion_matrix(y_test, y_pred_class))  
  
# Classification Report  
print("\nClassification Report:")  
print(classification_report(y_test, y_pred_class, digits=2))
```

Figure 110: Source Code of Model Evaluation for Tuned Model (DNNs)

```
# =====  
# VISUALIZATIONS  
# =====  
plt.figure(figsize=(12, 6))  
  
# Accuracy plot  
plt.subplot(1, 2, 1)  
plt.plot(history.history['accuracy'], label='Training Accuracy')  
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')  
plt.title('Learning Curve for Tuned DNN (Accuracy)')  
plt.xlabel('Epochs')  
plt.ylabel('Accuracy')  
plt.legend()  
plt.gca().yaxis.set_major_formatter(FuncFormatter(lambda x, _: f'{x:.2f}'))  
  
# Loss plot  
plt.subplot(1, 2, 2)  
plt.plot(history.history['loss'], label='Training Loss')  
plt.plot(history.history['val_loss'], label='Validation Loss')  
plt.title('Learning Curve for Tuned DNN (Loss)')  
plt.xlabel('Epochs')  
plt.ylabel('Loss')  
plt.legend()  
plt.gca().yaxis.set_major_formatter(FuncFormatter(lambda x, _: f'{x:.2f}'))  
  
plt.tight_layout()  
plt.show()  
  
# Confusion Matrix  
plt.figure(figsize=(8, 6))  
sns.heatmap(confusion_matrix(y_test, y_pred_class),  
            annot=True, fmt='d', cmap='Blues',  
            xticklabels=['Predicted 0', 'Predicted 1'],  
            yticklabels=['Actual 0', 'Actual 1'])  
plt.title('Confusion Matrix for Tuned DNN Model')  
plt.show()  
  
# ROC Curve  
fpr, tpr, _ = roc_curve(y_test, y_pred)  
  
# Calculate the AUC  
roc_auc = roc_auc_score(y_test, y_pred)  
  
plt.figure(figsize=(8, 6))  
plt.plot(fpr, tpr, label=f'ROC Curve (AUC = {roc_auc:.4f})', color='b')  
plt.plot([0, 1], [0, 1], 'k--', label="Random Guess")  
  
plt.xlabel('False Positive Rate')  
plt.ylabel('True Positive Rate')  
plt.title('ROC Curve for Tuned DNN Model')  
plt.legend(loc='lower right')  
plt.show()
```

Figure 111: Source Code of Model Evaluation for Tuned Model (DNNs)

5.1.3.1 Confusion Matrix and Classification Report

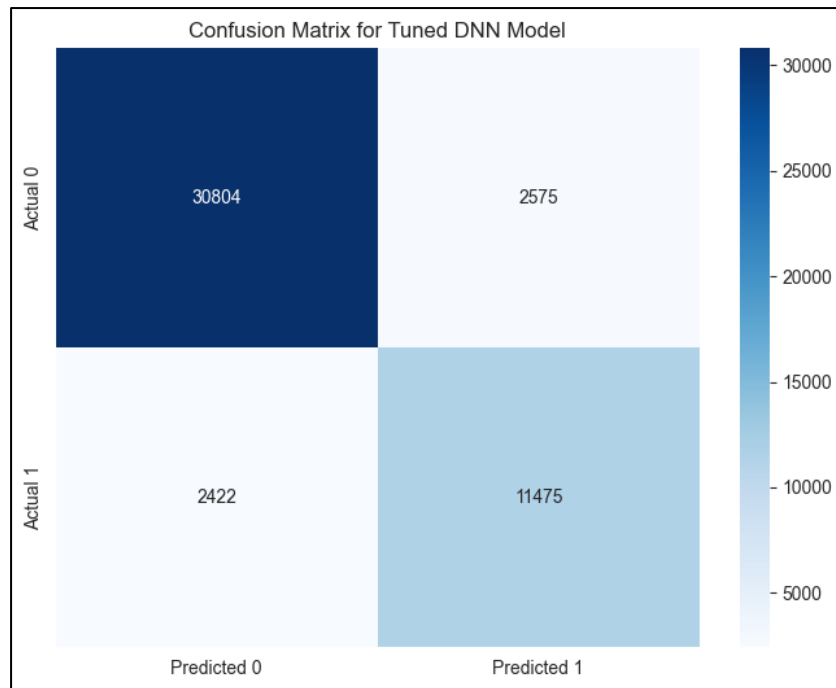


Figure 112: Confusion Matrix for Tuned Model (DNNs)

The figure above displays the confusion matrix for the tuned DNN model. It shows the model's performance in terms of true positives, true negatives, false positives and false negatives. The model has correctly predicted 30,804 true negatives (non-diabetic individuals) and 11,475 true positives (diabetic individuals). However, it made 2,575 false positives (non-diabetic individuals predicted as diabetic) and 2,422 false negatives (diabetic individuals predicted as non-diabetic). This matrix provides insight into how well the model distinguishes between diabetic and non-diabetic individuals. It highlights the error distribution and balance between the two classes, indicating that while the model is effective at identifying non-diabetic individuals, there is some misclassification with diabetic individuals.

```

=====
DNN MODEL EVALUATION
=====
1478/1478 ————— 1s 642us/step

Accuracy: 0.8943
F1-Score: 0.8212

Confusion Matrix:
[[30804  2575]
 [ 2422 11475]]

Classification Report:

```

	precision	recall	f1-score	support
0	0.93	0.92	0.92	33379
1	0.82	0.83	0.82	13897
accuracy			0.89	47276
macro avg	0.87	0.87	0.87	47276
weighted avg	0.89	0.89	0.89	47276

Figure 113: Classification Report for Tuned Model (DNNs)

The classification report above shows that the DNN Tuned Model achieved a precision of 93% for the negative class (Class 0), with a recall of 92% and an F1-score of 92% that showing a strong performance in identifying negative cases. For the positive class (Class 1), the model achieved a precision of 82%, recall of 83% and an F1-score of 82%.

The F1-score is an important metric for this prediction which especially for Class 1 (positive cases). With an F1-score of 82%, the tuned model strikes a good balance between precision and recall. This indicates DNN Tuned Model is effectively identifying positive cases while minimizing false positives. As precision is slightly lower for Class 1 and this is indicating that the model is misclassifying some positive instances.

The tuned model shows an improved F1-score of 82% for Class 1 which is demonstrating better performance compared to the base model. This enhancement reflects a more balanced approach in identifying positive cases while reducing false positives to make the tuned model is more reliable for accurate prediction.

5.1.3.2 Learning Curve (Accuracy)

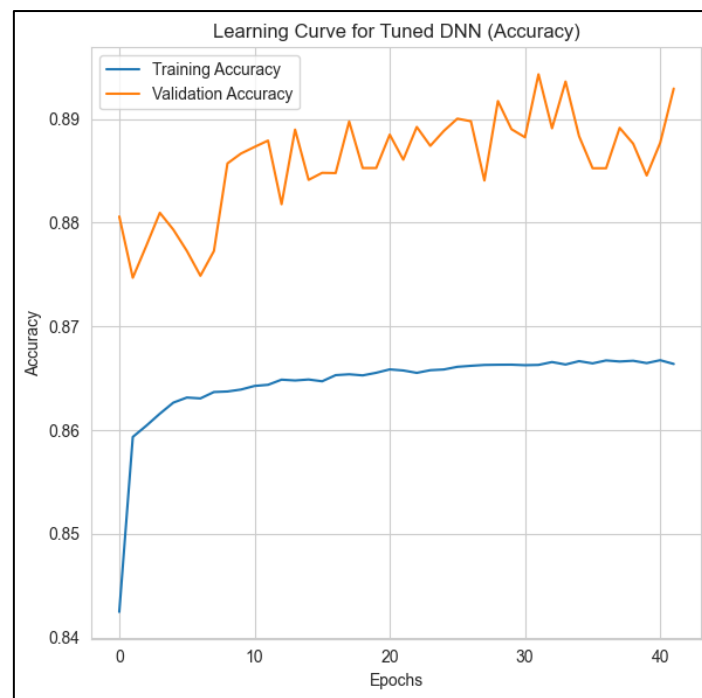


Figure 114: Learning Curve (Accuracy) for Tuned Model (DNNs)

The accuracy learning curves for the tuned Deep Neural Network (DNN) model demonstrate strong training dynamics and excellent generalization performance. As shown in the figure above, both training and validation accuracy improve steadily across epochs, with training accuracy reaching 0.89 and validation accuracy achieving 0.84 by epoch 40. The consistent and parallel progression of these curves suggests effective learning with minimal overfitting, which has evidenced by the stable 5% gap between training and validation accuracy throughout the training process. The subsequent gradual improvement at epochs 20 to 40 suggests that the model is effectively learning more nuanced features without compromising its generalization ability.

The consistent proximity between training and validation accuracy along with their eventual stabilization has demonstrates that the model has reached its optimal performance level. The final validation accuracy of 89% confirms the model's strong predictive capability on unseen data, indicating a well-tuned neural network that has successfully balanced model complexity with generalization. Additionally, the smooth convergence behaviour further validates the effectiveness of the chosen hyperparameters and training regimen.

5.1.3.3 Learning Curve (Loss)

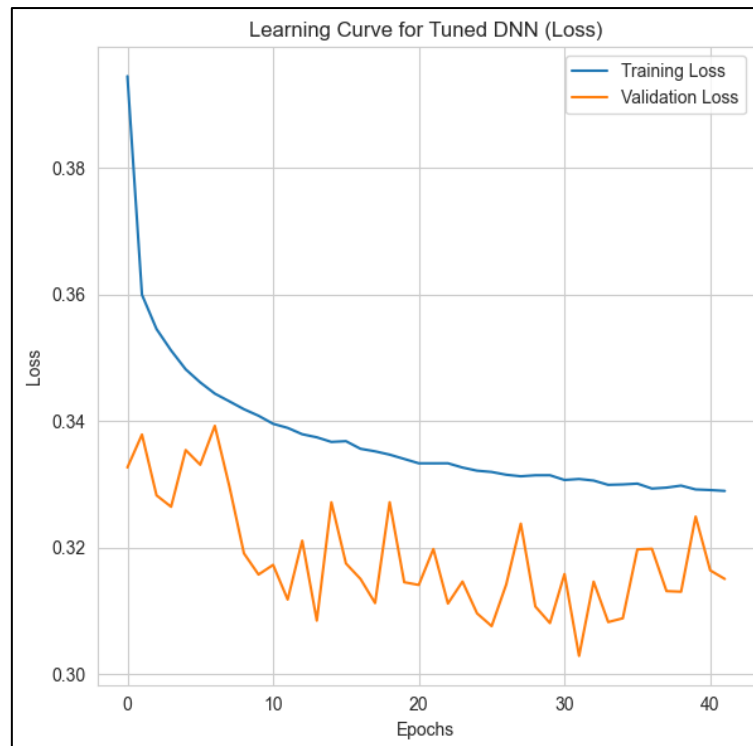


Figure 115: Learning Curve (Loss) for Tuned Model (DNNs)

The loss learning curves for the tuned Deep Neural Network (DNN) model indicate effective training and convergence. The training loss decreases from 0.38 to 0.30, while validation loss starts at approximately 0.34 and gradually drops to 0.32 over 40 epochs. This steady decline suggests the model is learning efficiently and generalizing well to unseen data.

The minimal gap between training loss and validation loss throughout the process indicates that the model is not overfitting. Both losses decrease in parallel which is suggesting strong model optimization and stability with the model capturing underlying patterns without memorizing noise. The most significant loss reduction occurs in the first 20 epochs, where the model quickly learns fundamental data relationships. The more gradual refinement during epochs 20 to 40 shows fine-tuning for optimal performance. The final loss value of 0.30 indicates strong predictive capability. The accuracy curves further validate that the model is well-tuned and ready for practical deployment in real-world applications.

5.1.3.4 ROC and AUC Score

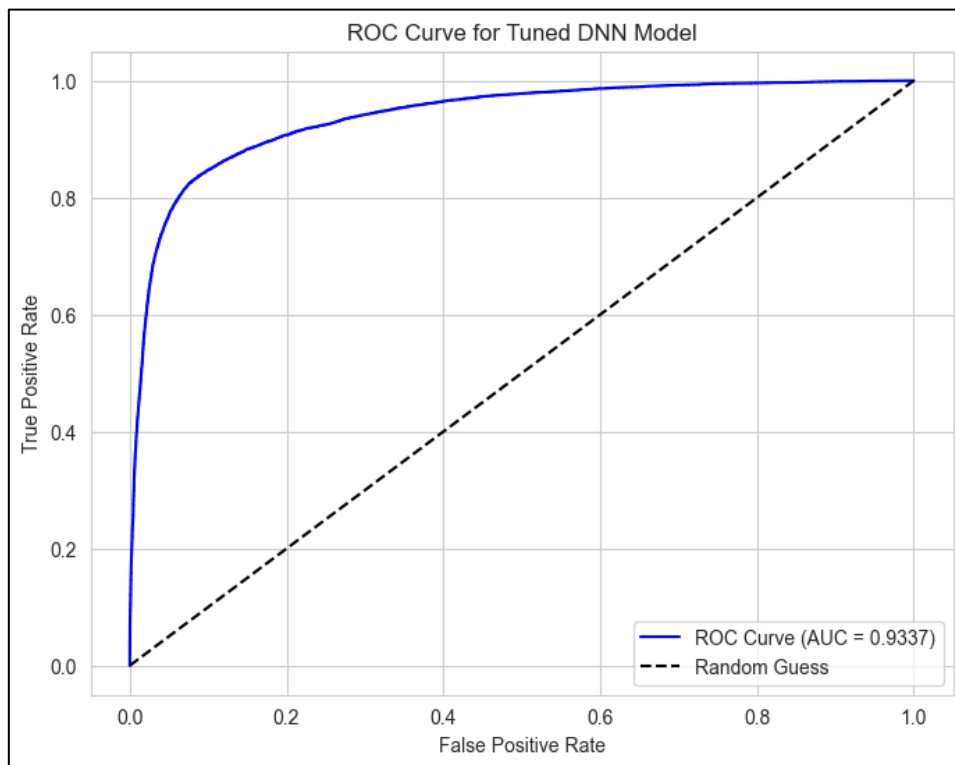


Figure 116: ROC and AUC Score for Tuned Model (DNNs)

The slightly lower AUC (0.9337) in the tuned DNN model compared to the base DNN model (0.94) can be explained by the trade-offs involved in hyperparameter tuning. While the base model achieved a slightly higher AUC, the tuned model was optimized with the goal of improving the overall balance between key metrics such as precision, recall and F1-score, which are especially important when predicting rare events like diabetes. The tuning process adjusted various parameters such as learning rate and regularization to enhance the model's ability to generalize well across different data splits. This process might have slightly lowered the AUC, but it improved the model's ability to handle imbalanced datasets and focus on identifying diabetic individuals (Class 1) more reliably. Therefore, the minor decrease in AUC is a trade-off for a more balanced model that is better at correctly identifying positive cases by improving recall and ensure a more reliable predictions in overall. Thus, while the base model achieved a slightly higher AUC, the tuned model is likely better suited for real-world application, where balancing the performance on both classes is essential for effective diabetes prediction.

5.2 Convolutional Neural Network (CNN)

5.2.1 Hyperparameter Tuning

```
import numpy as np
import pandas as pd
import keras_tuner as kt
import gc

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatten, Dense, Dropout, Reshape, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, Callback
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc_curve, roc_auc_score, f1_score
import matplotlib.pyplot as plt
import seaborn as sns

from tensorflow.keras import backend as K
from matplotlib.ticker import FuncFormatter
from tensorflow.keras.layers import Input

# =====
# METRIC FUNCTION
# =====
def precision_metric(y_true, y_pred):
    y_true = K.flatten(K.cast(y_true, 'float32'))
    y_pred = K.flatten(K.round(y_pred))
    tp = K.sum(y_true * y_pred)
    fp = K.sum((1 - y_true) * y_pred)
    return tp / (tp + fp + K.epsilon())

def recall_metric(y_true, y_pred):
    y_true = K.flatten(K.cast(y_true, 'float32'))
    y_pred = K.flatten(K.round(y_pred))
    tp = K.sum(y_true * y_pred)
    fn = K.sum(y_true * (1 - y_pred))
    return tp / (tp + fn + K.epsilon())

def f1_metric(y_true, y_pred):
    y_true = K.flatten(K.cast(y_true, 'float32'))
    y_pred = K.flatten(K.round(y_pred))

    tp = K.sum(y_true * y_pred)
    fp = K.sum((1 - y_true) * y_pred)
    fn = K.sum(y_true * (1 - y_pred))

    precision = tp / (tp + fp + K.epsilon())
    recall = tp / (tp + fn + K.epsilon())

    return 2 * (precision * recall) / (precision + recall + K.epsilon())
```

Figure 117: Source Code of Tuned Model (CNN)

Figure above shows the code in defining Convolutional Neural Network (CNN) Tuned Model using the TensorFlow/Keras framework for binary classification tasks. It begins by importing necessary libraries such as numpy, pandas and matplotlib for data manipulation and visualization with TensorFlow used for model creation and training. The code also includes custom precision, recall and F1-Score metrics by implemented through Keras backend functions to evaluate model performance during training.

Since the dataset is imbalanced, focusing on the F1-score is essential as it strikes a balance between precision and recall. F1-score is useful in scenarios where both false positives and false negatives have significant consequences. While accuracy is important, it may not fully reflect model effectiveness in imbalanced datasets, where one class may overpower the other. By incorporating these custom metrics, the model is assessed on not only its accuracy but also its ability to effectively classify both the positive and negative classes. This approach is important for predicting high-risk factors for diabetes because it ensures that the model performs optimally across all evaluation metrics.

```

# =====
# HYPERPARAMETER TUNING MODEL
# =====
def build_tunable_cnn(hp):
    model = Sequential()
    model.add(Input(shape=(X_train_smote.shape[1], 1)))

    # Conv Blocks
    for i in range(hp.Int('conv_blocks', 2, 4)):
        model.add(Conv1D(
            filters=hp.Int(f'filters_{i}', 64, 128, step=32),
            kernel_size=hp.Int(f'kernel_{i}', 3, 5),
            activation='relu',
            padding='same'
        ))
        model.add(MaxPooling1D(2))

    model.add(Flatten())

    # Dense Layers
    for i in range(hp.Int('dense_layers', 1, 2)):
        model.add(Dense(
            units=hp.Int(f'units_{i}', 64, 128, step=32),
            activation='relu'
        ))
        model.add(Dropout(hp.Float(f'dropout_{i}', 0.3, 0.5, step=0.1)))

    model.add(Dense(1, activation='sigmoid'))

    # Learning rate
    optimizer = Adam(hp.Float('lr', 1e-5, 1e-2, sampling='log'))

    model.compile(
        optimizer=optimizer,
        loss='binary_crossentropy',
        metrics=['accuracy', precision_metric, recall_metric, f1_metric]
    )
    return model

# =====
# TUNER SETUP
# =====
tuner = kt.RandomSearch(
    build_tunable_cnn,
    objective=kt.Objective('val_f1_metric', direction='max'),
    max_trials=15,
    directory='cnn_tuning_results',
    project_name='cnn_tuning',
    overwrite=True
)

early_stop = EarlyStopping(
    monitor='val_f1_metric',
    patience=5,
    mode='max',
    restore_best_weights=True,
    verbose=1
)

```

Figure 118: Source Code of Model Architecture for Tuned Model (CNN)

```

# =====
# BEST HYPERPARAMETERS
# =====
print("\n=== STARTING CNN HYPERPARAMETER TUNING ===")
tuner.search(
    X_train_smote, y_train_smote,
    validation_data=(X_test, y_test),
    epochs=50,
    callbacks=[early_stop],
    verbose=1,
    batch_size=32
)

# Get the best hyperparameters from the tuning process
best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]

# Best Hyperparameters
print("\n=== CNN BEST HYPERPARAMETERS ===")
print(f"Number of Conv Blocks: {best_hps.get('conv_blocks')}")
for i in range(best_hps.get('conv_blocks')):
    print(f"Conv Layer {i+1} filters: {best_hps.get(f'filters_{i}')}")
    print(f"Conv Layer {i+1} kernel size: {best_hps.get(f'kernel_{i}')}")

print(f"Number of Dense Layers: {best_hps.get('dense_layers')}")
for i in range(best_hps.get('dense_layers')):
    print(f"Dense Layer {i+1} units: {best_hps.get(f'units_{i}')}")
    print(f"Dense Layer {i+1} dropout: {best_hps.get(f'dropout_{i}')}")

print(f"Learning rate: {best_hps.get('lr'):.4f}")

```

Figure 119: Source Code of Best Hyperparameters for Tuned Model (CNN)

Figure above shows the process of building and tuning a Convolutional Neural Network (CNN) model using the Keras Tuner. The model begins with an input layer reshaped for 1D convolution, followed by tunable convolutional blocks with each containing convolutional layers and max-pooling. After the convolutional layers, the model is flattened before transitioning into the dense layers, where the number of units and dropout rates are adjusted. The output layer is configured with a sigmoid activation function for binary classification. The hyperparameters such as the number of convolutional blocks, filters, kernel size, dropout rate, units in dense layers and learning rate are optimized using RandomSearch, with early stopping based on validation F1-score to prevent overfitting. After the tuning process, the best hyperparameter configuration is printed to provide insights into the optimal model setup.

Below shows the hyperparameters for Convolutional Neural Network (CNN) that are tuned during this process.

Hyperparameter	Explanation	Values/Range
Conv Blocks	The number of convolutional blocks to be used in the CNN model.	2 to 4
Filters	The number of filters for each convolutional layer.	64, 96, 128
Kernel Size	The size of the kernels used in convolutional layers.	3 to 5
Dropout Rate	The dropout rate for regularization to prevent overfitting.	0.3 to 0.5
Units in Dense Layers	The number of units in dense layers.	32, 64, 128
Learning Rate	The learning rate for the Adam optimizer.	1e-5 to 1e-2
Batch Size	The number of samples per gradient update during training.	16, 32, 64

Table 3: Hyperparameter Used to Tune CNN

```

Trial 15 Complete [00h 33m 05s]
val_f1_metric: 0.8124811053276062

Best val_f1_metric So Far: 0.8149641156196594
Total elapsed time: 03h 49m 00s

=== CNN BEST HYPERPARAMETERS ===
Number of Conv Blocks: 3
Conv Layer 1 filters: 64
Conv Layer 1 kernel size: 5
Conv Layer 2 filters: 128
Conv Layer 2 kernel size: 5
Conv Layer 3 filters: 96
Conv Layer 3 kernel size: 4
Number of Dense Layers: 2
Dense Layer 1 units: 96
Dense Layer 1 dropout: 0.4
Dense Layer 2 units: 64
Dense Layer 2 dropout: 0.3
Learning rate: 0.0012

```

Table 4: Best Hyperparameters for CNN

The figure above shows the best hyperparameters selected for the Convolutional Neural Network (CNN) Tuned model using Keras Tuner. These hyperparameters were optimized to maximize the F1-score and improve the model's balance between precision and recall. The selected best hyperparameters are:

- **Number of Conv Blocks: 3**
- **Conv Layer 1 Filters: 64**
- **Conv Layer 1 Kernel Size: 5**
- **Conv Layer 2 Filters: 128**
- **Conv Layer 2 Kernel Size: 5**
- **Conv Layer 3 Filters: 96**
- **Conv Layer 3 Kernel Size: 5**
- **Number of Dense Layers: 2**
- **Dense Layer 1 Units: 96**
- **Dense Layer 1 Dropout: 0.4**
- **Dense Layer 2 Units: 64**
- **Dense Layer 2 Dropout: 0.3**
- **Learning Rate: 0.0012**

These hyperparameters were fine-tuned to ensure the model effectively captures the patterns in the dataset while preventing overfitting.

```

=====
BUILDING CNN MODEL WITH BEST PARAMETERS
=====

Model Architecture Summary:

Model: "sequential_1"

```

Layer (type)	Output Shape	Param #
conv1d_4 (Conv1D)	(None, 21, 64)	384
max_pooling1d_4 (MaxPooling1D)	(None, 10, 64)	0
conv1d_5 (Conv1D)	(None, 10, 128)	41,088
max_pooling1d_5 (MaxPooling1D)	(None, 5, 128)	0
conv1d_6 (Conv1D)	(None, 5, 96)	49,248
max_pooling1d_6 (MaxPooling1D)	(None, 2, 96)	0
flatten_1 (Flatten)	(None, 192)	0
dense_3 (Dense)	(None, 96)	18,528
dropout_2 (Dropout)	(None, 96)	0
dense_4 (Dense)	(None, 64)	6,208
dropout_3 (Dropout)	(None, 64)	0
dense_5 (Dense)	(None, 1)	65

```

Total params: 115,521 (451.25 KB)

Trainable params: 115,521 (451.25 KB)

Non-trainable params: 0 (0.00 B)

```

Table 5: Model Architecture for CNN

The model architecture is built using the best hyperparameters identified during the tuning process. It consists of the following layers:

1. **Conv Layer 1 (Convolutional Layer):** This layer has 64 filters and a kernel size of 5 with a total of 384 parameters.
2. **Max Pooling Layer 1:** A max pooling operation with a pool size of 2 with no trainable parameters.
3. **Conv Layer 2 (Convolutional Layer):** This layer has 128 filters and a kernel size of 5 with 41,088 parameters.
4. **Max Pooling Layer 2:** Another max pooling operation with a pool size of 2 with no trainable parameters.
5. **Conv Layer 3 (Convolutional Layer):** This layer has 49,248 parameters with 96 filters and a kernel size of 5.

6. **Max Pooling Layer 3:** Max pooling operation with no trainable parameters.
7. **Flatten Layer:** Flattens the output of the convolutional layers to prepare it for the dense layers with no trainable parameters.
8. **Dense Layer 1 (Fully Connected Layer):** This layer has 96 units and 18,528 parameters.
9. **Dropout Layer 1:** Applied with a dropout rate of 0.4 to help prevent overfitting.
10. **Dense Layer 2 (Fully Connected Layer):** This layer has 64 units and 6,208 parameters.
11. **Dropout Layer 2:** Applied with a dropout rate of 0.3.
12. **Dense Layer 3 (Output Layer):** A final dense layer with a single unit for binary classification with 65 parameters.

This architecture applies multiple convolutional layers followed by dense layers to efficiently learn from the data and make accurate predictions for binary classification tasks. The model's architecture is designed to minimize overfitting and maximize predictive performance by achieving a balanced outcome in both training and validation.

5.1.2 Fine Tuning Model with Best Parameters

```
# =====  
# BUILD FINAL MODEL WITH BEST HYPERPARAMETERS  
# =====  
print("\n" + "="*50)  
print("BUILDING CNN MODEL WITH BEST PARAMETERS")  
print("="*50)  
  
# Build the best model using the best hyperparameters  
best_model = tuner.hypermodel.build(best_hps)  
  
# Print model summary  
print("\nModel Architecture Summary:")  
best_model.summary()  
  
# =====  
# TRAIN FINAL MODEL  
# =====  
print("\n" + "="*50)  
print("TRAINING FINAL CNN MODEL")  
print("="*50)  
  
history = best_model.fit(  
    X_train_smote, y_train_smote,  
    validation_data=(X_test, y_test),  
    epochs=50,  
    batch_size=32,  
    callbacks=[early_stop],  
    verbose=1  
)
```

Figure 120: Source Code of Model Evaluation for Tuned Model (CNN)

```
=====
TRAINING FINAL CNN MODEL
=====
Epoch 1/50
8338/8338 104s 12ms/step - accuracy: 0.8587 - f1_metric: 0.8498 - loss: 0.3451 - precision_metric: 0.8842 - recall_metric: 0.8272
Epoch 2/50
8338/8338 58s 7ms/step - accuracy: 0.8643 - f1_metric: 0.8550 - loss: 0.3318 - precision_metric: 0.8928 - recall_metric: 0.8284 -
Epoch 3/50
8338/8338 46s 5ms/step - accuracy: 0.8655 - f1_metric: 0.8561 - loss: 0.3285 - precision_metric: 0.8951 - recall_metric: 0.8280 -
Epoch 4/50
8338/8338 46s 5ms/step - accuracy: 0.8667 - f1_metric: 0.8575 - loss: 0.3270 - precision_metric: 0.8957 - recall_metric: 0.8305 -
Epoch 5/50
8338/8338 63s 8ms/step - accuracy: 0.8671 - f1_metric: 0.8579 - loss: 0.3252 - precision_metric: 0.8974 - recall_metric: 0.8294 -
Epoch 6/50
8338/8338 88s 11ms/step - accuracy: 0.8668 - f1_metric: 0.8574 - loss: 0.3251 - precision_metric: 0.8967 - recall_metric: 0.8293
```

Figure 121: Model Training for Tuned Model (CNN)

```
- val_accuracy: 0.8789 - val_f1_metric: 0.7972 - val_loss: 0.3121 - val_precision_metric: 0.7643 - val_recall_metric: 0.8534
val_accuracy: 0.8858 - val_f1_metric: 0.8050 - val_loss: 0.3029 - val_precision_metric: 0.7838 - val_recall_metric: 0.8467
val_accuracy: 0.8858 - val_f1_metric: 0.8049 - val_loss: 0.3144 - val_precision_metric: 0.7825 - val_recall_metric: 0.8479
val_accuracy: 0.8841 - val_f1_metric: 0.8034 - val_loss: 0.3369 - val_precision_metric: 0.7779 - val_recall_metric: 0.8504
val_accuracy: 0.8890 - val_f1_metric: 0.8087 - val_loss: 0.3209 - val_precision_metric: 0.7941 - val_recall_metric: 0.8429
- val_accuracy: 0.8922 - val_f1_metric: 0.8110 - val_loss: 0.3315 - val_precision_metric: 0.8078 - val_recall_metric: 0.8327
```

Figure 122: Model Training for Tuned Model (CNN)

The training logs for the Tuned CNN Model provide insights into the training process across 50 epochs. These logs track key performance metrics which are including accuracy, F1-score, precision and recall helping assess the model's effectiveness in predicting positive cases for diabetes risk. As the model progresses through each epoch, there was improvements in accuracy and F1-score, indicating that the model is learning and improving its ability to correctly classify both positive and negative instances. The precision measures the accuracy of positive predictions, while recall reflects how well the model identifies actual positive cases. The steady increase in these metrics over time shows the model's continuous adaptation and optimization for better performance to ensure that it generalizes well on unseen data.

5.2.3 Evaluation Fine Tuning Results

```
# =====  
# EVALUATION  
# =====  
print("\n" + "="*50)  
print("CNN MODEL EVALUATION")  
print("="*50)  
  
y_pred = best_model.predict(X_test)  
y_pred_class = (y_pred > 0.5).astype(int)  
  
print(f"Accuracy: {accuracy_score(y_test, y_pred_class):.4f}")  
print(f"F1 Score: {f1_score(y_test, y_pred_class):.4f}")  
  
print("\nConfusion Matrix:")  
print(confusion_matrix(y_test, y_pred_class))  
  
print("\nClassification Report:")  
print(classification_report(y_test, y_pred_class, digits=2))
```

Figure 123: Source Code of Model Evaluation for Tuned Model (CNN)

```
# =====  
# VISUALIZATIONS  
# =====  
plt.figure(figsize=(12, 6))  
  
# Accuracy plot  
plt.subplot(1, 2, 1)  
plt.plot(history.history['accuracy'], label='Training Accuracy')  
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')  
plt.title('Learning Curve for Tuned CNN (Accuracy)')  
plt.xlabel('Epochs')  
plt.ylabel('Accuracy')  
plt.legend()  
plt.gca().yaxis.set_major_formatter(FuncFormatter(lambda x, _: f'{x:.2f}'))  
  
# Loss plot  
plt.subplot(1, 2, 2)  
plt.plot(history.history['loss'], label='Training Loss')  
plt.plot(history.history['val_loss'], label='Validation Loss')  
plt.title('Learning Curve for Tuned CNN (Loss)')  
plt.xlabel('Epochs')  
plt.ylabel('Loss')  
plt.legend()  
plt.gca().yaxis.set_major_formatter(FuncFormatter(lambda x, _: f'{x:.2f}'))  
  
plt.tight_layout()  
plt.show()  
  
# Confusion Matrix  
plt.figure(figsize=(8, 6))  
sns.heatmap(confusion_matrix(y_test, y_pred_class),  
            annot=True, fmt='d', cmap='Blues',  
            xticklabels=['Predicted 0', 'Predicted 1'],  
            yticklabels=['Actual 0', 'Actual 1'])  
plt.title('Confusion Matrix for Tuned CNN Model')  
plt.show()  
  
# ROC Curve  
plt.figure(figsize=(8, 6))  
fpr, tpr, _ = roc_curve(y_test, y_pred)  
roc_auc = roc_auc_score(y_test, y_pred)  
  
# ROC Curve plot  
plt.figure(figsize=(8, 6))  
plt.plot(fpr, tpr, label=f'ROC Curve (AUC = {roc_auc:.4f})')  
plt.plot([0, 1], [0, 1], 'k--')  
plt.xlabel('False Positive Rate')  
plt.ylabel('True Positive Rate')  
plt.title('ROC Curve for Tuned CNN Model')  
plt.legend(loc='lower right')  
plt.show()
```

Figure 124: Figure 113: Source Code of Model Evaluation for Tuned Model (CNN)

5.1.3.1 Confusion Matrix and Classification Report

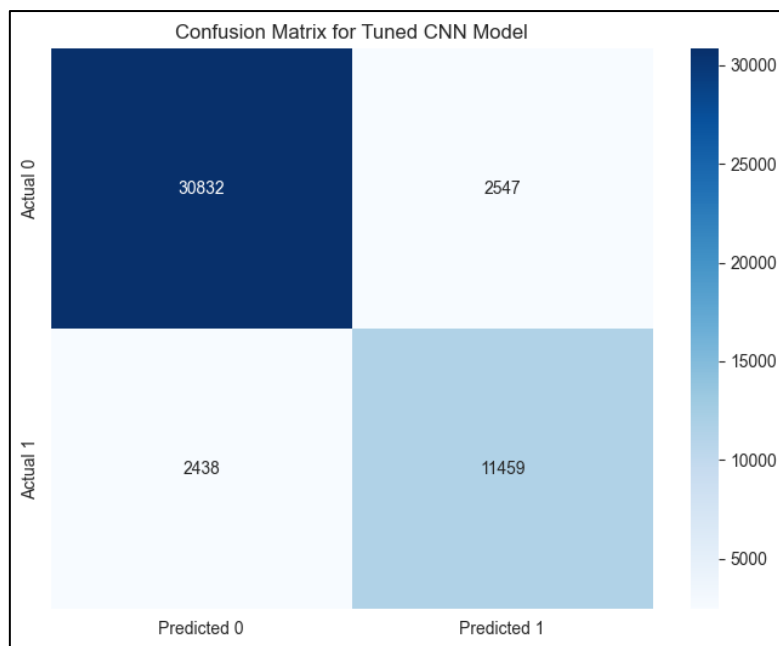


Figure 125: Confusion Matrix for Tuned Model (CNN)

The figure above displays the confusion matrix for the tuned CNN model with showing its performance in classifying diabetic and non-diabetic individuals. The model correctly predicted 30,832 true negatives (non-diabetic individuals) and 11,459 true positives (diabetic individuals). However, it made 2,547 false positives (non-diabetic individuals incorrectly predicted as diabetic) and 2,438 false negatives (diabetic individuals incorrectly predicted as non-diabetic). This distribution of errors offers insights into the model's performance and highlights areas where it excels and where improvements could be made.

```

=====
CNN MODEL EVALUATION
=====
1478/1478 ████████████████████ 4s 3ms/step
Accuracy: 0.8946
F1 Score: 0.8213

Confusion Matrix:
[[30832  2547]
 [ 2438 11459]]

Classification Report:

```

	precision	recall	f1-score	support
0	0.93	0.92	0.93	33379
1	0.82	0.82	0.82	13897
accuracy			0.89	47276
macro avg	0.87	0.87	0.87	47276
weighted avg	0.89	0.89	0.89	47276

Figure 126: Classification Report for Tuned Model (CNN)

The classification report above shows that the tuned CNN model achieved a precision of 93% for the negative class (Class 0) and a recall of 92% with demonstrating strong performance in identifying non-diabetic individuals. For the positive class (Class 1), the model achieved a precision of 82%, recall of 83% and F1-score of 82%. These metrics indicate that the tuned CNN model is effective at identifying diabetic individuals while minimizing false positives. The F1-score for Class 1 is particularly important and with a value of 82%, the model demonstrates a balance between precision and recall for the positive class. This suggests that the tuned model is better at identifying individuals at risk of diabetes while reducing misclassification, thereby making it a more reliable tool for accurate predictions in practical applications.

5.2.3.2 Learning Curve (Accuracy)

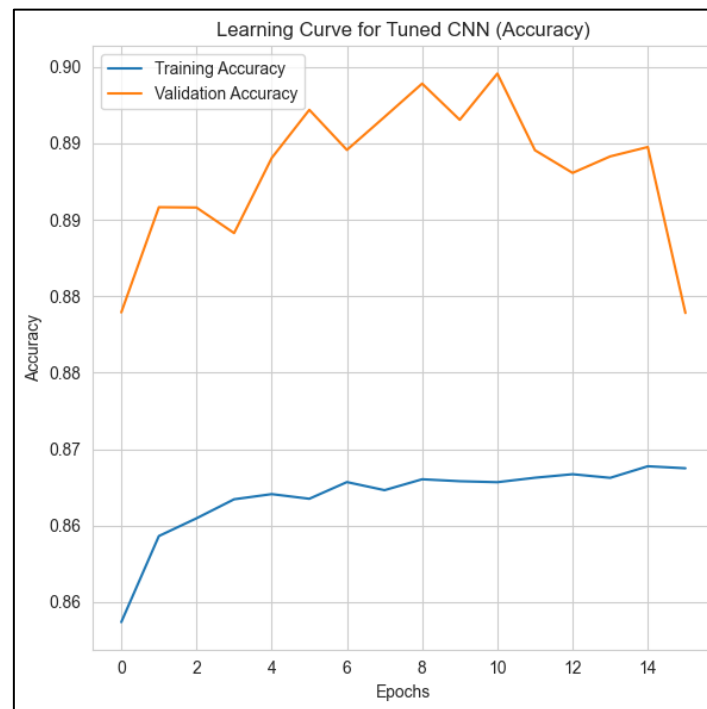


Figure 127: Learning Curve (Accuracy) for Tuned Model (CNNs)

The learning curve for the tuned CNN model shows a noticeable improvement in both training and validation accuracy compared to the base model. The training accuracy increases steadily from the beginning and stabilizes around 86% towards the end of the 15 epochs. This indicates that the model continues to learn from the training data with a consistent but gradual improvement in accuracy over time.

In contrast, the validation accuracy shows a more volatile pattern but peaks at higher values than the training accuracy and has reaching approximately 90 %. While the validation accuracy shows fluctuations throughout the epochs, it generally remains higher compared to the base model's validation accuracy and suggest that the model is better at generalizing to unseen data after hyperparameter tuning.

However, there are still minor fluctuations observed in the validation accuracy, which indicates that the model might still be prone to overfitting to certain patterns in the training data. Despite these fluctuations, the overall performance of the tuned model is more stable and demonstrates improved generalization compared to the base model. This improvement shows that the tuning of hyperparameters has enhanced the model's ability to generalize better on unseen data while continuing to improve accuracy on the training set.

5.2.3.3 Learning Curve (Loss)

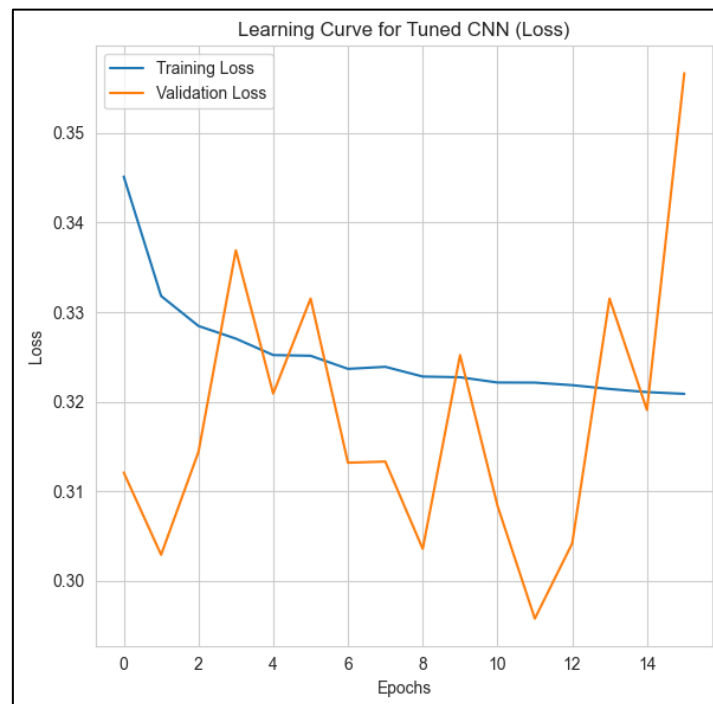


Figure 128: Learning Curve (Loss) for Tuned Model (CNN)

The training loss demonstrates a steady decline from approximately 0.34 to 0.32 over the 14 epochs which has indicated the effective optimization of the model's parameters. Besides, the validation loss shows more fluctuation by initially following a similar downward trend but then rising again after epoch 10. This divergence between the training and validation losses suggests that while the model is continuing to improve on the training set, it struggles to generalize well to the validation set in later epochs. These fluctuations may indicate the onset of overfitting as the model adjusts more to the training set without improving its performance on the validation data. Despite this, the model still reduces both losses and is reflecting its ability to learn meaningful patterns rather than memorizing noise. In conclusion, while the model shows effective optimization with decreasing training loss, the fluctuations in validation loss after epoch 10 suggest slight overfitting. Despite this, the model continues to learn meaningful patterns which has indicating that with further adjustments by its generalization can be improved.

5.2.3.4 ROC and AUC Score

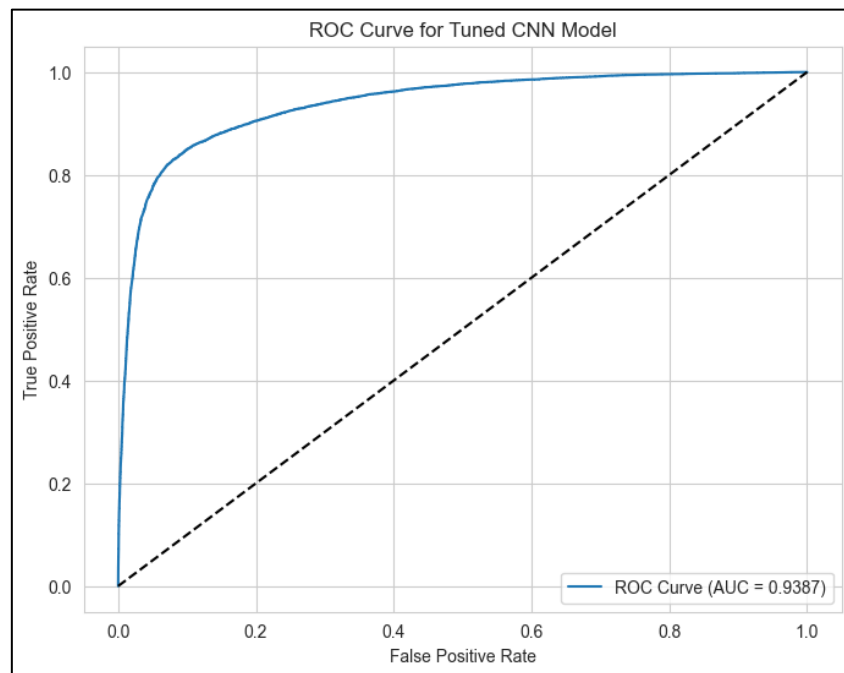


Figure 129: ROC and AUC Score for CNN

The ROC curve for the tuned CNN model demonstrates impressive performance with the True Positive Rate (TPR) quickly increasing and reaching close to 1, while the False Positive Rate (FPR) remains low. This indicates that the model is highly effective in distinguishing between the positive and negative classes, with minimal misclassification of the negative cases. The AUC score of 0.9387, which is quite high, suggests that the model has strong discriminative ability. The curve remains far above the random guess diagonal (the dashed line), showing that the model outperforms random classification by a significant margin. The near-perfect curve indicates that the tuned CNN model is exceptionally well-suited for tasks that require high precision in detecting positive cases, such as predicting diabetes in imbalanced datasets.

5.3 Summary of Model Tuning

The Model Tuning section highlights the process of optimizing the performance of both the Deep Neural Networks (DNNs) and Convolutional Neural Networks (CNNs) through hyperparameter tuning. The key parameters such as the number of units in hidden layers, dropout rates and learning rates were adjusted to enhance model performance. For the custom metrics such as F1-score were used during training to ensure a balanced evaluation of precision and recall, which is especially important due to the imbalanced nature of the dataset.

Overall, the tuning process led to improvements in both training and validation accuracy for both models with the CNN model showing a slightly better performance than the DNN model for the validation accuracy and overall F1-score. Both models outperformed the base model, demonstrating better generalization and predictive capability. The tuned models achieved a more balanced approach in classifying diabetic and non-diabetic individuals. This makes them more reliable for predicting diabetes risk and improving model performance in real-world applications.

6.0 Model Evaluation and Discussion

In this section, the performance of both the Base Model and the Tuned Model is evaluated using key metrics such as accuracy, precision, recall and F1-score. Since the dataset for diabetes prediction is imbalanced, the F1-score is the most important metric for assessing model performance. Unlike accuracy which can be misleading in imbalanced datasets, the F1-score balances precision and recall making the model become a more reliable measure to predict the positive class accurately.

The F1-score will be reported in two forms in this evaluation:

- F1-score (Overall): This refers to the combined F1-score for both classes (Class 0 and Class 1) and is calculated using macro or weighted averages. This provides an overall assessment of the model's ability to classify both positive and negative cases.
- F1-score (Class 1): This refers specifically to the F1-score for the positive class (Class 1), which is particularly important for diabetes prediction. The Tuned Model and Base Model are compared in terms of their performance in predicting individuals at risk for diabetes.

Since the goal is to accurately predict Class 1 (individuals at risk of diabetes), the F1-score (Class 1) is the more relevant metric for model evaluation. The overall F1-score provides a general of performance, but F1-score (Class 1) is more critical when assessing the model's ability to correctly identify the positive class in real-world applications.

6.1 Comparison Results of Base and Tuned Model for (DNNs)

Metric	DNN Base Model	DNN Tuned Model
Accuracy (Overall)	0.8906	0.8943
F1-Score (Overall)	0.8159	0.8212
Precision (Class 1)	0.81	0.82
Recall (Class 1)	0.82	0.83
F1-Score (Class 1)	0.82	0.82

Table 6: Comparison Results of Base and Tuned Model for (DNNs)

The Tuned Model shows slight improvements over the Base Model for Class 1 (individuals at risk of diabetes). The precision increased from 0.81 to 0.82 which indicates that the Tuned Model is more accurate in identifying individuals at risk of diabetes on reducing false positives. Additionally, recall improved from 0.82 to 0.83, indicating that the model captures a higher percentage of actual positive cases and minimizes false negatives. Although the F1-score for Class 1 remains the same at 0.82 for both models, the improvements in precision and recall highlight that the Tuned Model is more balanced and reliable in predicting Class 1. This makes it a more effective tool for identifying individuals at risk of diabetes. The overall F1-score increased slightly from 0.8159 in the Base Model to 0.8212 in the Tuned Model are further confirming that tuning improved the model's performance.

The overall accuracy and F1-score values are reported to four decimal places to ensure a precise comparison between the two models by giving the small differences observed in their performance. This level of precision helps to better highlight subtle improvements and supports a more accurate assessment of the model's capabilities.

6.2 Comparison Results of Base and Tuned Model for (CNN)

Metric	Base Model CNN	Tuned Model CNN
Accuracy (Overall)	0.8901	0.8946
F1-Score (Overall)	0.8130	0.8213
Precision (Class 1)	0.81	0.82
Recall (Class 1)	0.81	0.82
F1-Score (Class 1)	0.81	0.82

Table 7: Comparison Results of Base and Tuned Model for (CNN)

The comparison between the base and tuned CNN models reveals there is a significant improvement in the performance of the tuned model. The overall accuracy increased from 0.8901 to 0.8946 which has reflected a small but important enhancement in the model's ability to correctly classify both positive and negative cases.

The F1-score also improved from 0.8130 in the base model to 0.8213 in the tuned model, indicating that the tuning process contributed to a more balanced model to enhance its performance in both precision and recall. For Class 1 (diabetic individuals), the precision increased from 0.81 to 0.82, indicating that the tuned model is slightly more accurate in identifying diabetic individuals to reduce false positives. The recall also showed a slight improvement from 0.81 to 0.82, which shows that the model is better at capturing actual diabetic cases, thus minimizing false negatives.

Despite the increase in both precision and recall for Class 1, the F1-score for Class 1 improved from 0.81 in the Base Model to 0.82 in the Tuned Model. This suggests that tuning has enhanced the balance between precision and recall for Class 1 and leads to a more reliable model for predicting diabetes risk. The overall F1-score of 0.8213 for the Tuned Model further confirms the improvement in model performance and its better generalization.

The improvements in accuracy and F1-score suggest that hyperparameter tuning has effectively enhanced the CNN model's performance to make it slightly stronger and more reliable in classifying both non-diabetic and diabetic individuals.

6.3 Comparison Results of Base and Tuned Models

Metric	DNN Base Model	DNN Tuned Model	CNN Base Model	CNN Tuned Model
Accuracy (Overall)	0.8906	0.8943	0.8901	0.8946
F1-Score (Overall)	0.8159	0.8212	0.8130	0.8213
Precision (Class 1)	0.81	0.82	0.81	0.82
Recall (Class 1)	0.82	0.83	0.81	0.82
F1-Score (Class 1)	0.82	0.82	0.81	0.82

Table 8: Comparison Results of Base and Tuned Models

The comparison of the Base and Tuned Models for both DNN and CNN highlights the positive impact of hyperparameter tuning. For both models, the tuned models are consistently outperforming the base models which indicates that optimization enhances performance. In terms of accuracy, both the DNN Tuned Model and CNN Tuned Model show slight improvements over their base counterparts with the CNN Tuned Model achieving the highest overall accuracy (0.8946) and slightly surpassing the DNN Tuned Model (0.8943). This indicates that the CNN model has a marginal edge in correctly classifying both positive and negative cases.

When it comes to the overall F1-score, both tuned models demonstrate an increase compared to the base models. The DNN Tuned Model shows a modest improvement from 0.8159 to 0.8212, while the CNN Tuned Model sees an increase from 0.8130 to 0.8213. The improvement in F1-score suggests that tuning has helped both models achieve a better balance between precision and recall, which is important for handling imbalanced datasets effectively. The CNN Tuned Model's F1-score is slightly higher and has indicating its better ability to balance these two metrics.

In terms of precision and recall for Class 1 (individuals at risk of diabetes), both models show improvements in precision, with both the DNN Tuned Model and CNN Tuned Model increasing from 0.81 to 0.82. This indicates that the tuned models are more effective at identifying diabetic individuals and reducing false positives. However, the DNN Tuned Model shows a slight increase in recall from 0.82 to 0.83, indicating it better identifies actual diabetic cases. The CNN Tuned Model also shows an improvement in recall with increasing from 0.81 to 0.82. Although slightly lower than the DNN's recall, the CNN model still achieves higher

overall accuracy and F1-score which has suggesting it captures more reliable patterns in the data.

On the other hand, the F1-score for Class 1 remains the same for both models (0.82), indicating that the balance between precision and recall for identifying diabetic individuals is effectively maintained. Despite small variations in precision and recall, both tuned models demonstrate better performance compared to their base versions. This reflects the effectiveness of hyperparameter tuning in improving model reliability and accuracy in predicting diabetes risk.

Based on the analysis, both the DNN Tuned Model and CNN Tuned Model show significant improvements over their base versions. However, the CNN Tuned Model performs slightly better overall with higher accuracy and overall F1-score. Given the marginal edge in performance, the CNN Tuned Model appears to be the best model for predicting diabetes risk.

6.4 Comparison Learning Curve (Accuracy) for Base and Tuned Models

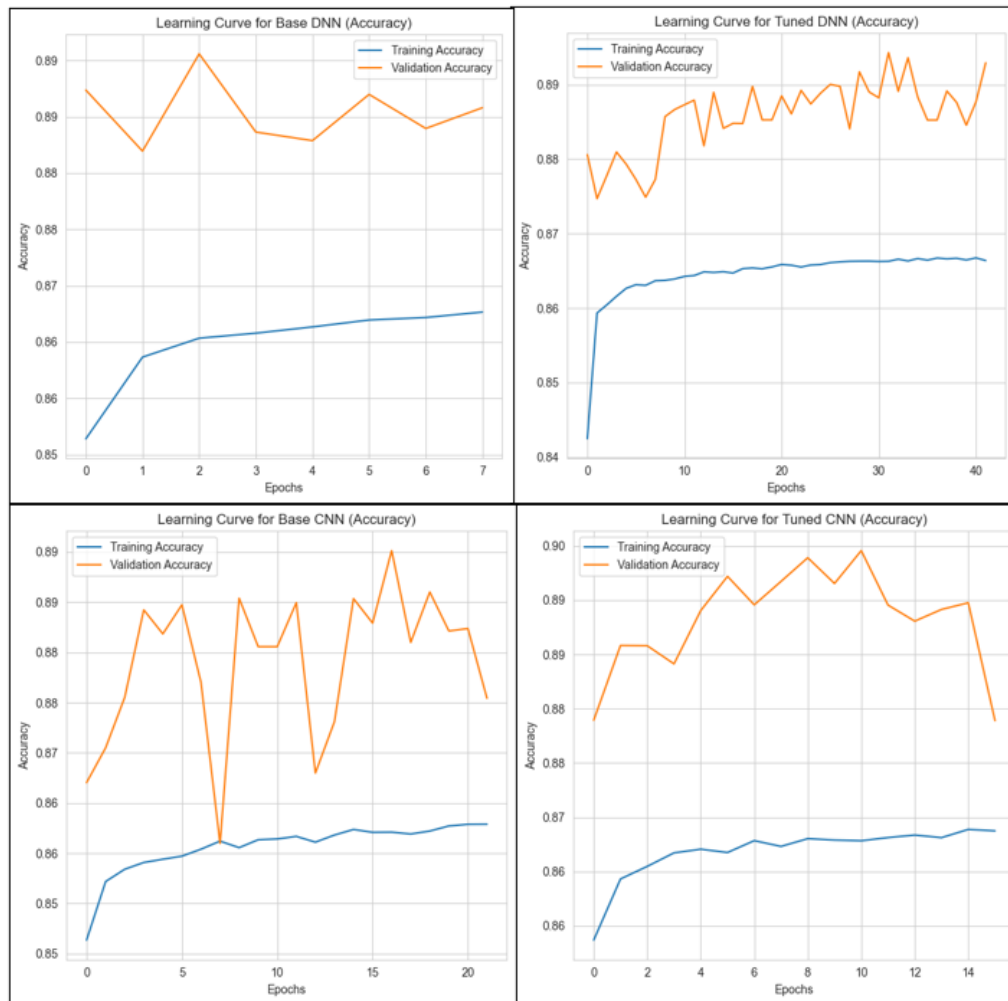


Figure 130: Comparison Learning Curve (Accuracy) for Base and Tuned Models

The learning curve for accuracy provides a clear indication of how well each model improves during training. The DNN Base Model starts with an accuracy of around 0.86 and shows a fluctuating pattern throughout the epochs. This indicates that while the model does improve over time, it does not have the stability or consistency needed for optimal performance. In contrast, the DNN Tuned Model exhibits a gradual increase in accuracy that is reflecting the positive impact of hyperparameter tuning. The tuning appears to help the model generalize better with fewer fluctuations and achieves a slightly higher accuracy than the base model.

Moving to the CNN Base Model, there was a similar behaviour to the DNN base model, where accuracy fluctuates significantly between epochs. This highlights the instability of the base CNN model. On the other hand, the CNN Tuned Model has a slightly accuracy increasing and reaching higher values compared to its base models. This steady improvement indicates that

the tuned CNN model benefits from enhanced generalization capabilities, which is important for a reliable performance in real-world predictions. Overall, the CNN Tuned Model outperforms both the base models in terms of accuracy to prove that its superior ability to correctly classify both positive and negative cases.

6.5 Comparison Learning Curve (Loss) for Base and Tuned Models

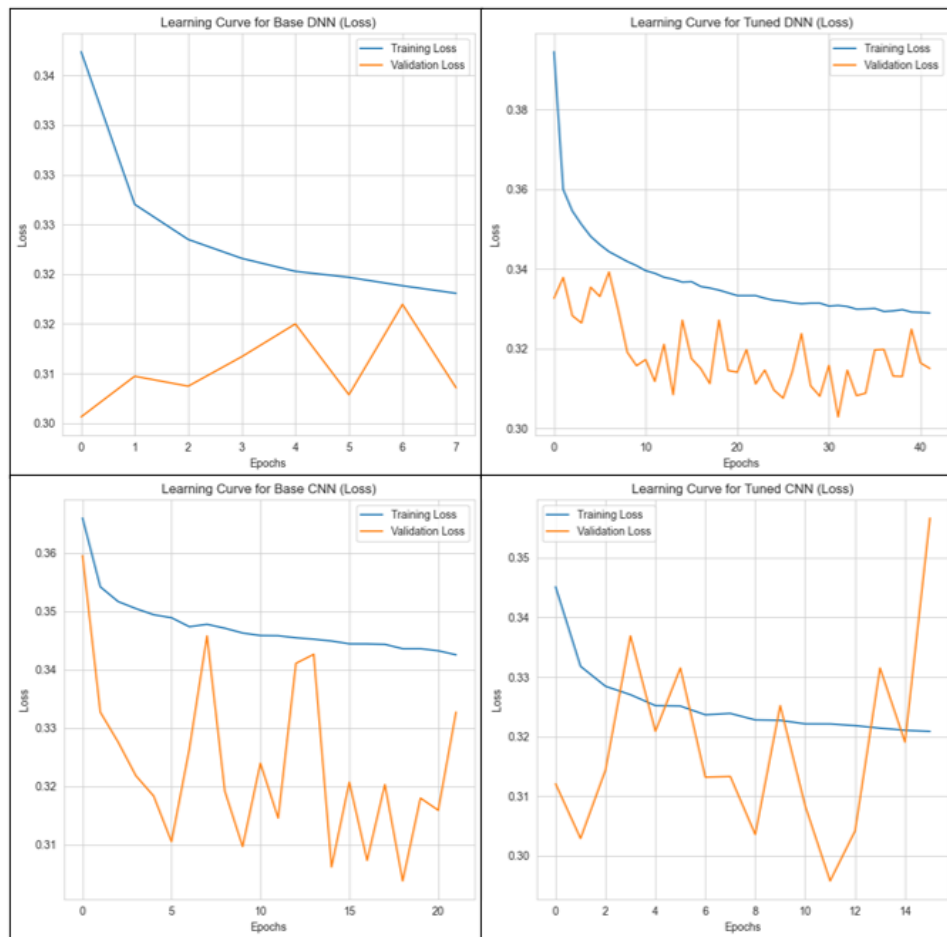


Figure 131: Comparison Learning Curve (Loss) for Base and Tuned Models

The learning curve for loss provides insights into how well each model learns and generalizes. For the DNN Base Model, the training loss steadily decreases, suggesting that the model is effectively learning from the data. However, the validation loss for the model's ability to generalize to unseen data is less consistent. For the DNN Tuned Model, it has exhibited a more stable learning process with both the training and validation loss decreasing more consistently. This indicates that the hyperparameter tuning has helped the model achieve better generalization to reduce the risk of overfitting compared to the base model.

The CNN Base Model shows a similar pattern to the DNN base, where the training loss decreases steadily but the validation loss fluctuates significantly which is suggesting the model struggles to generalize well on unseen data. On the other hand, the CNN Tuned Model demonstrates the performance with a decline in both training and validation loss. The loss curves are much more stable, reflecting the model's ability to learn effectively without overfitting. The consistency in the validation loss indicates that the CNN Tuned Model has better generalization and robustness. Therefore, in terms of loss, the CNN Tuned Model again emerges as the top performer, which has offering both lower and more stable loss values compared to the other models.

In conclusion, both the DNN Tuned Model and CNN Tuned Model perform well with hyperparameter tuning, but the CNN Tuned Model consistently performs better compared to the DNN in both accuracy and loss curves. The steady increase in accuracy and the stable decrease in loss make the CNN Tuned Model the most reliable and effective model for predicting diabetes risk.

6.6 Summary of Model Evaluation and Discussion

The evaluation results highlight the positive effects of hyperparameter tuning on both the DNN and CNN models. After tuning, both models outperformed their base versions with the CNN Tuned Model achieving a slightly higher accuracy of 0.8946 compared to the DNN Tuned Model at 0.8943. In terms of overall F1-score, the CNN Tuned Model also led with a value of 0.8213 just ahead of the DNN Tuned Model's score of 0.8212. While both models showed improved precision and recall for Class 1 (diabetic individuals), the DNN Tuned Model slightly outperformed the CNN model in recall with a value of 0.83 compared to 0.82.

Furthermore, the learning curve analysis supports the performance improvements observed in both models. The CNN Tuned Model displayed a higher increase in accuracy in both training and validation loss which is indicating better generalization and stability compared to its base version. In contrast, the DNN Tuned Model showed similar improvements but had slightly more fluctuations in the learning curves.

Overall, while the DNN Tuned Model demonstrated a better ability to identify diabetic individuals with a higher recall, the CNN Tuned Model stood out with its stronger overall performance in terms of both accuracy and F1-score as well as a more stable learning curves. Both models showed significant improvements in precision, recall and F1-score which proves that hyperparameter tuning led to better generalization and model reliability. Given the marginal advantage in overall performance, the CNN Tuned Model is the best model for predicting high-risk factors for diabetes.

7.0 Conclusion

This study focused on a deep learning models in Deep Neural Networks (DNNs) and Convolutional Neural Networks (CNNs) for predicting diabetes risk. The models were built, tuned and evaluated to determine the best approach for accurate diabetes prediction. Both models showed significant improvements after hyperparameter tuning with CNNs demonstrating a slight improve in overall performance which particularly in accuracy and F1-score. The findings highlight the importance of using advanced machine learning models in healthcare applications for diabetes prediction. It is because early detection is important to prevent complications and improve patient outcomes.

Deep learning techniques with their ability to capture complex relationships in data proved to be effective in handling imbalanced datasets. Despite the slight differences in performance between DNNs and CNNs, both models exhibited improved precision and recall for identifying diabetic individuals to make them valuable tools for clinical decision-making. The study concludes that both DNNs and CNNs are effective models for diabetes prediction. However, the CNN model with its superior feature extraction capabilities proves to be slightly more reliable for this topic which is predicting high risk diabetes.

7.1 Comparison with Literature

The results of the DNN Tuned Model and CNN Tuned Model in this study demonstrate the impact of hyperparameter tuning on model performance and their performance had surpasses compared to the similar studies in the literature.

For the accuracy, the DNN Tuned Model achieved 89.43% and the CNN Tuned Model slightly outperformed it with 89.46%. These values are consistent with the findings of Qadeer et al. (2025), where a CNN model achieved an accuracy of 75.32% though their study reported lower performance than the models in this study. The performance of both tuned models in this study is significantly higher which has reflected the effectiveness of hyperparameter tuning in improving both the DNN and CNN models for diabetes prediction.

For the F1-score, both the DNN Tuned Model (82.12%) and the CNN Tuned Model (82.13%) show notable improvements over their base models. These F1-scores are higher than the similar works study which is reported in the study by Qadeer et al. (2025), where their CNN model achieved an F1-score of 80%. The improvement in the F1-score suggests that the tuning process significantly enhanced the model's ability to balance precision and recall and makes the models

more reliable in predicting diabetes risk. Furthermore, both the DNN Tuned and CNN Tuned Models achieved better overall performance compared to previous studies, indicating that hyperparameter tuning contributed to enhanced generalization and more effective predictions in the identification of diabetic individuals. This demonstrates the continued potential of DNN and CNN models with optimized performance surpassing earlier work in diabetes prediction.

Overall, both DNN and CNN Tuned Model in this study has demonstrated a higher performance metrics compared to the literature. The DNN Tuned Model and CNN Tuned Model showed improved in accuracy and F1-scores with supporting the effectiveness of hyperparameter tuning in enhancing the models' ability to predict diabetes. While Qadeer et al. (2025) reported solid performance for their CNN model, the DNN and CNN Tuned Models in this study show a more balanced performance with slight improvements in both precision and recall. This further shows that hyperparameter tuning plays an important role in model optimization for diabetes prediction.

7.2 Limitations

Despite the promising results, several limitations should be considered in this study. The models used were trained on a single and imbalanced dataset, and this may have influenced the model's performance. While methods like SMOTE were applied to handle this, the imbalance between diabetic and non-diabetic cases remains a challenge which could potentially leading to biased predictions and suboptimal performance on the minority class. Moreover, the computational intensity of training deep learning models such as CNNs and DNNs poses a significant limitation especially in real-world applications where resources are often constrained. The time and computational power required for training these models may restrict their use in smaller healthcare settings.

Another notable limitation is the lack of model interpretability, as both CNN and DNN models are often considered "black-box" models. In healthcare, where transparency and explainability are essential, this lack of insight into the decision-making process could impede the adoption of these models by clinicians. Furthermore, the dataset used in this study lacked more detailed, and longitudinal data such as continuous monitoring of blood glucose levels. The absence of these features may limit the model's ability to provide highly personalized and timely predictions.

7.3 Recommendations

To address these limitations, future work should focus on enhancing the diversity and balance of datasets by incorporating techniques that more effectively handle class imbalance. Using more advanced sampling methods or generating synthetic data could improve model performance in predicting the minority class. Additionally, optimizing the computational efficiency of deep learning models is important. The techniques like model pruning and quantization could help reduce the computational burden for a more accessible in clinical environments. Another important area for future development is improving the interpretability of these models. By integrating explainable AI (XAI) techniques, future models could provide greater transparency for allowing healthcare professionals to better understand and trust the predictions.

By expanding the dataset to include more comprehensive and granular data such as real-time glucose monitoring or genetic information would also help refine predictions to individual patients. By combining different types of health data, the models could achieve a more holistic understanding of diabetes risk. This helps in improving both accuracy and the potential for personalized treatment plans. Through these improvements, future models could significantly advance diabetes prediction and management which helps in leading to better patient outcomes and more efficient healthcare systems.

8.0 References

- Abousaber, I., Abdallah, H. F., & El-Ghaish, H. (2025). Robust predictive framework for diabetes classification using optimized machine learning on imbalanced datasets. *Frontiers in Artificial Intelligence*, 7. <https://doi.org/10.3389/frai.2024.1499530>
- Alfian, G., Syafrudin, M., Fitriyani, N. L., Anshari, M., Stasa, P., Svub, J., & Rhee, J. (2020). Deep Neural Network for Predicting Diabetic Retinopathy from Risk Factors. *Mathematics*, 8(9), 1620. <https://doi.org/10.3390/math8091620>
- Behavioral Risk Factor Surveillance System (BRFSS) - Healthy People 2030* | [odphp.health.gov](https://odphp.health.gov/healthypeople/objectives-and-data/data-sources-and-methods/data-sources/behavioral-risk-factor-surveillance-system-brfss). (n.d.). <https://odphp.health.gov/healthypeople/objectives-and-data/data-sources-and-methods/data-sources/behavioral-risk-factor-surveillance-system-brfss>
- Bontha, S. S., Jammalamadaka, S. K. R., Vudatha, C. P., Jammalamadaka, S. B., Duvvuri, B. K., & Vudatha, B. C. (2025). Predicting risk and complications of diabetes through Built-In Artificial Intelligence. *Computers*, 14(7), 277. <https://doi.org/10.3390/computers14070277>
- DCHS-CHS—National Center for Health Statistics--5300. (2023). *Collecting data*. Washington State Department of Health. <https://doh.wa.gov/data-and-statistical-reports/data-systems/behavioral-risk-factor-surveillance-system-brfss/brfss-collecting-data>
- Diabetes Atlas. (2025, June 17). IDF Diabetes Atlas | Global Diabetes Data & Statistics. <https://diabetesatlas.org/>
- El-Bashbishy, A. E., & El-Bakry, H. M. (2024). Pediatric diabetes prediction using deep learning. *Scientific Reports*, 14(1). <https://doi.org/10.1038/s41598-024-51438-4>
- Fallucchi, F., & Cabroni, A. (2021). Predicting Risk of Diabetes using a Model based on Multilayer Perceptron and Features Extraction. *Journal of Computer Science*, 17(9), 748–761. <https://doi.org/10.3844/jcssp.2021.748.761>
- García-Mosquera, J., Villa-Monedero, M., Gil-Martín, M., & San-Segundo, R. (2025). Transformer-Based prediction of hospital readmissions for diabetes patients. *Electronics*, 14(1), 174. <https://doi.org/10.3390/electronics14010174>

- Gong, W., Pu, Y., Ning, T., Zhu, Y., Mu, G., & Li, J. (2025). Deep learning for enhanced prediction of diabetic retinopathy: a comparative study on the diabetes complications data set. *Frontiers in Medicine*, 12. <https://doi.org/10.3389/fmed.2025.1591832>
- Haidarh, M., Mu, C., Liu, Y., & He, X. (2025). Exploring Traditional, Deep Learning and Hybrid Methods for Hyperspectral Image Classification: A Review. *Journal of Information and Intelligence*. <https://doi.org/10.1016/j.jiixd.2025.04.002>
- Hammarlund, N., Jo, A., Walker, A. F., Haller, M. J., Maahs, D. M., Churba, B., & Jacobs, M. (2025). Evaluating the Economic Impact of Project ECHO Diabetes: Cost Savings from HbA1c Reduction in Type 1 and Type 2 Diabetes. *Endocrine Practice*. <https://doi.org/10.1016/j.eprac.2025.06.025>
- Hassija, V., Chamola, V., Mahapatra, A., Singal, A., Goel, D., Huang, K., ... & Hussain, A. (2024). Interpreting black-box models: a review on explainable artificial intelligence. *Cognitive Computation*, 16(1), 45-74. <https://link.springer.com/article/10.1007/s12559-023-10179-8>
- International Diabetes Federation. (2024). *IDF Diabetes Atlas 11th edition*. https://diabetesatlas.org/media/uploads/sites/3/2025/04/IDF_Atlas_11th_Edition_2025_Global-Factsheet.pdf
- Ismail, W. N., Hassan, M. M., Alsalamah, H. A., & Fortino, G. (2020). CNN-based health model for regular health factors analysis in internet-of-medical things environment. *IEEE Access*, 8, 52541-52549. <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=9036869>
- Joel, L. O., Doorsamy, W., & Paul, B. S. (2022). A review of missing data handling techniques for machine learning. *International Journal of Innovative Technology and Interdisciplinary Sciences*, 5(3), 971-1005. <https://doi.org/10.1515/IJITIS.2022.5.3.971-1005>
- Khokhar, P. B., Gravino, C., & Palomba, F. (2025). Advances in artificial intelligence for diabetes prediction: insights from a systematic literature review. *Artificial intelligence in medicine*, 103132. <https://doi.org/10.1016/j.artmed.2025.103132>

- Li, Y., Rao, S., Solares, J. R. A., Hassaine, A., Ramakrishnan, R., Canoy, D., & Salimi-Khorshidi, G. (2020). BEHRT: transformer for electronic health records. *Scientific reports*, 10(1), 7155. <https://www.nature.com/articles/s41598-020-62922-y.pdf>
- Madan, P., Singh, V., Chaudhari, V., Albagory, Y., Dumka, A., Singh, R., Gehlot, A., Rashid, M., Alshamrani, S. S., & AlGhamdi, A. S. (2022). An Optimization-Based diabetes prediction model using CNN and Bi-Directional LSTM in Real-Time environment. *Applied Sciences*, 12(8), 3989. <https://doi.org/10.3390/app12083989>
- Mall, P. K., Singh, P. K., Srivastav, S., Narayan, V., Paprzycki, M., Jaworska, T., & Ganzha, M. (2023). A comprehensive review of deep neural networks for medical image processing: Recent developments and future opportunities. *Healthcare Analytics*, 4, 100216. <https://doi.org/10.1016/j.health.2023.100216>
- Prasetyo, S. Y., & Izdihar, Z. N. (2024, July). Multi-layer perceptron approach for diabetes risk prediction using BRFSS data. In *2024 IEEE 10th International Conference on Smart Instrumentation, Measurement and Applications (ICSIMA)* (pp. 303-308). IEEE. <https://ieeexplore.ieee.org/abstract/document/10675535>
- Qadeer, T., Ali, N. I., Brohi, I. A., & Jamali, A. (2025, April 11). *Enhancing Early Diabetes Detection A Comparative Analysis Of Machine Learning And Deep Learning Techniques*. <https://www.sesjournal.com/index.php/1/article/view/253>
- Sonia, J. J., Jayachandran, P., Quadir, A., MD, Mohan, S., Sivaraman, A. K., & Tee, K. F. (2023). Machine-Learning-Based diabetes mellitus risk prediction using Multi-Layer Neural Network No-PROP algorithm. *Diagnostics*, 13(4), 723. <https://doi.org/10.3390/diagnostics13040723>
- Ullah, Z., Saleem, F., Jamjoom, M., Fakieh, B., Kateb, F., Ali, A. M., & Shah, B. (2022). Detecting High-Risk factors and early diagnosis of diabetes using machine learning methods. *Computational Intelligence and Neuroscience*, 2022, 1–10. <https://doi.org/10.1155/2022/2557795>
- Understanding Diabetes: the importance of early detection and management*. (2024, October 27). UChicago Medicine AdventHealth. <https://www.uchicagomedicineadventhealth.org/blog/understanding-diabetes-importance-early-detection-and-management>

Zhu, T., Kuang, L., Piao, C., Zeng, J., Li, K., & Georgiou, P. (2024). Population-specific glucose prediction in diabetes care with transformer-based deep learning on the edge. *IEEE transactions on biomedical circuits and systems*, 18(2), 236-246.
<https://ieeexplore.ieee.org/document/10379014>